DEVOPS ENGINEER COURSE

Linux for DevOps Engineers

The operating system behind everything you deploy

Beginner | Intermediate | Advanced

Concepts · hands-on labs · troubleshooting · interview Q&A · cheat sheet

Contents

Part 1 — Foundations 5

1.1 What Linux Is 5
1.2 Distributions You Will Meet 6
1.3 The Boot Process 7
1.4 The Filesystem Hierarchy 10
1.5 Getting Help 11
1.6 Lab: Explore a Fresh Linux System 11
Key takeaways 12

Part 2 — The Shell, Files and Text Processing 13

2.1 Shell Basics 13
2.2 Streams, Pipes and Redirection 16
2.3 Finding Things 17
2.4 Text Processing: The DevOps Toolkit 17
2.5 Editors: vim and nano 20
2.6 Lab: Log Analysis with Pipelines 21
Key takeaways 22

Part 3 — Users, Permissions and Processes 23

3.1 Users and Groups 23
3.2 File Permissions 25
3.3 Processes 27
3.4 Lab: Shared Team Directory and Process Control 30
Key takeaways 31

Part 4 — systemd, Packages, Scheduling and Logs 32

4.1 systemd in Depth 32
4.2 journald and journalctl 35
4.3 rsyslog and logrotate 36
4.4 Package Management 37
4.5 Scheduling: cron and systemd Timers 40
4.6 Lab: Deploy a Supervised Service with a Timer 41
Key takeaways 42

Part 5 — Disks, Filesystems and Storage 44

5.1 The Storage Stack 44
5.2 Discovering Block Devices 44
5.3 Partitioning 45
5.4 Filesystems 45
5.5 Mounting and fstab 46
5.6 LVM: Logical Volume Manager 48
5.7 RAID Basics 49
5.8 Disk Usage: df and du 49
5.9 Checking and Repairing Filesystems 50
5.10 Lab: Build, Grow and Break Storage Safely 51

Key takeaways 52

Part 6 — Networking and SSH 53

6.1 Network Fundamentals in Five Minutes 53
6.2 Interfaces, Addresses and Routes with ip 53
6.3 Sockets and Ports with ss 55
6.4 DNS on Linux 55
6.5 Testing Connectivity 56
6.6 Firewalls 57
6.7 SSH Deep Dive 58
6.8 Lab: Network Inspection, Firewall and SSH Tunnels 62
Key takeaways 63

Part 7 — Bash Scripting: Basics to Production Quality 64

7.1 The First Script 64
7.2 Variables and Expansion 64
7.3 Conditions 66
7.4 Loops 67
7.5 Functions 67
7.6 Arrays 68
7.7 Strict Mode and Error Handling 68
7.8 Argument Parsing with getopts 70
7.9 Useful Idioms 70
7.10 Real DevOps Scripts 71
7.11 When Not to Use Bash 74
7.12 Lab: Write a Production-Grade Health Check Script 74
Key takeaways 75

Part 8 — Performance, Troubleshooting Tools and Kernel Tuning 76

8.1 A Method: USE 76
8.2 CPU and Load 77
8.3 Memory 78
8.4 Disk I/O 80
8.5 Processes, Files and System Calls 80
8.6 Kernel Tuning with sysctl 81
8.7 Resource Limits: ulimits 82
8.8 Lab: Create and Diagnose Bottlenecks 83

Part 9 — Security Hardening, Containers Under the Hood, Cloud and Automation 84

9.1 Security Hardening 84
9.2 Namespaces and cgroups: the Bridge to Containers 87
9.3 Linux in the Cloud 90
9.4 Automation Hand-off: Ansible 92
9.5 Lab: Harden, Containerize by Hand, and Automate 95
Key takeaways 96

Part 10 — Real-World Troubleshooting, Interview Prep and Cheat Sheet 97

10.1 Scenario: Disk Full 97
10.2 Scenario: High CPU / High Load 98

10.3 Scenario: Service Won’t Start 99
10.4 Scenario: Can’t SSH to a Server 100
10.5 Scenario: DNS Failing 102
10.6 Scenario: Server Doesn’t Boot After Changes (Bonus) 103
10.7 Lab: Break-Fix Game 103
10.8 Interview Questions and Answers 104
10.9 Command Cheat Sheet 108

Part 1 — Foundations

This course takes you from “I can open a terminal” to the level expected of a senior DevOps or SRE engineer: someone who can build, automate, secure, tune and debug Linux systems under pressure. Every Part explains *why* things work the way they do, because the commands change between distros and versions, while the underlying model stays the same.

How to use this course:

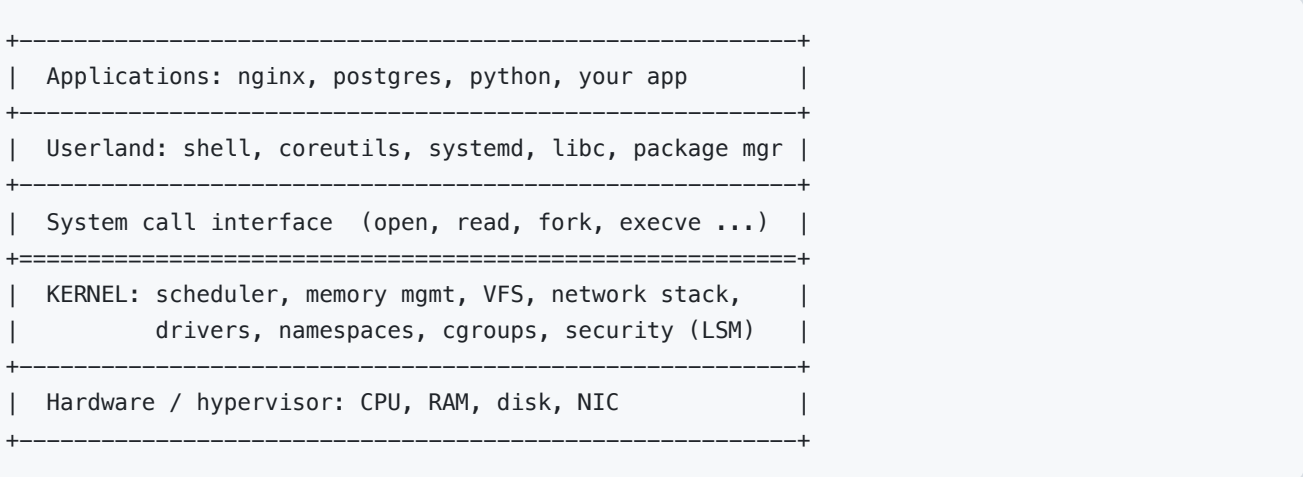
- Read each Part in order; later Parts assume the earlier ones.
- Do every Lab on a disposable machine: a local VM (Multipass, Vagrant, UTM, VirtualBox), a cloud instance, or a container where noted. Never practise destructive commands on a machine you care about.
- Type the commands rather than pasting them. Muscle memory is the point.

1.1 What Linux Is

Strictly speaking, **Linux is a kernel**: the program that runs with full hardware privileges, schedules processes, manages memory, talks to devices through drivers, and exposes all of this to programs through **system calls** (`open`, `read`, `write`, `fork`, `execve`, `mmap`, and a few hundred more). Linus Torvalds released it in 1991, and it is licensed under the GPLv2.

What most people call “Linux” is a **distribution**: the kernel plus a userland, which typically includes:

- An init system and service manager (almost always **systemd** today)
- Core utilities (`ls`, `cp`, `cat`, usually GNU coreutils, or BusyBox on Alpine)
- A C library (glibc on most distros, musl on Alpine)
- A shell (bash, dash, zsh)
- A package manager and repositories
- Default configuration, security policy and a release/support lifecycle



1.1.1 Kernel space vs user space

The CPU has privilege levels. The kernel runs in **kernel space** (ring 0 on x86) and can touch any memory or device. Your programs run in **user space** and must ask the kernel for anything privileged. That boundary is why:

- A buggy program crashes itself, not the machine.
- `strace` can show you everything a program asks the kernel for (Part 8).
- Containers are possible: the kernel can present different "views" of the system to different processes (Part 9).

1.1.2 Why DevOps runs on Linux

- The overwhelming majority of cloud workloads, all Kubernetes nodes in practice, and nearly every container image are Linux.
- Everything is scriptable and text-configured, so it can be version-controlled and automated.
- Containers are just Linux kernel features (namespaces + cgroups) with good tooling.
- It is open: when something is weird, you can read the source, the docs, and the kernel's own counters in `/proc` and `/sys`.

1.1.3 The "everything is a file" idea

Unix design exposes most resources as files or file-like objects: regular files, directories, devices (`/dev/sda`), kernel state (`/proc/meminfo`), sockets and pipes. One set of tools (`cat`, `echo`, redirection) works across all of them:

```
cat /proc/loadavg              # kernel load statistics
cat /sys/class/net/eth0/mtu     # NIC MTU
echo 1 | sudo tee /proc/sys/net/ipv4/ip_forward   # change a kernel setting
```

1.2 Distributions You Will Meet

Distributions differ mainly in **package format, release cadence, default security tooling, and support lifecycle**. The kernel and most tools are the same.

Family	Examples	Package tools	Format	Typical use
Debian	Debian, Ubuntu (LTS)	`apt`, `dpkg`	`.deb`	Cloud VMs, dev machines, many container bases
Red Hat	RHEL, Rocky Linux, AlmaLinux, CentOS Stream, Fedora	`dnf` (`yum` alias), `rpm`	`.rpm`	Enterprise servers, regulated environments
Amazon	Amazon Linux 2023	`dnf`	`.rpm`	AWS EC2, Fedora-derived
Alpine	Alpine Linux	`apk`	`.apk`	Tiny container images (musl + BusyBox)

Family	Examples	Package tools	Format	Typical use
SUSE	SLES, openSUSE	`zypper`	`.rpm`	Enterprise, SAP

Key facts worth knowing:

- **Ubuntu LTS** ships every two years (22.04, 24.04, 26.04) with 5 years of standard support, more with Ubuntu Pro. Uses AppArmor and `ufw` by default.
- **RHEL** has ~10-year lifecycles. **Rocky** and **Alma** are free, binary-compatible rebuilds. **CentOS Stream** is upstream of RHEL, not a rebuild. RHEL-family uses SELinux (enforcing) and `firewalld` by default.
- **Amazon Linux 2023 (AL2023)** is based on Fedora, uses `dnf`, SELinux in permissive mode by default, and is the default choice on AWS. Amazon Linux 2 (yum-based) reached end of support in 2026.
- **Alpine** images are about 5–8 MB. The catch: **musl libc** behaves differently from glibc (DNS resolution quirks, Python wheels that need recompiling, different `ls` / `ps` flags in BusyBox).

To identify any system:

```
cat /etc/os-release
uname -r           # kernel release
hostnamectl        # systemd summary (not on Alpine)
```

```
PRETTY_NAME="Ubuntu 24.04.1 LTS"
NAME="Ubuntu"
VERSION_ID="24.04"
ID=ubuntu
ID_LIKE=debian
```

> **Tip:** In scripts, source `/etc/os-release` and branch on `$ID` or `$ID_LIKE` rather than parsing `uname` or guessing from file existence.

```
. /etc/os-release
case "$ID $ID_LIKE" in
  *debian*) PKG="apt-get" ;;
  *rhel*|*fedora*) PKG="dnf" ;;
  *alpine*) PKG="apk" ;;
esac
```

1.3 The Boot Process

Knowing the boot chain lets you fix machines that don't come up, and explains things like "why did my disk mount fail at boot" or "why is the instance stuck in emergency mode".

```
Power on
   |
   v
Firmware (BIOS or UEFI) -- POST, finds boot device
   |
   v
Bootloader (GRUB 2 / systemd-boot) -- loads kernel + initramfs
   |
   v
Kernel -- decompresses, inits CPU/memory/drivers, mounts initramfs
   |
   v
initramfs (/init) -- loads storage drivers, unlocks LUKS, assembles
                     LVM/RAID, finds and mounts the real root fs
   |
   v
switch_root -> /sbin/init = systemd (PID 1)
   |
   v
systemd -- reaches default.target (multi-user or graphical)
   |
   v
Login prompt / sshd / your services
```

1.3.1 Firmware: BIOS vs UEFI

- **BIOS (legacy)**: reads the first 512 bytes of the boot disk (the MBR), which contains a tiny first-stage bootloader. MBR partitioning supports max 4 primary partitions and 2 TiB disks.
- **UEFI**: modern firmware that understands filesystems. It reads an **EFI System Partition (ESP)**, a small FAT32 partition usually mounted at `/boot/efi`, and runs a `.efi` binary (e.g. `shimx64.efi` → `grubx64.efi`). Pairs with **GPT** partitioning. Supports **Secure Boot**, where each stage verifies the signature of the next.

```
[ -d /sys/firmware/efi ] && echo UEFI || echo BIOS
efibootmgr -v    # list UEFI boot entries (UEFI only)
```

1.3.2 GRUB 2

GRUB shows a menu, lets you choose a kernel, and passes the **kernel command line**. Configuration is *generated*: you edit `/etc/default/grub` and regenerate.

```
# /etc/default/grub (excerpt)
GRUB_TIMEOUT=5
GRUB_CMDLINE_LINUX="console=ttyS0 crashkernel=auto"

# Regenerate
sudo update-grub                                  # Debian/Ubuntu
```

```
sudo grub2-mkconfig -o /boot/grub2/grub.cfg        # RHEL family (BIOS & UEFI on RHEL 9+)
sudo grubby --update-kernel=ALL --args="audit=1"  # RHEL-family convenience tool
```

See what the running kernel was booted with:

```
cat /proc/cmdline
```

```
BOOT_IMAGE=/vmlinuz-6.8.0-45-generic root=UUID=3f1c... ro quiet splash
```

> **Tip:** At the GRUB menu press `e` to edit an entry for one boot only. Adding `systemd.unit=rescue.target` or `rd.break` (RHEL, breaks inside initramfs) is how you recover a lost root password or a broken fstab.

1.3.3 Kernel and initramfs

The kernel image lives at `/boot/vmlinuz-<version>`. The kernel cannot mount the root filesystem if the driver for it (NVMe, virtio, LVM, RAID, encryption) is a module stored *on* that root filesystem. The **initramfs** (`/boot/initrd.img-*` or `/boot/initramfs-*.img`) solves this chicken-and-egg problem: it's a small compressed cpio archive the bootloader loads into RAM, containing just enough tools and modules to find and mount the real root.

Rebuild it after changing storage drivers, crypttab, or certain kernel modules:

```
sudo update-initramfs -u           # Debian/Ubuntu
sudo dracut -f                     # RHEL family, Fedora, AL2023
lsinitramfs /boot/initrd.img-$(uname -r) | head   # inspect (Debian)
lsinitrd | head                                   # inspect (dracut)
```

1.3.4 systemd as PID 1

Once root is mounted, the kernel executes `/sbin/init`, which is a symlink to systemd. PID 1 is special: it adopts orphaned processes, and if it dies the kernel panics. systemd reads unit files, builds a dependency graph, and starts everything in parallel towards a **target**.

Target	Old runlevel	Meaning
`poweroff.target`	0	Shut down
`rescue.target`	1	Single-user, root fs mounted, minimal services
`multi-user.target`	3	Full system, no GUI (servers)
`graphical.target`	5	multi-user + display manager
`reboot.target`	6	Reboot
`emergency.target`	-	Bare minimum shell, root mounted read-only

```
systemctl get-default
sudo systemctl set-default multi-user.target
systemd-analyze                    # total boot time
systemd-analyze blame | head       # slowest units
systemd-analyze critical-chain     # what the boot actually waited for
```

```
Startup finished in 2.112s (kernel) + 6.904s (userspace) = 9.016s
multi-user.target reached after 6.851s in userspace.
```

Interview: "Walk me through what happens when a Linux server boots." Answer with the chain: firmware (UEFI/BIOS) → bootloader (GRUB) loads kernel and initramfs and passes the command line → kernel initializes hardware and runs initramfs → initramfs mounts the real root → `switch_root` and exec systemd as PID 1 → systemd activates units up to `default.target`. Bonus points for mentioning where you'd intervene when each stage fails.

1.4 The Filesystem Hierarchy

Linux has a single tree rooted at `/`. Other disks are **mounted** onto directories in that tree; there are no drive letters. The layout follows the **Filesystem Hierarchy Standard (FHS)**.

Path	Purpose	DevOps notes
`/`	Root of the tree	
`/bin`, `/sbin`, `/lib`	Essential binaries/libs	On modern distros these are symlinks to `/usr/...` ("usrmerge")
`/usr`	Installed software, read-only-ish	`/usr/bin`, `/usr/lib`, `/usr/share`
`/usr/local`	Locally built software	Where `make install` goes by default
`/opt`	Self-contained third-party apps	Vendor software, `/opt/app/`
`/etc`	System configuration	Text files; version this with Ansible/Git
`/var`	Variable data	`/var/log`, `/var/lib` (databases, docker), `/var/cache`
`/var/log`	Logs	First place to look when something breaks
`/var/lib`	Persistent state of services	`/var/lib/docker`, `/var/lib/postgresql`
`/tmp`	Temporary files	Often tmpfs (RAM); cleared at reboot
`/home`	User homes	`/root` is root's home
`/boot`	Kernels, initramfs, GRUB	Running out of space here breaks kernel updates

Path	Purpose	DevOps notes
`/dev`	Device nodes	Managed by udev (`/dev/sda`, `/dev/nvme0n1`, `/dev/null`)
`/proc`	Virtual: processes and kernel state	`/proc/<pid>/`, `/proc/meminfo`
`/sys`	Virtual: devices, drivers, kernel objects	cgroups live under `/sys/fs/cgroup`
`/run`	Runtime state since boot (tmpfs)	PID files, sockets
`/mnt`, `/media`	Manual / removable mounts	
`/srv`	Data served by this system	Sometimes used for web roots

> **Note:** `/proc` and `/sys` consume no disk space. They are windows into kernel data structures, generated on read. That's why `cat /proc/meminfo` is always current.

Understanding this layout tells you where to look: config in `/etc`, logs in `/var/log` or the journal, data in `/var/lib`, binaries in `/usr/bin`, runtime sockets in `/run`.

```
man hier          # the hierarchy documented on your system
findmnt           # what is mounted where, as a tree
```

1.5 Getting Help

Professional Linux users read docs constantly. Learn to do it fast.

```
man ls              # manual page; '/' to search, 'n' next, 'q' quit
man 5 crontab       # section 5 = file formats (vs crontab(1) the command)
man -k "disk usage" # search man page descriptions (apropos)
ls --help           # short usage
help cd             # bash builtins
type cd ls ll       # is it a builtin, alias, function or file?
tldr tar            # community examples (install 'tldr')
```

Man page sections you'll use: **1** user commands, **5** file formats, **7** overviews (`man 7 signal`, `man 7 capabilities`), **8** admin commands.

1.6 Lab: Explore a Fresh Linux System

Goal: get oriented on a new machine the way you would on day one of an incident.

1. Start a VM. With Multipass: `multipass launch 24.04 --name lab1 && multipass shell lab1`. Or launch an Ubuntu/Rocky cloud instance and SSH in.
2. Identify the system:

```
bash cat /etc/os-release uname -a hostnamectl
```

3. Determine the firmware type and boot details:

```
bash [ -d /sys/firmware/efi ] && echo UEFI || echo BIOS cat /proc/cmdline ls -lh /boot
```

4. Measure the boot:

```
bash systemd-analyze systemd-analyze blame | head -5 systemctl get-default
```

5. Walk the hierarchy. For each, write down one thing you find:

```
bash ls /etc | head -30 ls /var/log ls -l /bin # is it a symlink to usr/bin? findmnt -t ext4,xfs,vfat
```

6. Read kernel state directly:

```
bash cat /proc/cpuinfo | grep "model name" | head -1 grep -E 'MemTotal|MemAvailable' /proc/meminfo cat /proc/uptime /proc/loadavg
```

7. Look up two man pages: `man 5 os-release` and `man 7 hier`. Find the section that explains `ID_LIKE`.
8. Stretch: reboot the VM, interrupt GRUB (hold Shift on BIOS, Esc on UEFI, or use the cloud serial console), press `e`, and look at the kernel line without changing it. Press Esc to boot normally.

Key takeaways

- Linux is the kernel; a distribution is the kernel plus userland, packaging and policy.
- Debian/Ubuntu (apt), RHEL/Rocky/Alma/Amazon Linux (dnf), and Alpine (apk, musl) are the families you'll meet. Use `/etc/os-release` to detect them.
- Boot chain: firmware → GRUB → kernel + initramfs → systemd (PID 1) → `default.target`. Each stage has its own failure mode and recovery hook.
- The initramfs exists so the kernel can mount a root filesystem whose drivers it doesn't have built in.
- `/etc` is config, `/var` is changing data, `/usr` is software, `/proc` and `/sys` are live kernel views.
- `man`, `--help`, `type`, and `man -k` are the fastest way to answer your own questions.

Part 2 — The Shell, Files and Text Processing

The shell is your primary interface to every Linux machine you'll ever operate. On a server there is no GUI; in a container there may not even be an editor. Fluency here is the single biggest productivity multiplier in this course.

2.1 Shell Basics

A **shell** is a program that reads commands, expands them (variables, globs, substitutions), and asks the kernel to run programs. **bash** is the default interactive shell on most distros; `/bin/sh` is often **dash** (Debian/Ubuntu) or BusyBox `ash` (Alpine), which are smaller POSIX shells. That matters for scripts: a script with `#!/bin/sh` must not use bash-only features.

A **terminal** (or terminal emulator, or SSH session) is just the window that carries text to and from the shell.

2.1.1 Anatomy of a command

```
 sudo   ls    -l -a   --color=auto   /var/log   > out.txt
 |      |     |       |              |          |
prefix command short   long option   argument   redirection
              options
```

- Short options can be combined: `ls -la` equals `ls -l -a`.
- `--` ends options: `rm -- -weird-filename`.
- The shell does the expansion *before* the program runs. `ls *.log` never sees `*`; it receives the list of matching filenames.

2.1.2 Navigation

```
pwd                 # print working directory
cd /var/log         # absolute path
cd ..               # parent
cd -                # previous directory
cd                  # home (~)
ls -lah             # long, all (hidden), human sizes
ls -lt | head       # newest first
ls -ld /etc         # the directory itself, not its contents
tree -L 2 /etc/ssh  # if installed
```

Relative paths are resolved from the current directory; absolute paths start with `/`. `~` expands to `$HOME`, `~alice` to Alice's home.

2.1.3 Working with files and directories

```
touch app.log                 # create empty / update mtime
mkdir -p project/{src,tests,docs}   # -p creates parents; braces expand
cp -a src/ backup/            # -a = archive: recursive, keep perms, times, links
mv old.txt new.txt            # rename or move
rm file; rm -r dir            # remove (no recycle bin!)
rm -rf -- "$dir"              # quote variables; never rm -rf "$UNSET/"
ln -s /opt/app/current/bin/app /usr/local/bin/app   # symlink
stat file                     # inode, perms, times
file mystery.bin              # identify content type
```

> **Warning:** There is no undo for `rm`. In scripts, guard with `${dir:?}` (fails if empty) so `rm -rf "${dir:?}/"` can never become `rm -rf /`.

Hard links vs symlinks. A filename is a directory entry pointing to an **inode** (the object holding metadata and data block pointers). A hard link is a second name for the same inode (same filesystem only, not for directories). A symlink is a tiny file containing a path; it can cross filesystems and can dangle if the target is removed.

```
ln data.txt hard.txt; ln -s data.txt soft.txt
ls -li data.txt hard.txt soft.txt
```

```
1311 -rw-r--r-- 2 abdul abdul 12 Oct 12 10:00 data.txt
1311 -rw-r--r-- 2 abdul abdul 12 Oct 12 10:00 hard.txt
1312 lrwxrwxrwx 1 abdul abdul  8 Oct 12 10:00 soft.txt -> data.txt
```

Note the same inode number (1311) and link count 2.

2.1.4 Viewing files

```
cat file                # whole file
less /var/log/syslog    # pager: / search, G end, g start, F follow, q quit
head -n 20 file; tail -n 50 file
tail -f /var/log/nginx/access.log   # follow appended lines
tail -F app.log         # follow by name, survives log rotation
wc -l file              # count lines
```

2.1.5 Globbing, quoting and expansion

Pattern	Matches
`*`	Any string (not leading `.`)
`?`	Any single character

Pattern	Matches
`[abc]`, `[0-9]`	One character from the set
`{a,b}`	Brace expansion (generates words, not a match)
`**`	Recursive (bash with `shopt -s globstar`)

Quoting decides what the shell expands:

```
name="world"
echo "Hello $name"     # double quotes: variables expanded -> Hello world
echo 'Hello $name'     # single quotes: literal -> Hello $name
echo "Today is $(date +%F)"   # command substitution
echo $((3 * 4))        # arithmetic -> 12
```

> **Tip:** Quote every variable expansion (`"$file"`) unless you specifically want word-splitting. Unquoted variables containing spaces or globs are the number one source of shell bugs.

2.1.6 History and productivity

```
history | tail
!!            # repeat last command (sudo !! is classic)
!$            # last argument of previous command
Ctrl-r        # reverse search history
Ctrl-a / Ctrl-e   # start / end of line
Ctrl-w / Ctrl-u   # delete word / to start of line
Ctrl-l        # clear screen
alias ll='ls -lah'      # put in ~/.bashrc
```

Startup files: login shells read `/etc/profile` and `~/.bash_profile` (or `~/.profile`); interactive non-login shells read `~/.bashrc`. Most distros make `~/.bash_profile` source `~/.bashrc`, so put aliases and functions in `~/.bashrc`, and environment variables in `~/.profile` or `~/.bash_profile`.

2.1.7 Environment variables and PATH

```
echo "$PATH"
export EDITOR=vim          # export: visible to child processes
env | sort | head
which -a python3; command -v python3   # command -v is POSIX, use in scripts
```

`PATH` is searched left to right for commands. A process inherits a *copy* of its parent's environment; a child can never change the parent's environment, which is why `cd` must be a shell builtin and why you `source` a file (rather than execute it) to set variables in your current shell.

2.2 Streams, Pipes and Redirection

Every process starts with three open file descriptors:

FD	Name	Default
0	stdin	keyboard / terminal
1	stdout	terminal
2	stderr	terminal

```
cmd > out.txt          # stdout to file (truncate)
cmd >> out.txt         # append
cmd 2> err.txt         # stderr to file
cmd > all.txt 2>&1     # both to file (order matters!)
cmd &> all.txt         # bash shorthand for the same
cmd 2>/dev/null        # discard errors
cmd < input.txt        # stdin from file
cmd1 | cmd2            # stdout of cmd1 -> stdin of cmd2
cmd 2>&1 | less        # pipe both streams
cmd | tee out.txt      # write to file AND stdout
echo "x" | sudo tee /etc/protected > /dev/null   # write as root
```

Why does `sudo echo x > /etc/file` fail? The redirection is performed by *your* shell (unprivileged) before `sudo` runs. `tee` is run by sudo, so it opens the file as root.

Why does `2>&1 > file` not work? Redirections are processed left to right: `2>&1` first points stderr at the *current* stdout (the terminal), then stdout moves to the file.

Here documents and here strings:

```
cat > /etc/myapp.conf <<'EOF'
listen = 0.0.0.0:8080
log_level = info
EOF

grep -c error <<< "$log_text"
```

Quoting the delimiter (`'EOF'`) disables variable expansion inside the heredoc, which is what you usually want for config files containing `$` .

Process substitution (bash) lets a command's output appear as a file:

```
diff <(ssh web1 cat /etc/nginx/nginx.conf) <(ssh web2 cat /etc/nginx/nginx.conf)
```

Exit codes. Every command returns 0 for success and 1–255 for failure. `$?` holds the last one. `&&` runs the next command only on success; `||` only on failure.

```
mkdir -p /opt/app && cd /opt/app || exit 1
```

2.3 Finding Things

2.3.1 find

`find` walks the directory tree and evaluates tests on each entry.

```
find /var/log -name "*.log" -mtime -1           # modified in last 24h
find / -xdev -type f -size +500M 2>/dev/null     # big files, stay on one fs
find . -type f -name "*.tmp" -delete
find /etc -type f -newer /tmp/marker             # changed since marker
find /srv -type d -perm -o+w                     # world-writable dirs
find /home -user alice -iname "*.PDF"
find . -name "*.py" -exec grep -l "TODO" {} +    # {} + batches args (fast)
find /data -mindepth 1 -maxdepth 1 -mtime +30 -print   # preview before delete
```

`-exec cmd {} \;` runs one process per file; `-exec cmd {} +` passes many files per invocation, like xargs.

2.3.2 xargs

`xargs` turns stdin into arguments. Always pair `find -print0` with `xargs -0` so filenames with spaces or newlines work.

```
find . -name "*.log" -print0 | xargs -0 gzip
cat hosts.txt | xargs -I{} -P 8 ssh {} uptime     # 8 in parallel
kubectl get pods -o name | xargs -n1 kubectl delete   # one arg per run
```

2.3.3 locate, which, whereis

`locate` (from `plocate` / `mlocate`) queries a prebuilt database: fast but may be stale (`sudo updatedb`). `which` and `command -v` find executables in PATH.

2.4 Text Processing: The DevOps Toolkit

Logs, configs, CSV exports, command output: it's all text. These tools compose through pipes.

2.4.1 grep

```
grep "ERROR" app.log
grep -i "timeout" app.log          # case-insensitive
grep -v "healthcheck" access.log   # invert
grep -n -C 3 "panic" app.log       # line numbers, 3 lines context
```

```
grep -r "db_host" /etc/myapp/      # recursive
grep -rl "password" .              # list matching files only
grep -c " 500 " access.log         # count matching lines
grep -E "5[0-9]{2} " access.log    # extended regex
grep -o -E "[0-9]{1,3}(\.[0-9]{1,3}){3}" access.log | sort -u   # extract IPs
grep -F "a.b[c]" file              # fixed string, no regex
grep -q "^ok$" status && echo healthy   # quiet: only exit code
```

`ripgrep` (`rg`) is a faster, gitignore-aware alternative worth installing on workstations.

2.4.2 cut, sort, uniq, wc, tr

```
cut -d: -f1,7 /etc/passwd          # fields 1 and 7, ':' delimiter
cut -c1-10 file                    # characters
sort -n                            # numeric
sort -k3,3nr                       # by 3rd field, numeric, reverse
sort -t, -k2 data.csv              # comma-separated
sort -u                            # sort + unique
uniq -c                            # count adjacent duplicates (sort first!)
tr 'a-z' 'A-Z'                     # translate characters
tr -d '\r'                         # strip Windows CR
tr -s ' '                          # squeeze repeated spaces
paste -sd+ nums.txt | bc           # sum a column
column -t                          # align into a table
```

The canonical “top N” pipeline:

```
# Top 10 client IPs in an nginx access log
awk '{print $1}' /var/log/nginx/access.log | sort | uniq -c | sort -rn | head
```

```
   4821 203.0.113.42
   1210 198.51.100.7
    388 192.0.2.15
```

2.4.3 sed

`sed` is a stream editor: it applies edits to each line.

```
sed 's/foo/bar/' file              # first match per line
sed 's/foo/bar/g' file             # all matches
sed -n '10,20p' file               # print lines 10-20
sed '/^#/d; /^$/d' nginx.conf      # drop comments and blank lines
sed -i 's/^#\?PermitRootLogin.*/PermitRootLogin no/' /etc/ssh/sshd_config
sed -i.bak 's|/old/path|/new/path|g' app.conf   # keep a backup; '|' delimiter
sed -E 's/([0-9]+)ms/\1 ms/g' file  # extended regex with capture group
```

> **Warning:** `sed -i` differs between GNU (Linux) and BSD (macOS). On macOS, `sed -i '' 's/a/b/' file`. On Linux, `sed -i 's/a/b/' file`. Using `-i.bak` works on both.

2.4.4 awk

`awk` is a small language for field-based text. It splits each line into `$1`, `$2`, … (`$0` is the whole line, `NF` the number of fields, `NR` the line number).

```
awk '{print $1, $9}' access.log                # IP and status code
awk -F: '$3 >= 1000 {print $1}' /etc/passwd    # human users
awk '$9 >= 500' access.log                     # filter: 5xx responses
awk '{sum += $10} END {print sum/1024/1024 " MB"}' access.log   # total bytes
awk '{c[$9]++} END {for (s in c) print s, c[s]}' access.log | sort  # count by status
df -h | awk 'NR>1 && $5+0 > 80 {print $6, $5}'  # filesystems over 80%
ps aux | awk '{mem[$1]+=$6} END {for (u in mem) print u, mem[u]/1024 " MB"}'
```

```
200 48211
301 1022
404 377
502 41
```

Rule of thumb: `grep` to filter lines, `cut` for simple delimiters, `awk` for fields with logic or aggregation, `sed` for substitutions. Past ~10 lines of awk, switch to Python.

2.4.5 jq: JSON on the command line

Cloud CLIs, Kubernetes and APIs speak JSON. `jq` is mandatory.

```
echo '{"name":"web","ports":[80,443],"tags":{"env":"prod"}}' | jq .
jq -r '.name'                    # raw string, no quotes -> web
jq '.ports[0]'                   # -> 80
jq '.tags.env'
jq -r '.items[].metadata.name'   # iterate arrays
jq '.items | length'
jq '.[] | select(.status == "running") | .id'
jq -r '.[] | [.name, .ip] | @tsv' # tab-separated output
jq '{name, env: .tags.env}'      # build a new object
jq --arg v "$VERSION" '.version = $v' package.json > tmp && mv tmp package.json
```

Real examples:

```
# All running EC2 instance IDs and private IPs
aws ec2 describe-instances \
  --filters Name=instance-state-name,Values=running \
  | jq -r '.Reservations[].Instances[] | [.InstanceId, .PrivateIpAddress] | @tsv'

# Pods not in Running phase
```

```
kubectl get pods -A -o json \
  | jq -r '.items[] | select(.status.phase != "Running")
           | "\(.metadata.namespace)/\(.metadata.name) \(.status.phase)"'
```

For YAML, `yq` (the Go version by Mike Farah) uses nearly identical syntax.

2.4.6 Other tools worth knowing

```
diff -u old.conf new.conf          # unified diff
comm -23 <(sort a) <(sort b)       # lines only in a
tar -czf backup.tgz /etc           # create gzip tarball
tar -tzf backup.tgz | head         # list
tar -xzf backup.tgz -C /restore    # extract into dir
tar --zstd -cf backup.tar.zst dir  # zstd: faster, better ratio
sha256sum file; sha256sum -c SHA256SUMS   # verify downloads
base64 -d <<< "aGVsbG8="            # decode -> hello
watch -n 2 'ss -s'                 # rerun every 2s
```

2.5 Editors: vim and nano

You will edit files over SSH on boxes with nothing but `vi`. Learn enough vim to survive; use nano when it's there and you're in a hurry.

2.5.1 nano

Commands are shown on screen: `Ctrl-O` save, `Ctrl-X` exit, `Ctrl-W` search, `Alt-U` undo.

2.5.2 vim survival guide

vim is **modal**. Normal mode is for moving and commanding; Insert mode is for typing.

Action	Keys
Enter insert mode	`i` (before cursor), `a` (after), `o` (new line below)
Back to normal mode	`Esc`
Save / quit / both / quit without saving	`:w` / `:q` / `:wq` or `ZZ` / `:q!`
Move	`h j k l`, `w` / `b` words, `0` / `$` line start/end
Top / bottom / line 42	`gg` / `G` / `42G` or `:42`
Delete line / word / char	`dd` / `dw` / `x`
Copy (yank) line / paste	`yy` / `p`
Undo / redo	`u` / `Ctrl-r`

Action	Keys
Search / next	`/pattern` / `n`
Replace in file	`:%s/old/new/g`
Visual select	`v` (chars), `V` (lines)
Show line numbers	`:set number`
Paste without auto-indent mess	`:set paste`

> **Tip:** `sudoedit /etc/file` (or `sudo -e`) edits a temporary copy with your own editor settings and writes it back as root. It's safer than running a whole editor as root.

Set your default editor for `crontab -e`, `visudo`, `git commit`, and `systemctl edit`:

```
echo 'export EDITOR=vim VISUAL=vim' >> ~/.bashrc
sudo update-alternatives --config editor   # Debian/Ubuntu system default
```

2.6 Lab: Log Analysis with Pipelines

Goal: answer real operational questions about a web server log using only shell tools.

1. Generate a sample access log (works anywhere with bash):

`bash mkdir -p ~/lab2 && cd ~/lab2 for i in $(seq 1 5000); do ip="10.0.$((RANDOM % 5)).$((RANDOM % 20))" codes=(200 200 200 200 301 404 500 502) code=${codes[$((RANDOM % 8))]} paths=(/ /api/users /api/orders /login /health) path=${paths[$((RANDOM % 5))]} bytes=$((RANDOM % 5000)) ms=$((RANDOM % 900)) echo "$ip - - [12/Oct/2026:10:$((RANDOM%60)):00 +0000] \"GET $path HTTP/1.1\" $code $bytes ${ms}ms" done > access.log wc -l access.log`

2. Top 5 client IPs:

`bash awk '{print $1}' access.log | sort | uniq -c | sort -rn | head -5`

3. Count requests per status code, as a table:

`bash awk '{print $9}' access.log | sort | uniq -c | sort -rn | column -t`

4. Error rate (percentage of 5xx):

`bash awk '$9 >= 500 {e++} END {printf "%.2f%%\n", e/NR*100}' access.log`

5. Which path produces most 5xx errors?

`bash awk '$9 >= 500 {print $7}' access.log | sort | uniq -c | sort -rn`

6. Average latency per path (strip the `ms` suffix):

```bash
awk '{gsub("ms","",$11); s[$7]+=$11; n[$7]++} END {for (p in s) printf "%-14s %6.1f ms\n", p, s[p]/n[p]}' access.log | sort -k2 -n
```

7. Exclude health checks, and save slow requests (>800 ms) to a file while also viewing them:

```bash
grep -v "/health" access.log | awk '{v=$11; sub("ms","",v); if (v+0 > 800) print}' | tee slow.log | wc -l
```

8. Use sed to anonymize the last octet of every IP, writing a new file:

```bash
sed -E 's/^([0-9]+\.[0-9]+\.[0-9]+)\.[0-9]+/\1.xxx/' access.log > anon.log head -3 anon.log
```

9. jq practice: convert the status counts to JSON and query it:

```bash
awk '{c[$9]++} END {printf "{"; for (k in c) printf "%s\"%s\":%d", (n++?",":""), k, c[k]; print "}"}' access.log > counts.json jq . counts.json jq 'to_entries | map(select(.key|tonumber >= 500)) | map(.value) | add' counts.json
```

10. Find and compress every `.log` file in `~/lab2` safely, keeping the originals (in real life you would add `-mtime +7` to target only old logs):

    ```bash
    find ~/lab2 -name "*.log" -print0 | xargs -0 -r gzip -k ls -lh ~/lab2
    ```

Key takeaways

- The shell expands globs, variables and substitutions *before* the program runs; quote variables to control that.
- stdin/stdout/stderr are file descriptors 0/1/2; redirections are processed left to right, and `sudo` does not apply to your shell's redirections (use `tee`).
- `find ... -print0 | xargs -0` is the safe way to act on many files.
- grep filters, cut slices, sort/uniq count, sed substitutes, awk computes, jq handles JSON. Chaining them answers most operational questions in one line.
- Know enough vim to edit, save and quit on any server; use `sudoedit` for root-owned files.

Part 3 — Users, Permissions and Processes

Linux is a multi-user system, and almost every security control and every "permission denied" you'll ever debug comes down to three questions: *who* is this process running as, *what* does the file allow, and *which* extra mechanism (ACL, capability, SELinux) applies. This Part also covers processes, the running instances of programs, and how to control them.

3.1 Users and Groups

3.1.1 Identities are numbers

The kernel doesn't know names, only numeric **UIDs** and **GIDs**. Names are mapped in files:

File	Contents
`/etc/passwd`	`name:x:UID:GID:comment:home:shell` (world-readable, no passwords)
`/etc/shadow`	Password hashes and aging (root-only)
`/etc/group`	`group:x:GID:member1,member2`
`/etc/gshadow`	Group passwords/admins (rarely used)

```
grep alice /etc/passwd
getent passwd alice     # also queries LDAP/SSSD, not just files
id alice
```

```
alice:x:1001:1001:Alice Ng:/home/alice:/bin/bash
uid=1001(alice) gid=1001(alice) groups=1001(alice),27(sudo),998(docker)
```

UID ranges by convention: **0** is root, **1–999** are system accounts (services), **1000+** are humans. Service accounts typically have shell `/usr/sbin/nologin`.

> **Note:** Because the kernel compares numbers, a file created by UID 1001 in a container appears as owned by whoever is UID 1001 on the host. UID mismatches are a common cause of permission errors on bind mounts and NFS.

3.1.2 Managing users and groups

```
sudo useradd -m -s /bin/bash -c "Alice Ng" alice   # -m creates home
sudo passwd alice
```

```
sudo useradd -r -s /usr/sbin/nologin -d /var/lib/myapp myapp   # system/service user
sudo usermod -aG docker alice        # -a APPEND to supplementary groups (never omit -a!)
sudo usermod -L alice                # lock password
sudo chage -l alice                  # password aging
sudo chage -E 2026-12-31 contractor  # account expiry
sudo userdel -r olduser              # remove with home
sudo groupadd devops
sudo gpasswd -d alice docker         # remove from a group
```

On Debian/Ubuntu, `adduser` is a friendlier interactive wrapper. Group membership changes apply at the next login (or `newgrp docker` in the current shell), because group lists are attached to processes at login time.

> **Warning:** `usermod -G docker alice` without `-a` *replaces* all supplementary groups, potentially removing `sudo` access. Always use `-aG`.

3.1.3 root, su and sudo

root (UID 0) bypasses normal permission checks. Logging in as root directly is discouraged because there's no audit trail of *who* did what and one mistake is unlimited.

- `su -` switches to root (needs root's password); `su - alice` becomes Alice with a login environment.
- `sudo cmd` runs one command as root (needs *your* password), logged to the journal/`/var/log/auth.log` or `/var/log/secure`.
- `sudo -i` gives a root login shell; `sudo -u postgres psql` runs as another user.

sudo policy lives in `/etc/sudoers` and drop-ins in `/etc/sudoers.d/`. **Always edit with `visudo`**, which syntax-checks before saving; a broken sudoers file can lock everyone out.

```
sudo visudo -f /etc/sudoers.d/devops
```

```
# Members of devops get full sudo
%devops ALL=(ALL:ALL) ALL

# Deploy user may only restart the app, without a password
deploy ALL=(root) NOPASSWD: /usr/bin/systemctl restart myapp.service, /usr/bin/systemctl
status myapp.service

# Keep a variable through sudo
Defaults env_keep += "HTTP_PROXY HTTPS_PROXY NO_PROXY"
```

Syntax: `who host=(runas_user:runas_group) [NOPASSWD:] commands`. Ubuntu grants sudo to the `sudo` group, RHEL-family to `wheel`.

```
sudo -l              # what am I allowed to run?
```

```
sudo visudo -c       # check all sudoers files
```

Interview: "Why is `NOPASSWD: /usr/bin/vim` dangerous?" Because vim can run shell commands (`:!bash`), giving a full root shell. Same for `less`, `find -exec`, `tar`, `python`, and many others (see GTFOBins). Grant narrowly scoped commands only.

3.2 File Permissions

3.2.1 Reading `ls -l`

```
-rwxr-x---  1 alice devops  4096 Oct 12 10:00 deploy.sh
| |  |  |      |     |
| |  |  |      owner group
| |  |  other (o)
| |  group (g)
| user/owner (u)
file type: - file, d dir, l symlink, c char dev, b block dev, s socket, p pipe
```

Bit	On a file	On a directory
`r` (4)	Read contents	List names (`ls`)
`w` (2)	Modify contents	Create, delete, rename entries inside
`x` (1)	Execute	Enter/traverse (`cd`, access files by path)

Two consequences people get wrong:

- Deleting a file requires `w` on the **directory**, not on the file. A read-only file in a writable directory can be deleted.
- To read `/a/b/c.txt` you need `x` on `/a` and `/a/b`, plus `r` on the file. One missing `x` anywhere in the path gives "Permission denied". Use `namei -l /a/b/c.txt` to see permissions along the whole path.

The kernel checks owner first, then group, then other, and uses the **first category that matches**, not the most permissive. If you own a file with mode `0077`, you can't read it even though "other" can.

3.2.2 chmod, chown, chgrp

```
chmod 640 app.conf            # rw-r-----
chmod 750 deploy.sh           # rwxr-x---
chmod u+x,g-w,o= script.sh    # symbolic
chmod -R g+rX /srv/shared     # capital X: execute only on dirs (and already-executable
files)
sudo chown alice:devops file
```

```
sudo chown -R www-data: /var/www/site    # trailing ':' = user's login group
sudo chgrp devops file
```

Common modes: `644` files, `755` dirs and scripts, `600` private keys and secrets, `700` `~/.ssh`, `640` configs readable by a service group.

3.2.3 umask

New files get permissions `requested & ~umask`. Programs typically request `666` for files and `777` for directories. With the default umask `022`, files become `644` and directories `755`; with `027`, `640` and `750`.

```
umask           # 0022
umask 027       # current shell only
```

Set it per service with `UMask=0027` in a systemd unit, or system-wide in `/etc/login.defs` / shell profiles.

3.2.4 Special bits: SUID, SGID, sticky

Bit	Octal	On a file	On a directory
SUID	4000	Runs with the **file owner's** UID	(no effect)
SGID	2000	Runs with the file's group	New files inherit the directory's **group**
Sticky	1000	(no effect)	Only the file owner (or root) can delete entries

```
ls -l /usr/bin/passwd   # -rwsr-xr-x root root  (s = SUID; lets users update /etc/shadow)
ls -ld /tmp             # drwxrwxrwt (t = sticky; you can't delete others' files)
sudo chmod 2775 /srv/shared && sudo chgrp devops /srv/shared   # team directory
find / -xdev -perm -4000 -type f 2>/dev/null     # audit SUID binaries
```

SGID directories are the standard way to build shared team folders: combined with umask `002`, everyone's files land in the team group and stay group-writable.

> **Warning:** Every SUID-root binary is a privilege-escalation candidate. Audit the list regularly; an unexpected SUID file is a classic sign of compromise. Shell scripts ignore SUID on Linux for safety.

3.2.5 ACLs

Standard permissions allow exactly one owner and one group. **POSIX ACLs** add per-user and per-group entries.

```
setfacl -m u:bob:rw report.csv          # give bob rw
setfacl -m g:auditors:r -R /var/log/app # group read, recursive
setfacl -d -m g:devops:rwX /srv/shared  # DEFAULT ACL: inherited by new files
getfacl report.csv
```

```
setfacl -x u:bob report.csv            # remove entry
setfacl -b report.csv                  # remove all ACLs
```

```
# file: report.csv
# owner: alice
# group: alice
user::rw-
user:bob:rw-
group::r--
mask::rw-
other::---
```

A `+` at the end of the mode in `ls -l` (`-rw-rw----+`) indicates an ACL is present. The **mask** caps the effective permissions of all named users and groups; `chmod g=...` on an ACL'd file changes the mask, which surprises people.

3.2.6 Capabilities and file attributes

Root's power is split into **capabilities** (`man 7 capabilities`). Instead of running a web server as root just to bind port 80, grant only that capability:

```
sudo setcap 'cap_net_bind_service=+ep' /usr/local/bin/myserver
getcap /usr/local/bin/myserver
```

In systemd: `AmbientCapabilities=CAP_NET_BIND_SERVICE`. Containers use capabilities heavily (`docker run --cap-drop ALL --cap-add NET_BIND_SERVICE`).

Extended attributes like **immutable** prevent even root from modifying a file until removed:

```
sudo chattr +i /etc/resolv.conf    # cannot be changed or deleted
lsattr /etc/resolv.conf
sudo chattr -i /etc/resolv.conf
```

3.3 Processes

A **process** is a running program: an address space, open file descriptors, credentials (UID/GID), a current directory, environment, and one or more threads. Each has a **PID** and a parent (**PPID**).

3.3.1 How processes are created

```
bash (PID 1200)
  |  fork()  -> child is a copy of bash (PID 1201)
  |  execve("/usr/bin/ls") in child -> PID 1201 now runs ls
  |  wait() -> parent collects child's exit status
```

- When a child exits, it becomes a **zombie** (`Z` state) until its parent calls `wait()`. Zombies hold no memory, just a process table entry. Many zombies mean a buggy parent; killing the zombie does nothing, you must fix or kill the parent.
- If the parent dies first, the child is an **orphan** and is re-parented to PID 1 (or a subreaper), which reaps it. This is why containers need a proper init (`tini`, `docker run --init`) as PID 1.

3.3.2 Viewing processes

```
ps aux                      # BSD style: all processes, user-oriented
ps -ef                      # System V style, shows PPID
ps -eo pid,ppid,user,%cpu,%mem,stat,etime,cmd --sort=-%cpu | head
ps -fp 1234                 # one process
pstree -p                   # tree with PIDs
pgrep -a nginx              # find by name
top / htop                  # live view (Part 8)
ls -l /proc/1234/           # everything about PID 1234
cat /proc/1234/status       # state, memory, threads, capabilities
ls -l /proc/1234/fd         # open files
tr '\0' '\n' < /proc/1234/environ   # its environment (root/owner only)
```

Process states (the `STAT` column):

State	Meaning
`R`	Running or runnable (on CPU or in run queue)
`S`	Interruptible sleep (waiting for event, e.g. network)
`D`	Uninterruptible sleep (usually waiting on disk/NFS I/O; can't be killed)
`T`	Stopped (Ctrl-Z or SIGSTOP)
`Z`	Zombie
`I`	Idle kernel thread

Many processes in `D` state usually means a storage or NFS problem, and they count towards load average (Part 8).

3.3.3 Signals

Signals are asynchronous notifications sent to processes (`man 7 signal`).

Signal	Number	Default	Typical use
`SIGHUP`	1	Terminate	Terminal closed; many daemons reload config on it
`SIGINT`	2	Terminate	Ctrl-C
`SIGQUIT`	3	Core dump	Ctrl-\

Signal	Number	Default	Typical use
`SIGKILL`	9	Terminate	Cannot be caught or ignored
`SIGTERM`	15	Terminate	Polite “please exit”; default for `kill`
`SIGSTOP`	19	Stop	Cannot be caught
`SIGCONT`	18	Continue	Resume stopped process
`SIGUSR1/2`	10/12	Terminate	App-defined (e.g. reopen logs)
`SIGCHLD`	17	Ignore	Child changed state

```
kill 1234                # SIGTERM
kill -HUP $(pidof nginx)  # reload (nginx master)
kill -9 1234             # last resort
pkill -f "python worker.py"
killall -USR1 myapp
```

> **Tip:** Always try `SIGTERM` first and wait. `SIGKILL` gives the process no chance to flush buffers, release locks, or remove temp files. Kubernetes and systemd both send SIGTERM, wait a grace period, then SIGKILL; your apps must handle SIGTERM to shut down gracefully.

3.3.4 Job control

```
long_task            # running in foreground
Ctrl-Z               # stop it -> [1]+ Stopped
bg                   # continue it in the background
jobs -l              # list jobs in this shell
fg %1                # bring job 1 back
sleep 600 &          # start in background
disown -h %1         # don't send SIGHUP to it when the shell exits
nohup ./backup.sh > backup.log 2>&1 &   # immune to hangup
```

For anything long-running over SSH, use **tmux** (or screen): the session lives on the server and survives disconnects.

```
tmux new -s deploy     # start
# Ctrl-b d to detach; later:
tmux attach -t deploy
tmux ls
```

For real services, don’t use `nohup` at all: write a systemd unit (Part 4).

3.3.5 Priority: nice and ionice

```
nice -n 10 ./batch-job.sh      # lower priority (19 lowest, -20 highest)
renice -n 5 -p 1234
ionice -c3 -p 1234             # idle I/O class: only uses disk when idle
ionice -c2 -n7 tar czf ...     # best-effort, lowest
```

Only root can increase priority (negative nice values).

3.4 Lab: Shared Team Directory and Process Control

Goal: build a correctly permissioned shared directory, grant narrow sudo, and practise signals.

1. Create users and a group:

`bash sudo groupadd devops for u in alice bob carol; do sudo useradd -m -s /bin/bash "$u"; done sudo usermod -aG devops alice sudo usermod -aG devops bob id alice; id carol`

2. Create a SGID team directory:

`bash sudo mkdir -p /srv/devops sudo chgrp devops /srv/devops sudo chmod 2770 /srv/devops ls -ld /srv/devops # drwxrws---`

3. Test it. Alice creates a file; check the group; Carol is denied:

`bash sudo -u alice bash -c 'umask 007; echo hi > /srv/devops/plan.txt' ls -l /srv/devops/plan.txt # group should be devops sudo -u bob cat /srv/devops/plan.txt sudo -u carol cat /srv/devops/plan.txt # Permission denied`

4. Give Carol read access with an ACL, and make it default for new files:

`bash sudo setfacl -m u:carol:rx /srv/devops sudo setfacl -m u:carol:r /srv/devops/plan.txt sudo setfacl -d -m u:carol:r /srv/devops sudo -u carol cat /srv/devops/plan.txt getfacl /srv/devops`

5. Add a sticky bit so members can't delete each other's files, then verify:

`bash sudo chmod +t /srv/devops sudo -u bob rm /srv/devops/plan.txt # Operation not permitted`

6. Grant Bob permission to run only `systemctl status`:

`bash echo 'bob ALL=(root) NOPASSWD: /usr/bin/systemctl status *' | sudo tee /etc/sudoers.d/bob sudo chmod 440 /etc/sudoers.d/bob sudo visudo -c sudo -u bob sudo -l`

7. Audit SUID binaries and record the list as a baseline:

`bash sudo find / -xdev -perm -4000 -type f 2>/dev/null | sort | tee ~/suid-baseline.txt`

8. Signals and job control:

`bash sleep 1000 & jobs -l kill -STOP %1; ps -o pid,stat,cmd -p $! # state T kill -CONT %1; ps -o pid,stat,cmd -p $! # state S kill %1`

9. Create a zombie to see one (Python is fine):

`bash python3 -c 'import os,time if os.fork()==0: os._exit(0) time.sleep(60)' & sleep 1; ps -eo pid,ppid,stat,cmd | grep -w Z`

10. Clean up: `sudo userdel -r carol` (etc.), `sudo rm -rf /srv/devops /etc/sudoers.d/bob`.

Key takeaways

- The kernel sees only UIDs/GIDs; `getent` and `id` show how names map to them.
- Use `usermod -aG`, `visudo`, and narrowly scoped sudo rules. Some commands (editors, pagers, interpreters) grant a full root shell.
- Directory `w` controls deletion; directory `x` controls traversal; `namei -l` diagnoses path permission problems.
- umask removes bits from new files; SGID directories plus a group give you team folders; sticky protects shared directories.
- ACLs and capabilities offer finer control than owner/group/other and all-or-nothing root.
- Processes are created by fork/exec; zombies need their parent to reap them; orphans get adopted by PID 1.
- Send SIGTERM first, SIGKILL last. Use tmux for interactive long jobs and systemd for services.

Part 4 — systemd, Packages, Scheduling and Logs

This Part covers the operational core of a Linux server: how services are started and supervised (systemd), how software is installed and patched (package managers), how work is scheduled (cron and timers), and where evidence goes when things break (logs).

4.1 systemd in Depth

systemd is PID 1 and the service manager on every mainstream distro except Alpine (which uses OpenRC) and minimal containers. It replaced shell-script SysV init because it offers dependency-based parallel startup, process supervision with automatic restarts, cgroup-based resource tracking (a service's child processes can't "escape"), socket activation, and a unified log (the journal).

4.1.1 Units

Everything systemd manages is a **unit**, defined by a file. The suffix is the type:

Type	Purpose
`.service`	A daemon or one-shot task
`.socket`	A listening socket that starts a service on demand
`.timer`	Scheduled activation of another unit (cron replacement)
`.target`	A named group / synchronization point (like runlevels)
`.mount`, `.automount`	Filesystem mounts (generated from fstab too)
`.path`	Start a unit when a file/directory changes
`.slice`	A cgroup node for resource control
`.device`, `.swap`, `.scope`	Kernel devices, swap, externally created process groups

Unit file locations, in order of precedence:

Path	Owner
`/etc/systemd/system/`	Administrator (you). Highest precedence
`/run/systemd/system/`	Runtime, generated
`/usr/lib/systemd/system/` (`/lib/...` on Debian)	Packages. Don't edit these; upgrades overwrite them

4.1.2 systemctl essentials

```
systemctl status nginx              # state, PID, cgroup, recent logs
sudo systemctl start|stop|restart nginx
sudo systemctl reload nginx         # re-read config without restart (if supported)
sudo systemctl enable --now nginx   # start at boot AND now
sudo systemctl disable nginx
sudo systemctl mask nginx           # make it impossible to start (symlink to /dev/null)
systemctl is-active nginx; systemctl is-enabled nginx; systemctl is-failed nginx
systemctl list-units --type=service --state=running
systemctl --failed                  # what's broken right now
systemctl list-unit-files --state=enabled
systemctl cat nginx                 # show the unit file(s) in effect
systemctl show nginx -p MainPID,Restart,MemoryCurrent
systemctl list-dependencies multi-user.target
sudo systemctl daemon-reload        # after editing unit files
```

```
* nginx.service - A high performance web server and a reverse proxy server
     Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
     Active: active (running) since Mon 2026-10-12 09:14:02 UTC; 2h 3min ago
       Docs: man:nginx(8)
   Main PID: 812 (nginx)
      Tasks: 3 (limit: 4558)
     Memory: 5.1M (peak: 6.0M)
        CPU: 147ms
     CGroup: /system.slice/nginx.service
             |-812 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
             `-813 "nginx: worker process"
```

`enable` means “start at boot” (it creates a symlink in a target’s `.wants/` directory). `start` means “start now”. They are independent, which is why `enable --now` exists.

4.1.3 Anatomy of a service unit

```
# /etc/systemd/system/myapp.service
[Unit]
Description=My API service
Documentation=https://git.example.com/myapp
After=network-online.target postgresql.service
Wants=network-online.target
Requires=postgresql.service

[Service]
Type=simple
User=myapp
Group=myapp
WorkingDirectory=/opt/myapp
EnvironmentFile=-/etc/myapp/env
ExecStartPre=/opt/myapp/bin/check-config
ExecStart=/opt/myapp/bin/server --port 8080
```

```
ExecReload=/bin/kill -HUP $MAINPID
Restart=on-failure
RestartSec=5
TimeoutStopSec=30
LimitNOFILE=65536

[Install]
WantedBy=multi-user.target
```

Key directives explained:

- **`After=` vs `Requires=` / `Wants=`**: ordering and dependency are separate. `Wants=` pulls a unit in (soft); `Requires=` pulls it in and stops us if it stops (hard); `After=` only orders. You usually need both `Wants=` and `After=`.
- **`network-online.target`**: "network is configured". `network.target` only means the network stack is up, which isn't enough for apps that connect elsewhere at startup.
- **`Type=`**:
- `simple` (default): the `ExecStart` process *is* the service. Don't let it daemonize.
- `exec`: like simple, but startup counts as successful only once the binary has been executed.
- `forking`: traditional daemons that fork and exit; pair with `PIDFile=`.
- `oneshot`: runs to completion (setup scripts); often with `RemainAfterExit=yes`.
- `notify`: the app tells systemd it's ready via `sd_notify()`.
- **`Restart=`**: `no`, `on-failure` (non-zero exit, signal, timeout), `always`. Combined with `RestartSec=`, and rate-limited by `StartLimitIntervalSec=` / `StartLimitBurst=` in `[Unit]`.
- **`EnvironmentFile=-/path`**: the `-` means "ignore if missing".
- **`[Install] WantedBy=`**: what `enable` hooks into.

4.1.4 Overriding vendor units safely

Never edit files in `/usr/lib/systemd/system`. Use a drop-in:

```
sudo systemctl edit nginx
```

This opens `/etc/systemd/system/nginx.service.d/override.conf`:

```
[Service]
LimitNOFILE=200000
Restart=always
```

To *replace* a list-type setting like `ExecStart`, clear it first:

```
[Service]
ExecStart=
ExecStart=/usr/sbin/nginx -g 'daemon off;' -c /etc/nginx/custom.conf
```

`systemctl edit` runs `daemon-reload` for you. `systemd-delta` lists all overrides on the system, which is handy when inheriting a mystery server.

4.1.5 Hardening and resource controls

systemd can sandbox a service with a few lines, often more effectively than you'd achieve manually:

```
[Service]
NoNewPrivileges=yes
ProtectSystem=strict          # /usr, /etc read-only
ProtectHome=yes
PrivateTmp=yes
ReadWritePaths=/var/lib/myapp /var/log/myapp
StateDirectory=myapp          # creates /var/lib/myapp owned by User=
CapabilityBoundingSet=CAP_NET_BIND_SERVICE
AmbientCapabilities=CAP_NET_BIND_SERVICE
MemoryMax=512M
CPUQuota=50%
TasksMax=200
```

```
systemd-analyze security myapp.service   # scores exposure, lists suggestions
systemd-cgtop                            # live resource use per unit
```

DynamicUser=yes goes further, allocating a transient UID for the service's lifetime.

4.1.6 Debugging a unit that won't start

```
systemctl status myapp -l --no-pager
journalctl -u myapp -b --no-pager -n 100
systemd-analyze verify /etc/systemd/system/myapp.service
sudo -u myapp /opt/myapp/bin/server --port 8080   # run by hand as the service user
```

Typical causes: wrong path in `ExecStart` (must be absolute), missing execute bit, the user can't read config, port in use, SELinux denial, `Type=forking` mismatch, or the start-limit being hit (`systemctl reset-failed myapp` clears it). Part 10 walks through this scenario end to end.

4.2 journald and journalctl

systemd-journald collects stdout/stderr of all services, kernel messages, and syslog messages into a structured, indexed binary journal.

```
journalctl -u nginx                  # one unit
journalctl -u nginx -f               # follow
journalctl -u nginx --since "1 hour ago"
journalctl --since "2026-10-12 09:00" --until "2026-10-12 10:00"
journalctl -b                        # this boot; -b -1 previous boot
journalctl -k                        # kernel ring buffer (like dmesg)
journalctl -p err -b                 # priority err and worse
journalctl _PID=1234
journalctl _SYSTEMD_UNIT=sshd.service + _COMM=sudo   # OR of matches
```

```
journalctl -o json-pretty -n 1       # see all structured fields
journalctl -o cat -u myapp           # message only
journalctl --list-boots
journalctl --disk-usage
sudo journalctl --vacuum-size=500M   # shrink
sudo journalctl --vacuum-time=14d
```

Persistence: if `/var/log/journal/` exists (or `Storage=persistent` is set), the journal survives reboots; otherwise it lives in `/run/log/journal` and is lost. Ubuntu and RHEL 9+ default to persistent. Configure limits in `/etc/systemd/journald.conf`:

```
[Journal]
Storage=persistent
SystemMaxUse=1G
MaxRetentionSec=1month
```

> **Tip:** `journalctl -b -1 -p warning` after an unexpected reboot shows warnings from the boot that crashed. If nothing is there, persistence is probably off.

4.3 rsyslog and logrotate

4.3.1 rsyslog

Many systems still run **rsyslog** alongside journald, writing classic text files: `/var/log/syslog` and `/var/log/auth.log` (Debian/Ubuntu) or `/var/log/messages` and `/var/log/secure` (RHEL). It's also the usual way to forward logs to a central server.

Syslog messages have a **facility** (auth, cron, daemon, kern, local0–7…) and a **severity** (emerg, alert, crit, err, warning, notice, info, debug).

```
# /etc/rsyslog.d/50-myapp.conf
# Write local0 messages to their own file
local0.*    /var/log/myapp.log

# Forward everything to a central collector over TCP with a disk-assisted queue
# (legacy one-line form: *.* @@logs.example.com:514  where @@ = TCP, @ = UDP)
*.*  action(type="omfwd" target="logs.example.com" port="514" protocol="tcp"
            queue.type="LinkedList" queue.filename="fwd" action.resumeRetryCount="-1")
```

```
sudo systemctl restart rsyslog
logger -p local0.info "test from $(hostname)"   # send a test message
```

In modern stacks you'll often ship logs with an agent (Fluent Bit, Vector, the OpenTelemetry Collector, CloudWatch agent) reading the journal or files directly.

4.3.2 logrotate

Text logs grow forever unless rotated. `logrotate` runs daily (systemd timer `logrotate.timer` or cron) and reads `/etc/logrotate.conf` plus `/etc/logrotate.d/*`.

```
# /etc/logrotate.d/myapp
/var/log/myapp/*.log {
    daily
    rotate 14
    compress
    delaycompress
    missingok
    notifempty
    maxsize 500M
    create 0640 myapp adm
    sharedscripts
    postrotate
        systemctl kill -s USR1 myapp.service >/dev/null 2>&1 || true
    endscript
}
```

Why `postrotate`? After renaming `app.log` to `app.log.1`, the app still holds the old file descriptor and keeps writing to the renamed file. Signalling it to reopen logs fixes that. If the app can't reopen, use `copytruncate` (copies then truncates in place; may lose a few lines).

```
sudo logrotate -d /etc/logrotate.d/myapp    # dry run / debug
sudo logrotate -f /etc/logrotate.d/myapp    # force rotation now
```

4.4 Package Management

Package managers install software with dependency resolution, verify signatures, track which files belong to which package, and make patching a single command. **Prefer distro packages** over hand-installed binaries: they get security updates.

4.4.1 Debian/Ubuntu: apt and dpkg

```
sudo apt update                     # refresh package index (do this first)
sudo apt upgrade                    # upgrade installed packages
sudo apt full-upgrade               # also allow removals for dependency changes
sudo apt install -y nginx=1.24.*    # install (optionally a version)
sudo apt remove nginx; sudo apt purge nginx   # purge also removes config
sudo apt autoremove
apt search redis; apt show redis-server
apt list --installed | grep nginx
apt-cache policy nginx              # candidate version and which repo
sudo apt-mark hold kubelet          # pin version
dpkg -l | grep ssl                  # installed packages (low level)
```

```
dpkg -L nginx                        # files from a package
dpkg -S /usr/sbin/nginx              # which package owns a file
sudo dpkg -i ./pkg.deb && sudo apt -f install   # local .deb, fix deps
```

In scripts and Dockerfiles use `apt-get` (stable output) with `DEBIAN_FRONTEND=noninteractive`:

```
apt-get update && apt-get install -y --no-install-recommends curl ca-certificates \
  && rm -rf /var/lib/apt/lists/*
```

Adding a third-party repository the modern way (signed-by keyring, not the deprecated `apt-key`):

```
sudo install -d -m 0755 /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o
/etc/apt/keyrings/docker.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
https://download.docker.com/linux/ubuntu $(. /etc/os-release; echo $VERSION_CODENAME) stable"
\
  | sudo tee /etc/apt/sources.list.d/docker.list
sudo apt update
```

Ubuntu 24.04 also supports the deb822 `.sources` format in `/etc/apt/sources.list.d/`.

4.4.2 RHEL/Rocky/Alma/Fedora/Amazon Linux: dnf and rpm

`yum` on RHEL 8+ is an alias for `dnf`.

```
sudo dnf check-update
sudo dnf upgrade -y
sudo dnf upgrade --security          # security errata only (RHEL)
sudo dnf install -y nginx
sudo dnf remove nginx
dnf search redis; dnf info redis
dnf provides /usr/bin/dig            # which package provides a file -> bind-utils
dnf list installed | grep kernel
dnf repolist
sudo dnf install -y epel-release     # Extra Packages for Enterprise Linux (Rocky/Alma)
sudo dnf history; sudo dnf history undo 15
sudo dnf install 'dnf-command(versionlock)' && sudo dnf versionlock add kubelet
rpm -qa | grep openssl
rpm -ql nginx                        # files in package
rpm -qf /etc/nginx/nginx.conf        # owner of file
rpm -V nginx                         # verify files against package metadata
```

Repositories are defined in `/etc/yum.repos.d/*.repo`:

```
[myrepo]
name=My internal repo
baseurl=https://repo.example.com/el9/$basearch
enabled=1
```

```
gpgcheck=1
gpgkey=https://repo.example.com/RPM-GPG-KEY
```

4.4.3 Alpine: apk

```
apk update
apk add --no-cache curl bash        # --no-cache: don't keep index (containers)
apk del curl
apk search nginx; apk info -L nginx
apk add --virtual .build-deps gcc musl-dev && ... && apk del .build-deps
```

4.4.4 Building from source

Sometimes you need a version or build flag the distro doesn't offer.

```
sudo apt install -y build-essential   # or: sudo dnf groupinstall "Development Tools"
curl -LO https://example.org/tool-2.4.1.tar.gz
sha256sum tool-2.4.1.tar.gz            # compare with published checksum!
tar xzf tool-2.4.1.tar.gz && cd tool-2.4.1
./configure --prefix=/usr/local
make -j"$(nproc)"
sudo make install
```

Downsides: no automatic security updates, no clean uninstall, no inventory. Better options, in order: the distro package, the vendor's official repo, a static binary from the vendor's releases (verify checksum), building your own `.deb` / `.rpm` (e.g. with `fpm` or proper packaging), or running it in a container.

> **Note:** Snap and Flatpak exist (Ubuntu ships some tools as snaps), but servers in DevOps environments rarely rely on them.

4.4.5 Patching strategy

- Enable unattended security updates: `unattended-upgrades` on Ubuntu, `dnf-automatic` on RHEL family.
- After kernel or glibc/openssl updates, services must restart or the host must reboot. Check: `needrestart` (Ubuntu), `dnf needs-restarting -r` (RHEL), `/var/run/reboot-required` (Ubuntu).
- Patch in rings: dev → staging → a canary slice of prod → everything. In immutable infrastructure you rebuild the image instead of patching in place.

4.5 Scheduling: cron and systemd Timers

4.5.1 cron

```
# m   h   dom mon dow   command
  */5 *   *   *   *     /usr/local/bin/healthcheck.sh
  0   2   *   *   *     /usr/local/bin/backup.sh >> /var/log/backup.log 2>&1
  30  6   *   *   1-5   /usr/local/bin/report.sh       # weekdays 06:30
  0   0   1   *   *     /usr/local/bin/monthly.sh
  @reboot               /usr/local/bin/on-boot.sh
```

```
crontab -e            # edit your crontab
crontab -l            # list
sudo crontab -u deploy -l
ls /etc/cron.d /etc/cron.daily   # system cron (files in cron.d have a USER field)
```

cron pitfalls that bite everyone:

- **Minimal environment**: `PATH` is usually just `/usr/bin:/bin`. Use absolute paths or set `PATH=` at the top of the crontab.
- `%` **is special** in crontab lines (newline). Escape it: `date +\%F`.
- **Output** is mailed (often nowhere). Redirect to a log file or `logger`.
- **Overlap**: if a job runs longer than its interval, copies pile up. Use `flock -n /run/lock/job.lock cmd`.
- **Timezone**: system timezone unless `CRON_TZ=` is set.

4.5.2 systemd timers

Timers fix most cron pitfalls: logs go to the journal, runs don't overlap (a service that's still active isn't started again), missed runs can be caught up (`Persistent=true`), and you get resource limits and dependencies for free.

```
# /etc/systemd/system/backup.service
[Unit]
Description=Nightly backup

[Service]
Type=oneshot
User=backup
ExecStart=/usr/local/bin/backup.sh
```

```
# /etc/systemd/system/backup.timer
[Unit]
Description=Run backup nightly

[Timer]
```

```
OnCalendar=*-*-* 02:00:00
RandomizedDelaySec=15m
Persistent=true

[Install]
WantedBy=timers.target
```

```
sudo systemctl daemon-reload
sudo systemctl enable --now backup.timer
systemctl list-timers
systemd-analyze calendar "Mon..Fri 06:30"     # validate an expression
sudo systemctl start backup.service           # run now, manually
journalctl -u backup.service
```

Other timer forms: `OnBootSec=5min`, `OnUnitActiveSec=1h` (monotonic, relative to last run), `OnCalendar=hourly|daily|weekly`.

> **Interview:** "Cron vs systemd timers?" Cron is universal and simple. Timers integrate with the journal, avoid overlapping runs, support catch-up after downtime and randomized delay, can depend on other units, and can be sandboxed and resource-limited. On systemd hosts, prefer timers for anything important.

4.6 Lab: Deploy a Supervised Service with a Timer

Goal: run a small HTTP service under systemd with hardening, automatic restart, log rotation, and a scheduled health check.

1. Create a service user and app:

`bash sudo useradd -r -s /usr/sbin/nologin -d /opt/hello hello sudo mkdir -p /opt/hello sudo tee /opt/hello/server.py > /dev/null <<'EOF' import http.server, socketserver, os, sys PORT = int(os.environ.get("PORT", "8080")) class H(http.server.SimpleHTTPRequestHandler): def do_GET(self): self.send_response(200); self.end_headers() self.wfile.write(b"hello from systemd\n") print(f"GET {self.path}", flush=True) socketserver.TCPServer.allow_reuse_address = True with socketserver.TCPServer(("", PORT), H) as s: print(f"listening on {PORT}", flush=True); s.serve_forever() EOF echo "PORT=8080" | sudo tee /etc/hello.env`

2. Write the unit:

```bash sudo tee /etc/systemd/system/hello.service > /dev/null <<'EOF' [Unit] Description=Hello demo service After=network-online.target Wants=network-online.target

[Service] User=hello EnvironmentFile=/etc/hello.env ExecStart=/usr/bin/python3 /opt/hello/server.py Restart=on-failure RestartSec=2 NoNewPrivileges=yes ProtectSystem=strict ProtectHome=yes PrivateTmp=yes MemoryMax=100M
```

[Install] WantedBy=multi-user.target EOF sudo systemctl daemon-reload sudo systemctl enable --now hello systemctl status hello --no-pager curl -s localhost:8080 ```

3. Prove supervision works. Kill it with a signal that counts as failure and watch it come back:

`bash sudo systemctl kill -s KILL hello sleep 3; systemctl status hello --no-pager | head -5 journalctl -u hello -n 20 --no-pager`

4. Add a drop-in override changing the port:

`bash sudo systemctl edit hello`

Add these lines in the editor, then save:

`ini [Service] EnvironmentFile= Environment=PORT=9090`

`bash sudo systemctl restart hello && curl -s localhost:9090 systemctl cat hello`

Why the empty `EnvironmentFile=` line? Variables read from `EnvironmentFile=` override those set with `Environment=`, so without clearing the list the file's `PORT=8080` would still win. An empty assignment resets a list-type setting, the same trick used for `ExecStart=`.

5. Check its security score: `systemd-analyze security hello`.

6. Create a health-check timer every minute:

`bash sudo tee /etc/systemd/system/hello-check.service > /dev/null <<'EOF' [Unit] Description=Check hello service [Service] Type=oneshot ExecStart=/bin/sh -c 'curl -fsS --max-time 3 localhost:9090 >/dev/null || systemctl restart hello' EOF sudo tee /etc/systemd/system/hello-check.timer > /dev/null <<'EOF' [Timer] OnBootSec=1min OnUnitActiveSec=1min [Install] WantedBy=timers.target EOF sudo systemctl daemon-reload && sudo systemctl enable --now hello-check.timer systemctl list-timers hello-check.timer`

7. Install a package and query it both ways: on Ubuntu `sudo apt install -y jq && dpkg -L jq | head && dpkg -S "$(command -v jq)"`; on Rocky `sudo dnf install -y jq && rpm -ql jq | head && rpm -qf "$(command -v jq)"`.

8. Create a logrotate config for a test file and force a rotation:

`bash sudo mkdir -p /var/log/hello && echo test | sudo tee /var/log/hello/app.log printf '/var/log/hello/*.log {\n daily\n rotate 3\n compress\n missingok\n}\n' | sudo tee /etc/logrotate.d/hello sudo logrotate -f /etc/logrotate.d/hello && ls /var/log/hello`

9. Clean up: disable and remove the units, `daemon-reload`, delete the user.

Key takeaways

- systemd units describe services declaratively; `enable` (boot) and `start` (now) are different things.
- Separate ordering (`After=`) from dependency (`Wants=` / `Requires=`); use `network-online.target` for apps that need the network.

- Never edit vendor units; use `systemctl edit` drop-ins and `daemon-reload`.
- Sandbox services with `ProtectSystem`, `NoNewPrivileges`, `PrivateTmp`, and set resource limits with `MemoryMax` / `CPUQuota`.
- `journalctl -u UNIT -b -p err --since` answers most “what happened” questions; make the journal persistent.
- logrotate needs the app to reopen its log (postrotate signal) or `copytruncate`.
- apt/dnf/apk all do the same jobs: refresh, install, upgrade, query ownership. Prefer distro or vendor packages over source builds.
- Prefer systemd timers to cron for important jobs; if you use cron, mind PATH, `%`, output and overlaps.

Part 5 — Disks, Filesystems and Storage

“Disk full” is the most common production incident there is, and storage changes (growing a volume, adding a disk, fixing a bad fstab) are among the riskiest routine tasks. This Part builds the mental model from block device up to mounted filesystem.

5.1 The Storage Stack

```
Application: write("/var/lib/pg/data/file")
       |
VFS (virtual filesystem layer)
       |
Filesystem: ext4 / xfs / btrfs        <- mkfs, mount, fsck
       |
[optional] LVM logical volume          <- lvcreate, lvextend
[optional] dm-crypt (LUKS), mdraid
       |
Partition: /dev/nvme0n1p2             <- parted, fdisk, gdisk
       |
Block device: /dev/nvme0n1, /dev/sda, /dev/xvdf  (disk, EBS volume, virtio disk)
```

Every layer is optional except the device and the filesystem. A cloud data volume is often formatted directly (no partition, no LVM), which keeps resizing simple.

5.2 Discovering Block Devices

```
lsblk                       # tree of devices, partitions, mountpoints
lsblk -f                    # add filesystem type, label, UUID
sudo blkid                  # UUIDs and types
sudo fdisk -l /dev/sda      # partition table
cat /proc/partitions
ls -l /dev/disk/by-uuid/ /dev/disk/by-id/
```

```
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
nvme0n1     259:0    0   30G  0 disk
|-nvme0n1p1 259:1    0   29G  0 part /
|-nvme0n1p14 259:2   0    4M  0 part
`-nvme0n1p15 259:3   0  106M  0 part /boot/efi
nvme1n1     259:4    0  100G  0 disk
```

Device naming:

Name	Meaning
`/dev/sda`, `/dev/sdb`	SCSI/SATA/SAS, and USB disks
`/dev/vda`	virtio disk (KVM)
`/dev/xvda`	Xen (older EC2)
`/dev/nvme0n1`	NVMe controller 0, namespace 1; partitions are `p1`, `p2`; all modern EC2 EBS volumes appear as NVMe

> **Warning:** Device names are *not stable*. `/dev/sdb` can become `/dev/sdc` after a reboot or when disks are attached in a different order. Always reference filesystems by **UUID** or label in fstab, and EBS volumes via `/dev/disk/by-id/nvme-Amazon_Elastic_Block_Store_vol...`.

5.3 Partitioning

Two partition table formats:

- **MBR (msdos)**: legacy, 4 primary partitions, max 2 TiB.
- **GPT**: modern, 128 partitions, huge disks, backup header at the end of the disk. Use GPT.

```
# Scriptable, non-interactive: one partition spanning the disk
sudo parted -s /dev/sdb mklabel gpt
sudo parted -s /dev/sdb mkpart primary 1MiB 100%
sudo parted -s /dev/sdb set 1 lvm on        # optional: flag for LVM
lsblk /dev/sdb
```

Interactive alternatives: `fdisk` (handles GPT too), `gdisk`, `cfdisk`. After partitioning a disk that's in use, the kernel may need `sudo partprobe` to re-read the table.

5.4 Filesystems

Filesystem	Strengths	Notes
ext4	Mature, default on Debian/Ubuntu; can grow and shrink (offline)	Fixed inode count at mkfs time
XFS	Default on RHEL family; great for large files and parallel I/O	Can grow, **cannot shrink**; dynamic inodes
btrfs	Snapshots, checksums, compression, subvolumes	Default on Fedora/openSUSE desktops
vfat	EFI System Partition	No Unix permissions
tmpfs	RAM-backed	`/run`, often `/tmp`

Filesystem	Strengths	Notes
overlayfs	Union of layers	What Docker/containerd use for image layers
NFS / EFS	Network filesystem	Mount options matter (`_netdev`, `nofail`)

```
sudo mkfs.ext4 -L data /dev/sdb1
sudo mkfs.xfs -L data /dev/sdb1
sudo tune2fs -l /dev/sdb1 | head -20      # ext4 details
sudo xfs_info /data                        # xfs details (mounted)
```

5.4.1 Inodes

Each file consumes one **inode** (metadata: owner, mode, timestamps, pointers to data blocks). Filenames live in directories, mapping names to inode numbers. On ext4 the number of inodes is fixed when you create the filesystem, so a disk can be "full" with plenty of free bytes if millions of tiny files used up all inodes.

```
df -i
```

```
Filesystem      Inodes   IUsed  IFree IUse% Mounted on
/dev/nvme0n1p1 1966080 1966080      0  100% /
```

Classic culprits: PHP session files, mail queues, cache directories, a runaway job creating temp files. Find directories with the most files:

```
sudo find / -xdev -type f | cut -d/ -f2-3 | sort | uniq -c | sort -rn | head
```

For filesystems that will hold millions of small files, create ext4 with more inodes (`mkfs.ext4 -i 4096`) or use XFS.

5.5 Mounting and fstab

Mounting attaches a filesystem to a directory (the **mount point**). Anything previously in that directory is hidden (not deleted) while the mount is active, a source of confusing "missing files" moments.

```
sudo mkdir -p /data
sudo mount /dev/sdb1 /data
sudo mount -o remount,ro /data      # change options live
findmnt /data                       # source, fstype, options
mount | column -t | grep data
sudo umount /data                   # 'target is busy'? see below
```

```
sudo lsof +f -- /data               # who's using it
sudo fuser -vm /data
```

5.5.1 /etc/fstab

`/etc/fstab` declares what to mount at boot. systemd converts each line into a `.mount` unit.

```
# <device>                                <mountpoint> <type> <options>
<dump> <pass>
UUID=3f1c2a9e-8d1b-4c1a-9f0e-2b7c5d1e9a01   /            ext4   defaults,discard          0
1
UUID=7B3A-11F2                            /boot/efi    vfat   umask=0077                0
1
UUID=a1b2c3d4-0000-4000-8000-123456789abc   /data        xfs    defaults,nofail,noatime   0
2
fs-0123abcd.efs.us-east-1.amazonaws.com:/   /mnt/efs     nfs4   nfsvers=4.1,_netdev,nofail 0
0
/swapfile                                 none         swap   sw                        0
0
```

Important options:

- **`nofail`**: don't fail the boot if this device is missing. Essential for secondary and cloud volumes; without it, a detached EBS volume drops the instance into emergency mode, where you can't SSH in.
- **`_netdev`**: network filesystem; wait for the network.
- **`noatime`**: don't update access time on every read (less write I/O).
- **`ro`**, **`noexec`**, **`nosuid`**, **`nodev`**: hardening for `/tmp`, data and removable mounts.
- **pass**: fsck order; `1` for root, `2` for others, `0` to skip.

Always test fstab before rebooting:

```
sudo findmnt --verify          # syntax and sanity check
sudo mount -a                  # mount everything not yet mounted; errors appear now, not at
boot
sudo systemctl daemon-reload   # regenerate mount units
```

> **Warning:** A typo in fstab is one of the most common reasons a cloud VM stops booting. If it happens: use the serial console, boot to rescue/emergency mode, or detach the root volume and attach it to another instance to fix the file.

5.5.2 Swap

```
sudo fallocate -l 2G /swapfile && sudo chmod 600 /swapfile
sudo mkswap /swapfile && sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
swapon --show; free -h
```

Kubernetes nodes historically required swap off (newer versions support it with configuration). Database hosts often keep a small swap and set `vm.swappiness` low (Part 8).

5.6 LVM: Logical Volume Manager

LVM inserts a flexible layer between partitions and filesystems, so you can pool disks, resize volumes online, and take snapshots.

```
Physical Volumes (PV):   /dev/sdb1      /dev/sdc1
                              \             /
Volume Group (VG):             vg_data (pool of extents)
                              /         \
Logical Volumes (LV):  lv_app (50G)   lv_logs (20G)    <- mkfs these
                        /data/app      /var/log/app
```

```
sudo pvcreate /dev/sdb1 /dev/sdc1
sudo vgcreate vg_data /dev/sdb1 /dev/sdc1
sudo lvcreate -n lv_app -L 50G vg_data
sudo lvcreate -n lv_logs -l 100%FREE vg_data
sudo mkfs.xfs /dev/vg_data/lv_app
sudo pvs; sudo vgs; sudo lvs          # summaries
sudo lvdisplay /dev/vg_data/lv_app
```

LVs appear as `/dev/vg_data/lv_app` and `/dev/mapper/vg_data-lv_app`.

5.6.1 Growing storage (the routine task)

```
# Add a new disk to the VG
sudo pvcreate /dev/sdd
sudo vgextend vg_data /dev/sdd
# Grow LV and filesystem in one step (-r runs the right fs resize tool)
sudo lvextend -r -L +20G /dev/vg_data/lv_app
# or use all free space
sudo lvextend -r -l +100%FREE /dev/vg_data/lv_app
```

Without `-r`, you'd follow with `resize2fs` (ext4) or `xfs_growfs /mountpoint` (XFS). Both grow online.

5.6.2 Growing a cloud root disk (no LVM)

After enlarging an EBS volume in AWS (or a disk in any cloud):

```
lsblk                                 # disk shows 50G, partition still 30G
sudo growpart /dev/nvme0n1 1          # grow partition 1 (from cloud-guest-utils / cloud-
utils-growpart)
sudo resize2fs /dev/nvme0n1p1         # ext4
```

```
sudo xfs_growfs -d /                    # xfs (takes the mount point)
df -h /
```

Many cloud images run `growpart` automatically at boot through cloud-init.

5.6.3 Snapshots

```
sudo lvcreate -s -n lv_app_snap -L 5G /dev/vg_data/lv_app   # copy-on-write snapshot
sudo mount -o ro,nouuid /dev/vg_data/lv_app_snap /mnt/snap  # nouuid needed for XFS
# ... back it up, then:
sudo umount /mnt/snap && sudo lvremove -y /dev/vg_data/lv_app_snap
```

The snapshot needs enough space to hold changed blocks; if it fills, it becomes invalid.

5.7 RAID Basics

RAID combines disks for redundancy and/or performance. Linux software RAID uses `mdadm`.

Level	Min disks	Usable capacity	Survives	Use
RAID 0	2	100%	No failures	Scratch space, speed only
RAID 1	2	50%	n-1 disks	Boot/OS mirrors
RAID 5	3	(n-1)/n	1 disk	Read-heavy; slow rebuilds on big disks
RAID 6	4	(n-2)/n	2 disks	Large arrays
RAID 10	4	50%	1 per mirror pair	Databases: performance + redundancy

```
sudo mdadm --create /dev/md0 --level=1 --raid-devices=2 /dev/sdb /dev/sdc
cat /proc/mdstat
sudo mdadm --detail /dev/md0
sudo mdadm --detail --scan | sudo tee -a /etc/mdadm/mdadm.conf   # persist (path varies:
/etc/mdadm.conf on RHEL)
sudo update-initramfs -u   # or dracut -f
```

> **Note:** RAID is not a backup. It protects against disk failure, not against `rm -rf`, corruption, or ransomware. In the cloud, EBS already replicates within an AZ; people use RAID 0 across volumes only for throughput.

5.8 Disk Usage: df and du

```
df -h                       # space per mounted filesystem
df -hT /var                 # with fs type, for the fs holding /var
```

```
df -i                       # inodes
du -sh /var/log             # total of a directory
du -h --max-depth=1 /var | sort -h      # biggest subdirectories
sudo du -xh / --max-depth=2 2>/dev/null | sort -h | tail -20   # -x: stay on one fs
ncdu -x /                   # interactive explorer (install it)
```

5.8.1 When df and du disagree

`df` reports what the filesystem has allocated; `du` adds up files it can *see* by walking directories. If `df` says 95% full and `du` finds far less, the usual cause is a **deleted file still held open by a process**. The name is gone, but the inode and blocks remain until the last file descriptor closes.

```
sudo lsof +L1                   # open files with link count 0 (deleted)
sudo lsof -nP | grep '(deleted)'
```

```
COMMAND  PID  USER   FD   TYPE DEVICE   SIZE/OFF NLINK  NODE NAME
java    2210  app    12w   REG  259,1 21474836480     0 39012 /var/log/app/app.log (deleted)
```

Fix: restart the process, or truncate the file through its descriptor without restarting:

```
sudo truncate -s 0 /proc/2210/fd/12
```

The other common cause is files **hidden under a mount point** (written to `/data` before the volume was mounted). Bind-mount root elsewhere to look: `sudo mount --bind / /mnt/rootview && sudo du -sh /mnt/rootview/data`.

> **Tip:** To empty a log that a process is writing, use `truncate -s 0 file` or `: > file`, not `rm`. Removing it leaves the space allocated until the process restarts.

5.9 Checking and Repairing Filesystems

```
sudo umount /dev/sdb1
sudo fsck -f /dev/sdb1          # ext4: e2fsck
sudo xfs_repair /dev/sdb1       # xfs (must be unmounted)
sudo dmesg -T | grep -iE 'I/O error|EXT4-fs error|XFS'   # look for disk problems
sudo smartctl -a /dev/sda       # SMART health (physical disks; smartmontools)
```

Never run fsck on a mounted read-write filesystem. If the kernel detects corruption, ext4 may remount itself read-only (`errors=remount-ro`), which shows up as “Read-only file system” errors in apps.

5.10 Lab: Build, Grow and Break Storage Safely

Goal: practise the full LVM workflow and fstab handling using loop devices, so no real disk is at risk.

1. Create three 1 GiB “disks” backed by files:

```
bash sudo mkdir -p /lab5 for i in 1 2 3; do sudo truncate -s 1G /lab5/disk$i.img; done for i in 1 2 3; do sudo losetup -fP --show /lab5/disk$i.img; done lsblk | grep loop
```

Note the device names printed (e.g. `/dev/loop10`, `/dev/loop11`, `/dev/loop12`). Below we call them `$D1 $D2 $D3`:

```
bash D1=/dev/loop10; D2=/dev/loop11; D3=/dev/loop12 # adjust!
```

2. Build LVM on two of them:

```
bash sudo pvcreate $D1 $D2 sudo vgcreate vg_lab $D1 $D2 sudo lvcreate -n lv_data -L 1G vg_lab sudo mkfs.ext4 -L labdata /dev/vg_lab/lv_data sudo mkdir -p /mnt/labdata sudo mount /dev/vg_lab/lv_data /mnt/labdata df -h /mnt/labdata
```

3. Persist it in fstab with UUID and `nofail`, then verify without rebooting:

```
bash UUID=$(sudo blkid -s UUID -o value /dev/vg_lab/lv_data) echo "UUID=$UUID /mnt/labdata ext4 defaults,nofail 0 2" | sudo tee -a /etc/fstab sudo umount /mnt/labdata sudo findmnt --verify && sudo mount -a && findmnt /mnt/labdata
```

4. Grow online: add the third disk and extend the LV and filesystem together:

```
bash sudo pvcreate $D3 && sudo vgextend vg_lab $D3 sudo lvextend -r -l +100%FREE /dev/vg_lab/lv_data df -h /mnt/labdata
```

5. Reproduce “df vs du” disagreement:

```
bash sudo dd if=/dev/zero of=/mnt/labdata/big.log bs=1M count=500 status=none sudo bash -c 'exec 3>>/mnt/labdata/big.log; sleep 600' & sleep 1; sudo rm /mnt/labdata/big.log df -h /mnt/labdata; sudo du -sh /mnt/labdata sudo lsof +L1 | grep big.log
```

Then kill the `sleep` process and watch `df` drop.

6. Exhaust inodes on a tiny filesystem:

```
bash sudo truncate -s 20M /lab5/tiny.img sudo mkfs.ext4 -q -N 128 /lab5/tiny.img sudo mkdir -p /mnt/tiny && sudo mount -o loop /lab5/tiny.img /mnt/tiny sudo bash -c 'for i in $(seq 1 5000); do touch /mnt/tiny/f$i || break; done' df -h /mnt/tiny; df -i /mnt/tiny # space free, inodes 100%
```

(ext4 rounds the inode count up to fill whole inode-table blocks, so you’ll get a few hundred inodes rather than exactly 128; the loop stops at the first “No space left on device”.)

7. Take an LVM snapshot. The VG is full after step 4, so add a fourth disk first:

```
bash sudo truncate -s 1G /lab5/disk4.img D4=$(sudo losetup -fP --show /lab5/disk4.img) sudo pvcreate $D4 && sudo vgextend vg_lab $D4 echo "before" | sudo tee /mnt/labdata/state.txt sudo lvcreate -s -n lv_snap -L 500M vg_lab/lv_data echo "after" | sudo tee /mnt/labdata/state.txt sudo mkdir -p /mnt/snap && sudo mount -o ro /dev/vg_lab/lv_snap /mnt/snap cat /mnt/snap/state.txt # still "before": the snapshot froze that moment sudo lvs vg_lab # Data% shows how full the COW area is sudo umount /mnt/snap && sudo lvremove -y vg_lab/lv_snap
```

8. Clean up carefully, in reverse order:

```
bash sudo umount /mnt/tiny /mnt/labdata sudo sed -i '|/mnt/labdata|d' /etc/fstab sudo lvremove -y vg_lab/lv_data && sudo vgremove vg_lab && sudo pvremove $D1 $D2 $D3 $D4 sudo losetup -d $D1 $D2 $D3 $D4 sudo rm -rf /lab5
```

Key takeaways

- The stack is device → (partition) → (LVM/RAID/crypt) → filesystem → mount point. Know which layer you're changing.
- Device names change; reference filesystems by UUID, and use `nofail` for non-root volumes, especially in the cloud.
- Always run `findmnt --verify` and `mount -a` after editing fstab, before rebooting.
- LVM makes online growth routine: `vgextend` then `lvextend -r`. XFS grows but never shrinks.
- "Disk full" can mean bytes (`df -h`) or inodes (`df -i`).
- When df and du disagree, look for deleted-but-open files (`lsof +L1`) or files hidden under a mount point.
- RAID handles disk failure, not data loss; you still need backups.

Part 6 — Networking and SSH

Most production incidents eventually involve the network: a service that can't reach its database, DNS that resolves to the wrong place, a firewall rule nobody remembers, or an SSH session that hangs. This Part gives you the Linux side of networking with modern tools (`ip`, `ss`, `nmcli`, `resolvectl`, `nft`) and a deep dive into SSH, the tool you'll use more than any other.

6.1 Network Fundamentals in Five Minutes

```
Layer         What                 Linux tools
-----------   ------------------   --------------------------------
7 App         HTTP, DNS, SSH       curl, dig, ssh, openssl s_client
4 Transport   TCP / UDP ports      ss, nc, iptables/nft
3 Network     IP, routing          ip addr, ip route, ping, traceroute, mtr
2 Link        Ethernet, MAC, ARP   ip link, ip neigh, ethtool
```

Quick vocabulary: an **IP address with prefix** (`10.0.1.15/24`) identifies a host and its subnet; the **default gateway** is where packets go when no specific route matches; **DNS** maps names to IPs; a **port** identifies a service on a host; a **socket** is one endpoint (IP:port) of a connection; **TCP** is reliable and connection-oriented (three-way handshake SYN, SYN-ACK, ACK); **UDP** is connectionless (DNS, much monitoring, QUIC).

6.2 Interfaces, Addresses and Routes with `ip`

`ifconfig`, `route` and `arp` (net-tools) are deprecated and often not installed. Use `ip` (iproute2).

```
ip -br addr                 # brief: interface, state, addresses
ip addr show dev eth0
ip -br link                 # link state, MAC
ip route                    # routing table
ip route get 8.8.8.8        # which route/interface/source IP would be used
ip neigh                    # ARP/neighbor table
ip -s link show eth0        # packet/error counters
```

```
lo               UNKNOWN        127.0.0.1/8 ::1/128
ens5             UP             10.0.1.15/24 fe80::8ff:fe12:3456/64

default via 10.0.1.1 dev ens5 proto dhcp src 10.0.1.15 metric 100
10.0.1.0/24 dev ens5 proto kernel scope link src 10.0.1.15
```

Temporary changes (lost on reboot):

```
sudo ip addr add 10.0.1.50/24 dev ens5
sudo ip link set ens5 up
sudo ip route add 192.168.50.0/24 via 10.0.1.254
sudo ip route del 192.168.50.0/24
sudo ip link set ens5 mtu 9001
```

Interface names like `ens5`, `enp0s3`, `eth0` come from "predictable network interface naming" (based on firmware/PCI location), so they don't change when hardware is added.

6.2.1 Persistent configuration

Different distros use different config front ends. Know which one your system uses before editing anything.

Distro	Tool	Config
Ubuntu (server/cloud)	**netplan** → systemd-networkd	`/etc/netplan/*.yaml`
RHEL/Rocky/Alma/Fedora	**NetworkManager** (`nmcli`)	`/etc/NetworkManager/system-connections/*.nmconnection`
Amazon Linux 2023	systemd-networkd	`/etc/systemd/network/`
Debian (classic)	ifupdown	`/etc/network/interfaces`

netplan example:

```
# /etc/netplan/60-static.yaml
network:
  version: 2
  ethernets:
    ens5:
      dhcp4: false
      addresses: [10.0.1.50/24]
      routes:
        - to: default
          via: 10.0.1.1
      nameservers:
        addresses: [10.0.0.2, 1.1.1.1]
        search: [corp.example.com]
```

```
sudo netplan try        # applies with automatic rollback if you lose connectivity
sudo netplan apply
```

NetworkManager with `nmcli`:

```
nmcli device status
nmcli connection show
sudo nmcli con mod "System eth0" ipv4.method manual \
```

```
     ipv4.addresses 10.0.1.50/24 ipv4.gateway 10.0.1.1 ipv4.dns "10.0.0.2 1.1.1.1"
sudo nmcli con up "System eth0"
nmcli -f IP4 device show eth0
```

> **Warning:** Changing the network config of a remote machine can cut off your own SSH session. Use `netplan try`, schedule a rollback (`sudo systemd-run --on-active=5min /path/restore.sh`), or have console access ready.

6.3 Sockets and Ports with `ss`

`ss` replaces `netstat`.

```
ss -tulpn            # TCP+UDP, listening, process, numeric
ss -tnp state established
ss -tn dst 10.0.2.20:5432       # connections to the DB
ss -s                # summary counts
ss -tan | awk 'NR>1 {print $1}' | sort | uniq -c   # connections by state
```

```
Netid State  Recv-Q Send-Q Local Address:Port  Peer Address:Port Process
tcp   LISTEN 0      511          0.0.0.0:80         0.0.0.0:*     users:
(("nginx",pid=812,fd=6))
tcp   LISTEN 0      4096       127.0.0.1:5432       0.0.0.0:*     users:
(("postgres",pid=990,fd=7))
tcp   LISTEN 0      128            [::]:22            [::]:*     users:
(("sshd",pid=701,fd=4))
```

Read the **Local Address** carefully: `127.0.0.1:5432` means Postgres only accepts local connections, which is why remote clients get "connection refused" even with the firewall open. `0.0.0.0` (or `[::]` / `*`) means all interfaces.

TCP states worth recognising: `LISTEN`, `ESTAB`, `TIME-WAIT` (normal after the side that closed first; lots of them on a busy client is expected), `CLOSE-WAIT` (the remote closed but *our app* hasn't closed its socket: usually an application bug or leak), `SYN-SENT` (we're trying to connect and getting no answer: firewall or host down).

6.4 DNS on Linux

Name resolution order is controlled by `/etc/nsswitch.conf` (`hosts: files dns` means check `/etc/hosts` first, then DNS). The resolver configuration is `/etc/resolv.conf`:

```
nameserver 127.0.0.53
options edns0 trust-ad
search ec2.internal corp.example.com
```

On Ubuntu and other systemd-resolved systems, `/etc/resolv.conf` points to a local stub (`127.0.0.53`) and the real upstream servers are managed by systemd-resolved:

```
resolvectl status         # per-link DNS servers and search domains
resolvectl query api.example.com
resolvectl flush-caches
ls -l /etc/resolv.conf    # usually a symlink to /run/systemd/resolve/stub-resolv.conf
```

> **Note:** Don't hand-edit `/etc/resolv.conf` if it's managed (by systemd-resolved, NetworkManager, or DHCP); your change will be overwritten. Configure DNS in netplan, nmcli, or `/etc/systemd/resolved.conf`.

Query tools:

```
dig api.example.com              # full answer with TTL
dig +short api.example.com
dig @8.8.8.8 api.example.com     # ask a specific server (bypass local config)
dig -x 10.0.1.15                 # reverse lookup
dig api.example.com AAAA; dig example.com MX; dig example.com TXT
dig +trace example.com           # follow delegation from the root
getent hosts api.example.com     # resolve the way applications do (nsswitch, /etc/hosts)
```

The difference between `dig` and `getent` matters: `dig` talks DNS directly and ignores `/etc/hosts` and nsswitch; `getent` uses the system resolver exactly like your app. If `dig` works but the app fails, check `/etc/hosts`, nsswitch and search domains.

6.5 Testing Connectivity

```
ping -c 4 10.0.2.20            # ICMP; often blocked in clouds, so failure isn't proof
traceroute -n 10.0.2.20        # or tracepath (no root)
mtr -rwc 50 api.example.com    # continuous traceroute + loss stats
nc -zv 10.0.2.20 5432          # is TCP port open?
timeout 3 bash -c '</dev/tcp/10.0.2.20/5432' && echo open   # no nc installed
curl -v https://api.example.com/health
curl -sS -o /dev/null -w '%{http_code} dns=%{time_namelookup} connect=%{time_connect} tls=%
{time_appconnect} total=%{time_total}\n' https://api.example.com
curl --resolve api.example.com:443:10.0.1.99 https://api.example.com/   # force IP, keep
SNI/Host
curl -H "Host: app.example.com" http://10.0.1.99/
openssl s_client -connect api.example.com:443 -servername api.example.com </dev/null |
openssl x509 -noout -dates -subject
sudo tcpdump -ni ens5 port 5432 -c 20      # watch packets on the wire
sudo tcpdump -ni any host 10.0.2.20 -w cap.pcap   # capture for Wireshark
```

Interpreting connection failures:

Symptom	Likely meaning
`Connection refused` (immediate)	Host reachable, nothing listening on that port/address (or REJECT rule)
Timeout / hang	Packets dropped: firewall, security group, NACL, routing, host down
`No route to host`	No route, or ICMP-unreachable from a firewall
`Name or service not known`	DNS failure
TLS errors	Wrong cert, expired cert, SNI mismatch, missing CA

6.6 Firewalls

Linux packet filtering is done in the kernel by **netfilter**. User-space front ends program it.

```
ufw (Ubuntu)      firewalld (RHEL)       \
       \               |                  >  front ends
        iptables-nft / nft  ---------------/
                 |
        netfilter (kernel) hooks: PREROUTING, INPUT, FORWARD, OUTPUT, POSTROUTING
```

Modern distros use **nftables** under the hood; the `iptables` command on them is usually `iptables-nft`, a compatibility layer. Pick **one** front end per host and don't mix them.

6.6.1 ufw (Ubuntu)

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow OpenSSH            # allow SSH BEFORE enabling!
sudo ufw allow from 10.0.0.0/16 to any port 5432 proto tcp
sudo ufw limit 22/tcp             # rate-limit brute force
sudo ufw enable
sudo ufw status numbered
sudo ufw delete 3
```

6.6.2 firewalld (RHEL family)

firewalld groups rules into **zones** applied to interfaces or source ranges.

```
sudo firewall-cmd --state
sudo firewall-cmd --get-active-zones
sudo firewall-cmd --list-all
sudo firewall-cmd --permanent --add-service=https
sudo firewall-cmd --permanent --add-port=8080/tcp
sudo firewall-cmd --permanent --zone=internal --add-source=10.0.0.0/16
sudo firewall-cmd --reload        # --permanent changes apply after reload
```

6.6.3 nftables directly

```
sudo nft list ruleset
```

```
# /etc/nftables.conf
table inet filter {
  chain input {
    type filter hook input priority 0; policy drop;
    ct state established,related accept
    iif lo accept
    ip protocol icmp accept
    ip6 nexthdr icmpv6 accept
    tcp dport 22 ip saddr 10.0.0.0/16 accept
    tcp dport { 80, 443 } accept
  }
  chain forward { type filter hook forward priority 0; policy drop; }
  chain output  { type filter hook output priority 0; policy accept; }
}
```

```
sudo nft -c -f /etc/nftables.conf     # check syntax
sudo systemctl enable --now nftables
```

6.6.4 iptables (legacy, still everywhere in docs)

```
sudo iptables -L -n -v --line-numbers
sudo iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
sudo iptables -A INPUT -p tcp --dport 22 -s 10.0.0.0/16 -j ACCEPT
sudo iptables -A INPUT -p tcp --dport 443 -j ACCEPT
sudo iptables -P INPUT DROP
sudo iptables -t nat -L -n -v          # NAT table (Docker and kube-proxy live here)
sudo iptables-save > /etc/iptables/rules.v4
```

Rule order matters: the first matching rule wins. The `ESTABLISHED,RELATED` rule near the top allows replies to connections you initiated.

> **Note:** Docker and Kubernetes (kube-proxy in iptables mode, many CNIs) write their own netfilter rules. A host firewall that flushes all rules will break container networking. In clouds, security groups usually do the heavy lifting; host firewalls are defence in depth.

6.7 SSH Deep Dive

SSH gives you an encrypted, authenticated channel for shells, file copy, port forwarding and automation (Ansible, Git, CI deploys all ride on it).

6.7.1 How it works

1. The client connects to TCP 22; both sides negotiate algorithms and perform a key exchange (e.g. curve25519) to derive session keys.
2. The server proves its identity with its **host key**. The client checks it against `~/.ssh/known_hosts`. A mismatch triggers the scary "REMOTE HOST IDENTIFICATION HAS CHANGED" warning: either the host was rebuilt or someone is intercepting.
3. The user authenticates, ideally with a **public key**: the client proves possession of the private key by signing data; the server checks the signature against `~/.ssh/authorized_keys`. The private key never leaves the client.

6.7.2 Keys

```
ssh-keygen -t ed25519 -C "abdul@laptop" -f ~/.ssh/id_ed25519   # use a passphrase
ssh-copy-id -i ~/.ssh/id_ed25519.pub user@server
# or manually:
cat ~/.ssh/id_ed25519.pub | ssh user@server 'mkdir -p ~/.ssh && chmod 700 ~/.ssh && cat >>
~/.ssh/authorized_keys && chmod 600 ~/.ssh/authorized_keys'
ssh-keygen -lf ~/.ssh/id_ed25519.pub    # fingerprint
ssh-keygen -R server                    # remove stale known_hosts entry
ssh-keyscan -t ed25519 server >> ~/.ssh/known_hosts   # pre-seed (verify fingerprint out of
band!)
```

Prefer **ed25519** (small, fast, secure). Use RSA 3072/4096 only for legacy systems. Permissions matter: sshd refuses keys if `~/.ssh` or `authorized_keys` is writable by others (`StrictModes`).

`authorized_keys` options restrict what a key can do:

```
from="10.0.0.0/16",command="/usr/local/bin/backup-receive",no-port-forwarding,no-pty ssh-
ed25519 AAAAC3... backup@ci
```

6.7.3 The agent

`ssh-agent` holds decrypted keys in memory so you type the passphrase once.

```
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519
ssh-add -l
ssh -A bastion        # agent forwarding: use with care
```

> **Warning:** With agent forwarding, anyone with root on the remote host can use your agent socket to authenticate as you while you're connected. Prefer `ProxyJump` (below), which never exposes your agent to intermediate hosts.

6.7.4 Client configuration

`~/.ssh/config` turns long commands into short names and is read by `ssh`, `scp`, `sftp`, `rsync`, `git` and Ansible.

```
Host *
    ServerAliveInterval 30
    ServerAliveCountMax 3
    AddKeysToAgent yes
    IdentitiesOnly yes

Host bastion
    HostName bastion.example.com
    User ec2-user
    IdentityFile ~/.ssh/id_ed25519

Host prod-*
    User ubuntu
    ProxyJump bastion
    IdentityFile ~/.ssh/prod_ed25519

Host prod-db1
    HostName 10.0.3.21

Host *.internal
    ControlMaster auto
    ControlPath ~/.ssh/cm-%r@%h:%p
    ControlPersist 10m
```

Now `ssh prod-db1` goes through the bastion automatically. `ControlMaster` multiplexes connections over one TCP session, making repeated SSH (Ansible!) much faster.

6.7.5 Bastions and ProxyJump

```
 laptop --SSH--> bastion (public subnet) --SSH--> app server (private subnet)
          ProxyJump: the second SSH is tunnelled end-to-end through the first;
          the bastion only forwards TCP and never sees your keys or session.
```

```
ssh -J ec2-user@bastion.example.com ubuntu@10.0.3.21
scp -J bastion file.tgz ubuntu@10.0.3.21:/tmp/
```

In AWS, **SSM Session Manager** (and EC2 Instance Connect Endpoint) can replace bastions entirely: no inbound port 22, IAM-based access, and session logging.

6.7.6 Tunnels (port forwarding)

```
# Local forward: reach a private DB through a jump host as localhost:15432
ssh -N -L 15432:prod-db.internal:5432 bastion
```

```
psql -h 127.0.0.1 -p 15432 -U app

# Remote forward: expose your local port 3000 on the server's port 8080
ssh -N -R 8080:localhost:3000 server

# Dynamic: SOCKS proxy for browsing internal sites
ssh -N -D 1080 bastion
curl --socks5-hostname localhost:1080 http://internal-dashboard/
```

`-N` means don't run a command (just forward); add `-f` to background it.

6.7.7 File transfer

```
scp -r ./dist user@server:/opt/app/          # simple copy
rsync -avz --delete ./dist/ user@server:/opt/app/   # incremental, trailing slash matters
rsync -avz -e "ssh -J bastion" ./data/ prod-db1:/backup/
sftp user@server
```

`rsync` only transfers differences and can resume, making it the right tool for deploys and backups. `src/` copies the *contents* of src; `src` copies the directory itself.

6.7.8 Server hardening

```
# /etc/ssh/sshd_config.d/10-hardening.conf
PermitRootLogin no
PasswordAuthentication no
KbdInteractiveAuthentication no
PubkeyAuthentication yes
AuthenticationMethods publickey
MaxAuthTries 3
LoginGraceTime 20
AllowGroups ssh-users
X11Forwarding no
AllowTcpForwarding no        # unless needed (bastions need it)
ClientAliveInterval 300
ClientAliveCountMax 2
```

```
sudo sshd -t                     # test config BEFORE restarting
sudo systemctl reload ssh        # Debian/Ubuntu unit name 'ssh'; RHEL uses 'sshd'
sudo sshd -T | grep -i passwordauth   # effective settings
```

> **Warning:** Keep your current session open while changing sshd config, and test a *new* login from a second terminal before disconnecting. A typo or an `AllowGroups` you're not in will lock you out.

On Ubuntu 22.10+, sshd may be **socket-activated** (`ssh.socket`); changing `Port` then requires editing the socket unit or disabling socket activation. Extra hardening: fail2ban (Part 9), restricting port 22 to VPN/bastion ranges, short-lived **SSH certificates** signed by a CA (Vault, Teleport, step-ca) instead of

distributing static keys.

6.7.9 Debugging SSH

```
ssh -vvv user@server            # client-side verbose: which keys tried, where it stops
sudo journalctl -u ssh -f       # server side (or -u sshd); /var/log/auth.log,
/var/log/secure
```

Common failures: wrong user for the AMI (`ubuntu`, `ec2-user`, `rocky`, `admin`), bad permissions on `~/.ssh`, key not offered (`IdentitiesOnly` / wrong `IdentityFile`), security group blocking 22, host key changed, `AllowGroups` excluding the user, SELinux contexts on a non-standard `authorized_keys` path (`restorecon -Rv ~/.ssh`). Part 10 has a full walkthrough.

6.8 Lab: Network Inspection, Firewall and SSH Tunnels

Goal: inspect networking with modern tools, lock down a host firewall, and use SSH config, ProxyJump and tunnels. You need two VMs (`vm1`, `vm2`) on the same network; one can act as the bastion.

1. On `vm1`, inventory the network:

`bash ip -br addr; ip route; ip route get 1.1.1.1 resolvectl status 2>/dev/null || cat /etc/resolv.conf ss -tulpn`

2. Compare resolution paths: add `10.9.9.9 fake.lab` to `/etc/hosts`, then:

`bash getent hosts fake.lab # found dig +short fake.lab # empty: dig ignores /etc/hosts`

3. Start a service listening only on localhost, and observe it from `vm2`:

`bash # vm1 python3 -m http.server 8000 --bind 127.0.0.1 & ss -tlnp | grep 8000 # vm2 curl -m 3 http://VM1_IP:8000/ # refused`

4. Rebind to all interfaces (`--bind 0.0.0.0`) and confirm `vm2` can connect. Then firewall it (Ubuntu):

`bash sudo ufw allow OpenSSH sudo ufw default deny incoming sudo ufw enable # vm2: curl now times out instead of being refused. Why the difference? sudo ufw allow from VM2_IP to any port 8000 proto tcp`

On Rocky use `firewall-cmd --permanent --add-rich-rule='rule family=ipv4 source address=VM2_IP port port=8000 protocol=tcp accept' && firewall-cmd --reload`.

5. Measure HTTP timing from `vm2`:

`bash curl -s -o /dev/null -w 'connect=%{time_connect} total=%{time_total}\n' http://VM1_IP:8000/`

6. Set up key-based SSH from your laptop to both VMs, and write `~/.ssh/config` with `vm1` as the bastion and `ProxyJump vm1` for `vm2`. Verify `ssh vm2 hostname` works through the jump.

7. Create a local tunnel to the localhost-only service (restore `--bind 127.0.0.1` first):

```
bash ssh -N -L 18000:127.0.0.1:8000 vm1 & curl http://localhost:18000/
```

8. Harden sshd on `vm2` with the drop-in from 6.7.8 (keep a session open), run `sudo sshd -t`, reload, and confirm password login now fails: `ssh -o PubkeyAuthentication=no vm2`.

9. Capture the handshake: on `vm1` run `sudo tcpdump -ni any port 8000 -c 10` while `vm2` curls it. Identify SYN, SYN-ACK and ACK in the output.

Key takeaways

- Use `ip` for addresses/routes/links and `ss` for sockets; `ip route get` and `ss -tulpn` answer "where does traffic go" and "what is listening where".
- A service bound to `127.0.0.1` is unreachable remotely regardless of firewall rules.
- Persistent network config depends on the distro (netplan, NetworkManager, networkd); use `netplan try` or a rollback plan on remote hosts.
- `dig` queries DNS directly; `getent hosts` resolves like applications do. On systemd-resolved hosts, use `resolvectl`.
- "Refused" means reachable but nothing listening; "timeout" means packets are being dropped.
- nftables is the modern netfilter interface; ufw and firewalld are front ends. Allow SSH before enabling a default-deny policy.
- Use ed25519 keys, `~/.ssh/config`, `ProxyJump` rather than agent forwarding, and `rsync` for transfers.
- Harden sshd with drop-ins, `sshd -t`, and a second session open; disable passwords and root login.

Part 7 — Bash Scripting: Basics to Production Quality

Shell scripts glue DevOps together: bootstrap scripts, CI steps, container entrypoints, backup jobs, health checks. Bash is everywhere, starts instantly and talks natively to every CLI tool. It is also full of sharp edges. This Part teaches the language and, more importantly, the habits that make scripts safe to run unattended.

> **Tip:** Install **ShellCheck** (`apt install shellcheck`, `dnf install ShellCheck`) and run it on every script, ideally in CI. It catches most of the bugs described in this Part.

7.1 The First Script

```
#!/usr/bin/env bash
# hello.sh - greet a user
echo "Hello, ${1:-world}! Today is $(date +%A)."
```

```
chmod +x hello.sh
./hello.sh Abdul        # Hello, Abdul! Today is Monday.
bash hello.sh           # runs without execute bit
bash -x hello.sh        # trace each command (debugging)
```

The **shebang** line tells the kernel which interpreter to use. `#!/usr/bin/env bash` finds bash in `PATH` (portable); `#!/bin/bash` is fine on Linux. Use `#!/bin/sh` only if you write strictly POSIX code, because on Debian/Ubuntu `/bin/sh` is dash and on Alpine it's BusyBox ash.

7.2 Variables and Expansion

```
name="web01"              # no spaces around '='
readonly REGION="eu-west-1"
count=3
echo "$name has $count cores"
echo "${name}-backup"     # braces separate the name from following text
files=$(ls /etc | wc -l)  # command substitution
today=$(date +%F)
```

Scoping: variables are global to the script unless declared `local` inside a function. `export VAR` makes it part of the environment passed to child processes.

7.2.1 Parameter expansion (learn these, they replace many sed/awk calls)

Syntax	Result
`${var:-default}`	Value, or `default` if unset/empty
`${var:=default}`	Same, and assign it
`${var:?message}`	Value, or exit with error `message` if unset/empty
`${var:+alt}`	`alt` if var is set, else empty
`${#var}`	Length
`${var#prefix}` / `${var##prefix}`	Strip shortest/longest prefix pattern
`${var%suffix}` / `${var%%suffix}`	Strip shortest/longest suffix pattern
`${var/old/new}` / `${var//old/new}`	Replace first / all
`${var:offset:len}`	Substring
`${var^^}` / `${var,,}`	Upper / lower case

```
path="/var/log/nginx/access.log.gz"
echo "${path##*/}"      # access.log.gz   (basename)
echo "${path%/*}"       # /var/log/nginx  (dirname)
echo "${path%.gz}"      # /var/log/nginx/access.log
file="report.tar.gz"; echo "${file%%.*}"   # report
tag="v1.4.2"; echo "${tag#v}"              # 1.4.2
env="Prod"; echo "${env,,}"                # prod
```

7.2.2 Special parameters

Parameter	Meaning
`$0`	Script name
`$1` .. `$9` , `${10}`	Positional arguments
`$#`	Number of arguments
`"$@"`	All arguments, each separately quoted (use this)
`"$*"`	All arguments as one string
`$?`	Exit status of last command
`$$`	PID of the script
`$!`	PID of last background job

```
for arg in "$@"; do echo "arg: $arg"; done   # handles spaces correctly
```

7.3 Conditions

`if` runs a command and branches on its exit status. `[[ ... ]]` is bash's test command (safer than `[`: no word-splitting, supports `&&`, `||`, `=~`).

```
if [[ -f /etc/nginx/nginx.conf ]]; then
  echo "nginx config present"
elif [[ -d /etc/httpd ]]; then
  echo "apache"
else
  echo "no web server"
fi
```

Test	True if
`-e f` / `-f f` / `-d f` / `-L f`	Exists / regular file / directory / symlink
`-r` / `-w` / `-x`	Readable / writable / executable
`-s f`	Size > 0
`-z s` / `-n s`	String empty / non-empty
`a == b`, `a != b`	String compare (`==` does glob matching inside `[[ ]]`)
`s =~ regex`	Regex match; captures in `BASH_REMATCH`
`-eq -ne -lt -le -gt -ge`	Integer compare
`f1 -nt f2`	f1 newer than f2

```
(( disk_pct > 90 )) && echo "disk critical"     # arithmetic context, no $ needed
[[ $version =~ ^v([0-9]+)\.([0-9]+) ]] && echo "major=${BASH_REMATCH[1]}"
if grep -q "^ok" /tmp/status; then echo healthy; fi   # any command works in if
command -v docker >/dev/null || { echo "docker missing" >&2; exit 1; }
```

7.3.1 case

```
case "$1" in
  start)   start_service ;;
  stop)    stop_service ;;
  restart) stop_service; start_service ;;
  status|st) show_status ;;
  *.tar.gz) extract "$1" ;;
  *) echo "Usage: $0 {start|stop|restart|status}" >&2; exit 2 ;;
esac
```

7.4 Loops

```
for host in web1 web2 web3; do
  ssh "$host" uptime
done

for i in {1..5}; do echo "attempt $i"; done
for ((i = 0; i < 10; i += 2)); do echo "$i"; done

for f in /var/log/app/*.log; do
  [[ -e $f ]] || continue          # glob matched nothing
  gzip "$f"
done

# Read a file line by line (the correct way)
while IFS= read -r line; do
  echo "line: $line"
done < hosts.txt

# Read command output without a subshell losing variables
count=0
while IFS= read -r pod; do
  ((++count))
done < <(kubectl get pods -o name)
echo "$count pods"

# Retry until success
until curl -fsS http://localhost:8080/health >/dev/null; do
  echo "waiting..."; sleep 2
done
```

`break` exits a loop; `continue` skips to the next iteration. `IFS= read -r` preserves leading whitespace and backslashes.

> **Warning:** `cat file | while read ...; do count=$((count+1)); done` runs the loop in a subshell, so `count` is lost afterwards. Use redirection `< file` or process substitution `< <(cmd)`.

7.5 Functions

```
log() {
  local level=$1; shift
  printf '%s [%s] %s\n' "$(date -u +%FT%TZ)" "$level" "$*" >&2
}

is_port_open() {
  local host=$1 port=$2
  timeout 2 bash -c "</dev/tcp/$host/$port" 2>/dev/null
```

```
}

if is_port_open db.internal 5432; then
  log INFO "database reachable"
else
  log ERROR "database unreachable"
fi
```

- Functions receive arguments as `$1`, `$2`, `"$@"`.
- `return N` sets the exit status (0–255); to return *data*, print it and capture with `$(...)`.
- Always declare internal variables `local`.
- Send diagnostics to stderr (`>&2`) so stdout stays clean for data.

7.6 Arrays

```
# Indexed arrays
servers=(web1 web2 "web 3")
echo "${servers[0]}"           # web1
echo "${#servers[@]}"          # 3
servers+=(web4)
for s in "${servers[@]}"; do echo "$s"; done    # quoted: "web 3" stays one item
echo "${servers[@]:1:2}"       # slice

# Build command arguments safely
args=(--region "$REGION" --output json)
[[ -n $PROFILE ]] && args+=(--profile "$PROFILE")
aws ec2 describe-instances "${args[@]}"

# Read lines into an array
mapfile -t users < <(cut -d: -f1 /etc/passwd)

# Associative arrays (bash 4+)
declare -A port=([web]=80 [api]=8080 [db]=5432)
port[cache]=6379
for svc in "${!port[@]}"; do echo "$svc -> ${port[$svc]}"; done
```

Using an array for command arguments is the correct alternative to building a command string and running `eval`.

7.7 Strict Mode and Error Handling

By default bash ignores failures and keeps going, which in automation means a failed `cd` followed by `rm -rf *` in the wrong directory. Start every script with:

```
#!/usr/bin/env bash
set -Eeuo pipefail
```

```
IFS=$'\n\t'
```

Option	Effect
`-e` (errexit)	Exit when a command fails
`-u` (nounset)	Error on unset variables (catches typos like `$DIRR`)
`-o pipefail`	A pipeline fails if *any* command in it fails, not just the last
`-E` (errtrace)	ERR traps are inherited by functions and subshells
`IFS=$'\n\t'`	Word-split only on newline/tab, not spaces (optional, debated)

Why `pipefail` matters:

```
set -e
curl -fsS https://bad.example.com/data | jq .   # without pipefail: jq succeeds on empty input, script continues
```

Know the limits of `set -e`. It is **not** triggered for commands in `if` / `while` conditions, commands before `&&` / `||`, or (without `inherit_errexit`) inside `$(...)` assignments combined with `local`. Write explicit checks where it counts:

```
local out              # declare separately...
out=$(some_cmd)        # ...so the failure isn't masked by 'local'

if ! systemctl restart myapp; then
  log ERROR "restart failed"; exit 1
fi

value=$(grep -c foo file || true)   # explicitly allow failure (grep exits 1 on no match)
```

7.7.1 traps

`trap` runs code when the script exits or receives a signal. It's how you guarantee cleanup.

```
tmpdir=$(mktemp -d)
cleanup() {
  local rc=$?
  rm -rf -- "$tmpdir"
  (( rc != 0 )) && log ERROR "failed with exit code $rc"
  exit "$rc"
}
trap cleanup EXIT
trap 'log ERROR "error on line $LINENO: $BASH_COMMAND"' ERR
trap 'log WARN "interrupted"; exit 130' INT TERM
```

Use `mktemp` for temporary files; never hard-code `/tmp/myscript.tmp` (races, symlink attacks, collisions between concurrent runs).

7.7.2 Locking

Prevent two copies running at once (cron overlap, double deploys):

```
exec 9>/run/lock/deploy.lock
flock -n 9 || { echo "another deploy is running" >&2; exit 1; }
```

7.8 Argument Parsing with getopts

```
usage() {
  cat <<EOF
Usage: ${0##*/} [-e env] [-n] [-v] <service>
  -e ENV   target environment (default: staging)
  -n       dry run
  -v       verbose
EOF
}

env="staging" dry_run=0 verbose=0
while getopts ":e:nvh" opt; do
  case $opt in
    e) env=$OPTARG ;;
    n) dry_run=1 ;;
    v) verbose=1 ;;
    h) usage; exit 0 ;;
    :) echo "Option -$OPTARG requires an argument" >&2; usage; exit 2 ;;
    \?) echo "Unknown option -$OPTARG" >&2; usage; exit 2 ;;
  esac
done
shift $((OPTIND - 1))
service=${1:?$(usage)}
(( verbose )) && set -x
```

`getopts` handles short options only. For long options (`--env`), write a `while (( $# )); do case $1 in --env) env=$2; shift 2;; ... esac; done` loop.

7.9 Useful Idioms

```
# Run a command with retries and backoff
retry() {
  local -i max=$1 n=1; shift
  until "$@"; do
    (( n >= max )) && { log ERROR "failed after $n attempts: $*"; return 1; }
```

```
    log WARN "attempt $n failed, retrying in $((2 ** n))s"
    sleep $((2 ** n)); ((n++))
  done
}
retry 5 curl -fsS https://api.example.com/health

# Require root
(( EUID == 0 )) || { echo "run as root" >&2; exit 1; }

# Require commands
for c in curl jq aws; do command -v "$c" >/dev/null || { echo "missing $c" >&2; exit 1; };
done

# Script's own directory (works regardless of cwd)
SCRIPT_DIR=$(cd -- "$(dirname -- "${BASH_SOURCE[0]}")" && pwd)

# Confirm dangerous actions
read -rp "Delete $target? [y/N] " ans
[[ $ans == [yY] ]] || exit 1

# Default config file, override with env var
CONFIG=${CONFIG:-/etc/myapp/config.env}
[[ -r $CONFIG ]] && source "$CONFIG"

# printf over echo for anything non-trivial (portable, no -e surprises)
printf '%-20s %5d\n' "$name" "$count"

# Timing
start=$SECONDS; do_work; echo "took $((SECONDS - start))s"

# Parallel work with a limit
xargs -P 4 -I{} ./process.sh {} < items.txt
```

7.10 Real DevOps Scripts

7.10.1 Disk usage alert

```
#!/usr/bin/env bash
# disk-alert.sh - warn when any filesystem exceeds a threshold
set -Eeuo pipefail

THRESHOLD=${THRESHOLD:-85}
WEBHOOK=${WEBHOOK:-}           # e.g. Slack/Teams incoming webhook URL
host=$(hostname -f 2>/dev/null || hostname)

alerts=()
while read -r target pcent; do
  pct=${pcent%\%}
  if (( pct >= THRESHOLD )); then
    alerts+=("$target at ${pct}%")
```

```
  fi
done < <(df --output=target,pcent -x tmpfs -x devtmpfs -x overlay -x squashfs | tail -n +2)

(( ${#alerts[@]} == 0 )) && exit 0

msg="Disk alert on $host: $(printf '%s; ' "${alerts[@]}")"
logger -t disk-alert "$msg"
echo "$msg" >&2

if [[ -n $WEBHOOK ]]; then
  curl -fsS -m 10 -H 'Content-Type: application/json' \
       -d "$(jq -n --arg t "$msg" '{text: $t}')" "$WEBHOOK" >/dev/null
fi
exit 1
```

Note the use of `jq -n --arg` to build JSON: string concatenation breaks as soon as the message contains a quote.

7.10.2 Backup with rotation

```
#!/usr/bin/env bash
# backup.sh - archive a directory, keep N days, upload to S3
set -Eeuo pipefail

SRC=${1:?usage: backup.sh <dir> [bucket]}
BUCKET=${2:-}
DEST=/var/backups/$(basename "$SRC")
KEEP_DAYS=${KEEP_DAYS:-7}
stamp=$(date -u +%Y%m%dT%H%M%SZ)
archive="$DEST/$(basename "$SRC")-$stamp.tar.zst"

log() { printf '%s %s\n' "$(date -u +%FT%TZ)" "$*" >&2; }

exec 9>"/run/lock/backup-$(basename "$SRC").lock"
flock -n 9 || { log "backup already running"; exit 0; }

mkdir -p "$DEST"
tmp=$(mktemp "$DEST/.partial.XXXXXX")
trap 'rm -f -- "$tmp"' EXIT

log "archiving $SRC"
tar --zstd -cf "$tmp" -C "$(dirname "$SRC")" "$(basename "$SRC")"
mv -- "$tmp" "$archive"                          # atomic: never leave half-written backups
sha256sum "$archive" > "$archive.sha256"
log "created $archive ($(du -h "$archive" | cut -f1))"

if [[ -n $BUCKET ]]; then
  aws s3 cp --only-show-errors "$archive" "s3://$BUCKET/$(hostname)/"
  aws s3 cp --only-show-errors "$archive.sha256" "s3://$BUCKET/$(hostname)/"
  log "uploaded to s3://$BUCKET"
fi
```

```
find "$DEST" -name '*.tar.zst*' -mtime +"$KEEP_DAYS" -print -delete | sed 's/^/pruned: /' >&2
```

7.10.3 Rolling restart across hosts

```
#!/usr/bin/env bash
# rolling-restart.sh - restart a service one host at a time, stop on first failure
set -Eeuo pipefail

service=${1:?service name}; shift
hosts=("$@")
(( ${#hosts[@]} )) || { echo "usage: $0 <service> host1 [host2...]" >&2; exit 2; }

for h in "${hosts[@]}"; do
  echo ">>> $h: restarting $service"
  ssh -o BatchMode=yes "$h" "sudo systemctl restart '$service'"
  for i in {1..30}; do
    if ssh -o BatchMode=yes "$h" "systemctl is-active --quiet '$service'" &&
       ssh -o BatchMode=yes "$h" "curl -fsS -m 2 http://localhost:8080/health >/dev/null";
then
      echo ">>> $h healthy"; break
    fi
    (( i == 30 )) && { echo "!!! $h failed health check, aborting" >&2; exit 1; }
    sleep 2
  done
done
echo "all hosts restarted"
```

At this scale, you'd normally reach for Ansible (`serial: 1`), covered in Part 9, but understanding the logic helps you design any rollout.

7.10.4 Container entrypoint

```
#!/bin/sh
# docker-entrypoint.sh - POSIX sh because Alpine/distroless-like images may lack bash
set -eu

: "${APP_PORT:=8080}"
if [ -n "${DB_HOST:-}" ]; then
  echo "waiting for $DB_HOST:${DB_PORT:-5432}"
  i=0
  until nc -z "$DB_HOST" "${DB_PORT:-5432}"; do
    i=$((i + 1)); [ "$i" -ge 30 ] && { echo "db not reachable" >&2; exit 1; }
    sleep 1
  done
fi

# exec replaces the shell so the app becomes PID 1 and receives SIGTERM directly
exec "$@"
```

The `exec "$@"` line is essential: without it, the shell stays PID 1, may not forward SIGTERM, and `docker stop` waits for the timeout and then SIGKILLs your app.

7.11 When Not to Use Bash

Switch to Python (or Go) when you need: data structures beyond flat arrays, JSON manipulation beyond a few `jq` calls, unit tests, complex error handling, HTTP APIs with pagination/retries, or the script passes ~200 lines. Bash is best as glue between existing tools.

7.12 Lab: Write a Production-Grade Health Check Script

Goal: build `svc-check.sh` that checks a list of services and endpoints, supports options, logs properly, cleans up, and passes ShellCheck.

1. Create a config file listing checks:

`bash mkdir -p ~/lab7 && cd ~/lab7 cat > checks.conf <<'EOF' # type target unit ssh unit cron tcp 127.0.0.1:22 http https://example.com EOF`

2. Write the script skeleton with strict mode, logging and a trap:

```bash cat > svc-check.sh <<'EOF' #!/usr/bin/env bash set -Eeuo pipefail

CONF="./checks.conf"; TIMEOUT=3; QUIET=0 log() { (( QUIET )) || printf '%s %-5s %s\n' "$(date +%T)" "$1" "${*:2}" >&2; } trap 'log ERROR "line $LINENO: $BASH_COMMAND"' ERR

usage() { echo "Usage: ${0##/} [-c conf] [-t timeout] [-q]"; } while getopts ":c:t:qh" o; do case $o in c) CONF=$OPTARG ;; t) TIMEOUT=$OPTARG ;; q) QUIET=1 ;; h) usage; exit 0 ;; ) usage >&2; exit 2 ;; esac done [[ -r $CONF ]] || { log ERROR "cannot read $CONF"; exit 2; }

check_unit() { systemctl is-active --quiet "$1"; } check_tcp() { local h=${1%:} p=${1##:}; timeout "$TIMEOUT" bash -c "/dev/null; } check_http() { curl -fsS -o /dev/null -m "$TIMEOUT" "$1"; }

declare -A results=() failed=0 while read -r type target; do [[ -z ${type:-} || $type == #* ]] && continue if "check_$type" "$target"; then results["$type:$target"]=OK; log INFO "OK $type $target" else results["$type:$target"]=FAIL; log WARN "FAIL $type $target"; ((++failed)) fi done < "$CONF"

printf '%-40s %s\n' CHECK RESULT for k in "${!results[@]}"; do printf '%-40s %s\n' "$k" "${results[$k]}"; done | sort exit $(( failed > 0 )) EOF chmod +x svc-check.sh ```

3. Run it, then break something and run again:

`bash ./svc-check.sh; echo "exit=$?" echo "tcp 127.0.0.1:9" >> checks.conf ./svc-check.sh -t 1; echo "exit=$?" ./svc-check.sh -q -c /nonexistent; echo "exit=$?"`

4. Why `((++failed))` and not `((failed++))`? Under `set -e`, `((failed++))` evaluates to 0 the first time (post-increment returns the old value), which counts as failure and would exit the script. Test it to see for yourself.
```

5. Run ShellCheck and fix anything it reports: `shellcheck svc-check.sh`.
6. Add a `-j` option that prints results as JSON using `jq -n` (hint: build with `--arg` per item, or print `key<TAB>value` lines and pipe to `jq -R -n '[inputs | split("\t") | {check: .[0], result: .[1]}]'`).
7. Schedule it every 5 minutes with a systemd timer (reuse the pattern from Part 4) and view results with `journalctl -u svc-check`.

Key takeaways

- Start scripts with `#!/usr/bin/env bash` and `set -Eeuo pipefail`; know where `set -e` doesn't apply.
- Quote every expansion; use `"$@"` and arrays for argument lists instead of strings and `eval`.
- Parameter expansion (`${var:-x}`, `${var##*/}`, `${var:?}`) replaces many external commands and protects against empty variables.
- Use `[[ ]]` for tests, `(( ))` for arithmetic, `case` for dispatch, `while IFS= read -r` for lines.
- `trap ... EXIT` plus `mktemp` guarantees cleanup; `flock` prevents concurrent runs; write to a temp file then `mv` for atomic output.
- Logs go to stderr, data to stdout; exit codes are your API.
- In container entrypoints, finish with `exec "$@"` so the app gets signals.
- Lint with ShellCheck; move to Python when the script outgrows glue.

Part 8 — Performance, Troubleshooting Tools and Kernel Tuning

"The server is slow" is a symptom, not a diagnosis. Senior engineers stand out by having a *method*: they quickly narrow down which resource is the bottleneck, prove it with numbers, and only then change something. This Part gives you that method, the tools behind it, and the kernel knobs you'll actually tune.

8.1 A Method: USE

Brendan Gregg's **USE method**: for every resource, check

- **Utilization**: how busy is it (% time busy, or % capacity used)?
- **Saturation**: is work queuing because it's overloaded (run queue, swap, I/O wait queue)?
- **Errors**: are there error events (disk errors, dropped packets, OOM kills)?

Resource	Utilization	Saturation	Errors
CPU	`mpstat -P ALL`, `top` (%us+%sy)	load avg vs cores, `vmstat r` column	rare (`dmesg` MCE)
Memory	`free -h` (used vs available)	swapping (`vmstat si/so`), PSI, OOM kills	`dmesg` OOM, failed allocations
Disk	`iostat -x` `%util`	`aqu-sz`, `await`	`dmesg` I/O errors, `smartctl`
Network	`sar -n DEV` throughput vs link speed	drops, retransmits (`nstat`, `ss -ti`)	`ip -s link` errors
File descriptors	`lsof`, `/proc/sys/fs/file-nr`	"Too many open files"	EMFILE in app logs

8.1.1 The first 60 seconds

When you land on a sick box, run these in order (each takes seconds):

```
uptime                    # load averages: rising or falling?
dmesg -T | tail -20       # OOM kills, disk errors, NIC resets
vmstat 1 5                # run queue, swap, I/O wait, context switches
mpstat -P ALL 1 3         # is one CPU pegged (single-threaded app)?
pidstat 1 3               # which processes use CPU
iostat -xz 1 3            # disk latency and utilization
free -h                   # memory and cache
sar -n DEV 1 3            # network throughput
sar -n TCP,ETCP 1 3       # connection rates, retransmits
top                       # overall picture, sorted by CPU (press M for memory)
```

`mpstat`, `pidstat`, `iostat` and `sar` come from the **sysstat** package. Install it everywhere, and enable its collector (`systemctl enable --now sysstat`) so `sar` has history you can query after an incident.

8.2 CPU and Load

8.2.1 Load average

```
uptime
```

```
 10:42:11 up 12 days,  3:01,  2 users,  load average: 7.82, 4.10, 2.03
```

The three numbers are exponentially damped averages over 1, 5 and 15 minutes of the number of tasks that are **runnable (R) or in uninterruptible sleep (D)**. Interpretations:

- Compare with CPU count (`nproc`). Load 7.8 on 8 cores is busy but OK; on 2 cores it's heavily saturated.
- 1-minute much higher than 15-minute: problem is starting now. The reverse: it's recovering.
- On Linux, load includes `D` state, so high load with **idle CPUs** usually means tasks blocked on disk or NFS, not CPU pressure. Check `vmstat`'s `b` column and `iostat`.

8.2.2 top and htop

```
top - 10:42:15 up 12 days, load average: 7.82, 4.10, 2.03
Tasks: 212 total,   9 running, 203 sleeping,   0 stopped,   0 zombie
%Cpu(s): 71.2 us,  6.1 sy,  0.0 ni, 20.3 id,  1.9 wa,  0.0 hi,  0.4 si,  0.1 st
MiB Mem :  15876.4 total,    412.3 free,  11020.1 used,   4444.0 buff/cache
MiB Swap:      0.0 total,      0.0 free,      0.0 used.   4501.7 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
   2210 app       20   0   12.1g   8.2g  31440 S 385.0  52.9  91:12.33 java
```

CPU fields: `us` user code, `sy` kernel, `ni` niced, `id` idle, `wa` waiting for I/O (CPU idle while I/O outstanding), `hi/si` interrupts, `st` **steal** (hypervisor gave your vCPU's time to someone else; high steal on cloud burstable instances means you've exhausted CPU credits or have a noisy neighbour).

`%CPU` is per core, so 385% means almost four cores busy. Useful keys in top: `1` per-CPU view, `P` / `M` sort by CPU/memory, `c` full command, `H` threads, `k` kill. **htop** is friendlier: tree view (`F5`), filtering, and colour-coded meters.

8.2.3 vmstat

```
vmstat 1 5
```

```
procs -----------memory---------- ---swap-- -----io---- -system-- ------cpu-----
 r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa st
 9  0      0 421900  81200 4470000    0    0     4   120 3100 5400 71  6 21  2  0
12  3      0 398000  81200 4471000    0    0  8200  9100 4200 7900 65  8  9 18  0
```

- `r` : runnable tasks (consistently > CPU count = CPU saturation)
- `b` : blocked in uninterruptible sleep (I/O)
- `si/so` : swap in/out per second (any sustained non-zero = memory pressure)
- `bi/bo` : blocks read/written
- `cs` : context switches (very high values can indicate lock contention or too many threads)

8.2.4 Finding what the CPU is doing

```
pidstat -u -p 2210 1          # per-process CPU over time
pidstat -t -p 2210 1          # per-thread
top -H -p 2210                # threads of one process
sudo perf top                 # live: hottest functions system-wide
sudo perf top -p 2210
sudo perf record -F 99 -g -p 2210 -- sleep 30   # sample stacks for 30s
sudo perf report --stdio | head -50
```

`perf` (package `linux-tools-$(uname -r)` on Ubuntu, `perf` on RHEL) samples where CPUs spend time, including inside the kernel. Rendering `perf script` output as a **flame graph** is the standard way to visualise hot code paths. For JIT runtimes (Java, Node) you need symbol maps (e.g. async-profiler for Java).

8.3 Memory

8.3.1 Reading free correctly

```
free -h
```

```
               total        used        free      shared  buff/cache   available
Mem:            15Gi        10Gi       412Mi       210Mi       4.3Gi       4.4Gi
Swap:          2.0Gi          0B       2.0Gi
```

Linux uses otherwise idle RAM as **page cache** for file data, so “free” is always low on a healthy server. **`available`** estimates how much memory can be given to applications without swapping (free + reclaimable cache). Watch `available`, not `free`.

```
grep -E 'MemAvailable|Dirty|Slab|SReclaimable|Committed_AS' /proc/meminfo
ps -eo pid,user,rss,vsz,cmd --sort=-rss | head     # RSS = resident memory in KiB
smem -tk                                            # proportional set size (if installed)
cat /proc/2210/status | grep -E 'VmRSS|VmSwap'
```

VSZ/VIRT is address space reserved (often huge and meaningless); **RSS/RES** is physical memory currently used (shared pages are counted in every process that maps them).

8.3.2 The OOM killer

Linux overcommits memory: `malloc` usually succeeds even if RAM isn't available, because many programs never touch all they allocate. When memory really runs out and nothing more can be reclaimed, the kernel's **OOM killer** picks a process (highest `oom_score`, mostly driven by memory size) and kills it with SIGKILL.

```
dmesg -T | grep -iE 'out of memory|oom-kill|killed process'
journalctl -k --since "1 hour ago" | grep -i oom
```

```
[Mon Oct 12 10:31:07 2026] Out of memory: Killed process 2210 (java) total-vm:12688128kB,
anon-rss:8601220kB, file-rss:0kB, shmem-rss:0kB, UID:1001 pgtables:17220kB oom_score_adj:0
```

Within a cgroup (a container or a systemd unit with `MemoryMax=`), the limit triggers a *cgroup* OOM: only processes in that group are candidates. In Kubernetes this appears as `OOMKilled` with exit code 137 (128 + 9 = SIGKILL).

Influence victim selection:

```
cat /proc/2210/oom_score
echo -1000 | sudo tee /proc/$(pidof sshd | cut -d' ' -f1)/oom_score_adj   # never kill
# Persistently in a unit:  OOMScoreAdjust=-900
```

Many modern distros also run **systemd-oomd**, which kills cgroups early based on memory pressure (PSI) before the kernel OOM killer has to act.

8.3.3 Pressure Stall Information (PSI)

PSI shows the percentage of time tasks were stalled waiting for a resource. It's the most direct "is this resource a bottleneck" signal the kernel offers.

```
cat /proc/pressure/cpu /proc/pressure/memory /proc/pressure/io
```

```
some avg10=12.40 avg60=8.11 avg300=3.02 total=88231001
full avg10=4.10 avg60=2.20 avg300=0.80 total=21033011
```

`some` = at least one task stalled; `full` = all non-idle tasks stalled simultaneously. Per-cgroup PSI lives in `/sys/fs/cgroup/<group>/memory.pressure`.

8.4 Disk I/O

```
iostat -xz 1
```

```
Device   r/s    w/s   rkB/s   wkB/s  r_await w_await aqu-sz  %util
nvme1n1  820.0 1250.0 52480.0 160000.0  4.10   38.20  48.90  99.80
```

- `r_await` / `w_await`: average ms per I/O including queue time. For SSD/NVMe, single-digit ms is normal; tens to hundreds means saturation.
- `aqu-sz`: average queue length. Sustained high values = saturated.
- `%util`: time the device was busy. For parallel devices (SSD, NVMe, EBS), 100% doesn't necessarily mean maxed out; trust latency and queue size more.

Find the process doing I/O:

```
sudo iotop -oPa                 # accumulated I/O per process
pidstat -d 1                    # per-process read/write kB/s
cat /proc/2210/io
```

In the cloud, EBS volumes have IOPS and throughput limits (gp3 baseline 3,000 IOPS and 125 MB/s unless provisioned higher) and instances have their own EBS bandwidth caps. A perfectly healthy disk can look "slow" because you're hitting the provisioned limit; CloudWatch `VolumeQueueLength` and throughput metrics confirm it.

8.5 Processes, Files and System Calls

8.5.1 lsof

```
sudo lsof -p 2210 | head          # all open files of a process
sudo lsof -i :8080                # who's listening/connected on port 8080
sudo lsof -i TCP -s TCP:ESTABLISHED
sudo lsof /var/log/app.log        # who has this file open
sudo lsof +D /data                # everything under a directory (slow)
sudo lsof +L1                     # deleted-but-open files
ls /proc/2210/fd | wc -l          # count FDs quickly
```

8.5.2 strace

`strace` shows every system call a process makes: the boundary between the program and the kernel. When logs say nothing, strace often shows exactly which file is missing, which permission is denied, or what it's blocked on.

```
strace -f -o /tmp/trace.txt ./myapp          # trace new program, follow children
sudo strace -p 2210 -f -tt -T                # attach; timestamps and time per call
sudo strace -p 2210 -e trace=network         # only network syscalls
strace -e trace=openat,stat -f ./myapp 2>&1 | grep -E 'ENOENT|EACCES'
sudo strace -c -p 2210                       # summary table (Ctrl-C to stop)
```

```
openat(AT_FDCWD, "/etc/myapp/config.yaml", O_RDONLY) = -1 EACCES (Permission denied)
connect(7, {sa_family=AF_INET, sin_port=htons(5432), sin_addr=inet_addr("10.0.2.20")}, 16) =
-1 ETIMEDOUT (Connection timed out)
```

> **Warning:** strace uses ptrace and can slow the target dramatically (10x or more on syscall-heavy workloads). Use it briefly on production, or prefer low-overhead eBPF tools (`bpftrace`, the `bcc` tools such as `execsnoop`, `opensnoop`, `biolatency`, `tcpretrans`), which are the modern standard for production tracing.

8.5.3 Other quick tools

```
ltrace ./myapp              # library calls
pmap -x 2210 | tail -1      # memory map summary
cat /proc/2210/limits       # effective ulimits of a running process
cat /proc/2210/stack        # kernel stack (where a D-state process is stuck), root only
nstat -az | grep -i retrans # network counters
ss -ti dst 10.0.2.20        # per-connection TCP info: rtt, cwnd, retrans
```

8.6 Kernel Tuning with sysctl

`sysctl` reads and writes kernel parameters under `/proc/sys`. Change them only for a measured reason, and record why.

```
sysctl net.core.somaxconn
sudo sysctl -w net.core.somaxconn=4096        # runtime only
sysctl -a | grep tcp_keepalive
```

Persist in a drop-in file and load it:

```
# /etc/sysctl.d/90-app.conf
# Larger accept backlog for busy web servers (app must also request it in listen())
net.core.somaxconn = 4096
net.ipv4.tcp_max_syn_backlog = 8192
# More ephemeral ports for clients making many outbound connections
net.ipv4.ip_local_port_range = 10240 65535
# Detect dead peers faster than the default 2 hours
net.ipv4.tcp_keepalive_time = 300
# Required for routers, NAT, Kubernetes nodes and Docker
```

```
net.ipv4.ip_forward = 1
# Prefer dropping page cache over swapping application memory
vm.swappiness = 10
# Elasticsearch/OpenSearch requirement
vm.max_map_count = 262144
# Many inotify watchers (IDEs, file sync, kubelet, log shippers)
fs.inotify.max_user_watches = 524288
# System-wide max open files
fs.file-max = 2097152
```

```
sudo sysctl --system      # load all sysctl.d files
```

Parameter	Why you'd touch it
`vm.swappiness`	Lower to keep app memory in RAM (databases)
`vm.dirty_ratio`, `vm.dirty_background_ratio`	Control how much dirty page cache builds before writeback (smooth I/O spikes)
`vm.overcommit_memory`	`2` for strict accounting; Redis recommends `1`
`net.ipv4.tcp_tw_reuse`	Reuse TIME-WAIT sockets for outbound connections
`net.core.rmem_max` / `wmem_max`, `tcp_rmem` / `tcp_wmem`	Socket buffers for high-bandwidth/high-latency links
`net.netfilter.nf_conntrack_max`	"nf_conntrack: table full, dropping packet" on busy NAT/K8s nodes
`kernel.pid_max`	Hosts with huge numbers of processes/threads

> **Note:** In containers, many `net.*` sysctls are namespaced and set per pod/container (Kubernetes `securityContext.sysctls`), while `vm.*` and `fs.*` are host-wide and must be set on the node.

8.7 Resource Limits: ulimits

Per-process limits (`man 2 setrlimit`) cap resources like open files and processes. The classic production error is **"Too many open files"** (EMFILE), because the default soft limit for open files is often 1024.

```
ulimit -a           # current shell's limits
ulimit -n           # open files (soft)
ulimit -Hn          # hard limit
ulimit -n 65536     # raise soft up to the hard limit (current shell only)
cat /proc/2210/limits   # what a running process actually has
```

Where to set them persistently, and the trap:

- **Login sessions** (SSH, su): `/etc/security/limits.conf` or `/etc/security/limits.d/*.conf`, applied by PAM.

`conf # /etc/security/limits.d/90-app.conf app soft nofile 65536 app hard nofile 65536 app soft nproc 4096`

- **systemd services ignore limits.conf** entirely (no PAM login). Set limits in the unit:

`ini [Service] LimitNOFILE=65536 LimitNPROC=4096`

or defaults for all services in `/etc/systemd/system.conf` (`DefaultLimitNOFILE=`).

- **Containers**: `docker run --ulimit nofile=65536:65536`, or the container runtime's defaults.

> **Interview:** "You raised the limit in limits.conf, but the service still hits 'Too many open files'. Why?" Because the service is started by systemd, which doesn't go through PAM. Set `LimitNOFILE=` in the unit, restart, and verify with `cat /proc/<pid>/limits`.

8.8 Lab: Create and Diagnose Bottlenecks

Goal: generate CPU, memory, disk and FD problems on purpose and identify each one with the right tool. Use a disposable VM with 2+ vCPUs.

1. Install tools: `sudo apt install -y sysstat stress-ng htop iotop-c strace` (Rocky: `sudo dnf install -y sysstat stress-ng htop iotop strace` with EPEL enabled).

2. **CPU saturation.** Start more CPU workers than cores:

`bash stress-ng --cpu $(( $(nproc) * 2 )) --timeout 120s & uptime; vmstat 1 5; mpstat -P ALL 1 2; pidstat 1 2 | head -20`

Confirm: `r` column greater than `nproc`, `id` near 0, load rising above core count.

3. **I/O saturation.** High load, idle CPU:

`bash stress-ng --hdd 2 --hdd-bytes 1G --timeout 120s & vmstat 1 5 # look at b and wa iostat -xz 1 3 # await, aqu-sz, %util sudo iotop -oPa -n 3 -b | head -20 cat /proc/pressure/io`

4. **Memory pressure and OOM.** Put a memory hog in a limited scope so it OOMs in its own cgroup and doesn't hurt the system:

`bash sudo systemd-run --unit=memhog -p MemoryMax=200M -p MemorySwapMax=0 \ stress-ng --vm 1 --vm-bytes 400M --timeout 60s sleep 10 systemctl status memhog --no-pager journalctl -k --since "2 min ago" | grep -i -E 'oom|killed'`

5. **File descriptor exhaustion.**

```bash bash -c 'ulimit -n 64; for i in $(seq 1 100); do exec {fd}
```

Part 9 — Security Hardening, Containers Under the Hood, Cloud and Automation

This Part covers the advanced topics that distinguish a DevOps engineer from a sysadmin: hardening hosts to a standard, understanding the kernel features that containers are made of, bootstrapping cloud instances, and handing configuration over to automation.

9.1 Security Hardening

Hardening is about reducing attack surface and adding layers so a single failure doesn't become a breach. The principles: **least privilege, minimal installed software, patch fast, log everything important, and verify continuously**.

9.1.1 A baseline checklist

Area	Action
Patching	Unattended security updates; reboot or live-patch for kernel CVEs
Accounts	No shared accounts; root login disabled; sudo via groups; remove stale users
SSH	Keys only, no root, `AllowGroups`, restricted source IPs or SSM/VPN only
Services	Remove/disable anything not needed (`systemctl list-unit-files --state=enabled`, `ss -tulpn`)
Firewall	Default deny inbound; allow only required ports and sources
Filesystems	`/tmp` with `nodev,nosuid,noexec` where feasible; separate `/var/log` on long-lived hosts
MAC	SELinux enforcing / AppArmor enforcing
Logging	Persistent journal, auditd, logs shipped off-host
Integrity	AIDE file integrity; SUID audit; package verification (`rpm -Va`, `debsums`)
Time	NTP via chrony or systemd-timesyncd (TLS, Kerberos and log correlation depend on it)
Secrets	Not in scripts, env files world-readable, or shell history; use a secrets manager

9.1.2 Mandatory Access Control: SELinux

Normal Unix permissions are **discretionary**: the owner decides. **Mandatory Access Control** adds a system-wide policy that even root processes are subject to. If nginx is compromised, SELinux can still stop it from reading `/home` or connecting to the database port, because the policy says the `httpd_t` domain may not.

SELinux (default on RHEL, Rocky, Alma, Fedora; present on Amazon Linux 2023 in permissive mode) labels every process and file with a **context** `user:role:type:level`. Policy rules are mostly about **types**.

```
getenforce                          # Enforcing / Permissive / Disabled
sestatus
ls -Z /var/www/html                 # file contexts
ps -eZ | grep nginx                 # process domain: httpd_t
sudo setenforce 0                   # permissive until reboot (for diagnosis only!)
```

```
-rw-r--r--. root root unconfined_u:object_r:httpd_sys_content_t:s0 index.html
```

Three things fix 95% of SELinux problems:

1. **Wrong file label.** Files moved (not copied) keep their old label; custom paths have no rule.

`bash sudo semanage fcontext -a -t httpd_sys_content_t "/srv/site(/.*)?" sudo restorecon -Rv /srv/site`

2. **Booleans** toggle common policy options.

`bash getsebool -a | grep httpd sudo setsebool -P httpd_can_network_connect on # let nginx proxy to upstream apps`

3. **Non-standard ports.**

`bash sudo semanage port -l | grep http_port_t sudo semanage port -a -t http_port_t -p tcp 8081`

Diagnose denials:

```
sudo ausearch -m AVC -ts recent
sudo journalctl -t setroubleshoot      # human-readable suggestions, if setroubleshoot
installed
sudo audit2why < /var/log/audit/audit.log | tail
```

```
type=AVC msg=audit(1760262000.123:812): avc:  denied  { name_connect } for  pid=812
comm="nginx" dest=8080 scontext=system_u:system_r:httpd_t:s0
tcontext=system_u:object_r:http_cache_port_t:s0 tclass=tcp_socket permissive=0
```

> **Warning:** “Disable SELinux” is the most common bad advice on the internet. Use `setenforce 0` briefly to *confirm* SELinux is the cause, then fix the label, boolean or port and set it back to enforcing. As a last resort `audit2allow -M mypolicy` builds a custom module, but review what it allows.

9.1.3 AppArmor

Ubuntu and Debian (and SUSE) use **AppArmor**, which confines programs by **path-based profiles** rather than labels. Simpler to read, slightly less granular.

```
sudo aa-status                          # loaded profiles and their mode
sudo aa-complain /usr/sbin/nginx        # log but don't block
sudo aa-enforce /usr/sbin/nginx
ls /etc/apparmor.d/
sudo journalctl -k | grep -i apparmor   # DENIED messages
```

Container runtimes apply a default AppArmor profile (`docker-default`) to containers on Ubuntu hosts.

9.1.4 fail2ban

fail2ban watches logs for repeated failures (SSH brute force, web login abuse) and bans offending IPs via the firewall.

```
# /etc/fail2ban/jail.local
[DEFAULT]
bantime  = 1h
findtime = 10m
maxretry = 5
backend  = systemd
ignoreip = 127.0.0.1/8 10.0.0.0/8

[sshd]
enabled = true
```

```
sudo systemctl enable --now fail2ban
sudo fail2ban-client status sshd
sudo fail2ban-client set sshd unbanip 203.0.113.42
```

With key-only SSH, brute force can't succeed, but fail2ban still reduces log noise and load. Restricting port 22 at the security group is better still.

9.1.5 auditd

The Linux audit system records security-relevant events in the kernel: file access, syscalls, logins, sudo, changes to identity files. It's required by most compliance frameworks (PCI DSS, SOC 2, HIPAA, FedRAMP).

```
sudo systemctl enable --now auditd
sudo auditctl -l                            # active rules
sudo auditctl -w /etc/passwd -p wa -k identity
sudo auditctl -w /etc/sudoers -p wa -k sudoers
```

Persist rules in `/etc/audit/rules.d/`:

```
# /etc/audit/rules.d/50-devops.rules
-w /etc/passwd -p wa -k identity
-w /etc/shadow -p wa -k identity
-w /etc/group -p wa -k identity
```

```
-w /etc/sudoers -p wa -k sudoers
-w /etc/sudoers.d/ -p wa -k sudoers
-w /etc/ssh/sshd_config -p wa -k sshd
-a always,exit -F arch=b64 -S execve -F euid=0 -F auid>=1000 -F auid!=unset -k root-commands
```

```
sudo augenrules --load
sudo ausearch -k identity -i --start today
sudo aureport --login --summary -i
```

`auid` (audit UID, the "login UID") survives `sudo` and `su`, so audit logs show which human actually ran a root command.

9.1.6 CIS Benchmarks and compliance scanning

The **CIS Benchmarks** are consensus hardening guides per OS (Ubuntu 24.04, RHEL 9, Amazon Linux 2023…), with Level 1 (practical baseline) and Level 2 (stricter) profiles. Don't apply them by hand:

- **OpenSCAP**: `oscap xccdf eval --profile cis ...` with the SCAP Security Guide content (RHEL family), generating HTML reports and remediation scripts.
- **Ubuntu Security Guide**: `usg audit cis_level1_server` / `usg fix` (Ubuntu Pro).
- **Lynis**: `sudo lynis audit system` for a quick, distro-agnostic score and suggestions.
- **Ansible roles** (e.g. ansible-lockdown) or hardened cloud images (CIS Hardened AMIs).

Treat findings as a backlog: some controls break workloads (e.g. `noexec` on `/tmp` breaks some installers).

9.1.7 Patching and vulnerability response

- Subscribe to your distro's security notices (USN for Ubuntu, RHSA for Red Hat, ALAS for Amazon Linux).
- Check exposure: `apt list --upgradable`, `dnf updateinfo list --security`, `dnf updateinfo info CVE-2024-6387`.
- Kernel updates need a reboot unless you use **live patching** (Canonical Livepatch, kpatch on RHEL). Know your current kernel (`uname -r`) vs installed (`ls /boot`).
- In immutable setups, rebuild golden images (Packer) or container base images on every patch release and redeploy.

9.2 Namespaces and cgroups: the Bridge to Containers

A container is **not** a VM. It's an ordinary Linux process (or group of processes) that the kernel isolates with **namespaces** (what it can *see*) and constrains with **cgroups** (what it can *use*), started from a filesystem image, with capabilities dropped and a seccomp filter applied.

```
                Host kernel (shared by all containers)
+-------------------------------------------------------------+
|  Container A                    Container B                 |
|  PID ns: sees itself as PID 1   PID ns: own PID 1           |
```

```
|  NET ns: own eth0, IP, routes    NET ns: own eth0              |
|  MNT ns: overlayfs rootfs        MNT ns: different rootfs      |
|  UTS ns: own hostname            cgroup: memory.max=512M       |
|  cgroup: cpu.max=50%                                           |
+----------------------------------------------------------------+
```

9.2.1 Namespaces

Namespace	Isolates	Effect in a container
`pid`	Process IDs	App sees itself as PID 1; can't see host processes
`net`	Interfaces, IPs, routes, ports, firewall	Own `eth0`, can bind port 80 independently
`mnt`	Mount table	Own root filesystem
`uts`	Hostname, domain name	Own hostname
`ipc`	SysV IPC, POSIX message queues	No shared memory with host
`user`	UID/GID mappings	Root inside maps to unprivileged UID outside (rootless containers)
`cgroup`	View of the cgroup tree	Sees its own cgroup as root
`time`	Some clocks	Rarely used

```
ls -l /proc/$$/ns                 # namespaces of your shell
lsns                              # all namespaces on the system
sudo lsns -t net
```

Build a "container" by hand with `unshare`:

```
sudo unshare --pid --fork --mount-proc --uts --net bash
hostname container-demo; hostname    # changes only inside
ps aux                               # you are PID 1, only a few processes visible
ip -br link                          # only 'lo', and it's down
exit
```

Enter an existing container's namespaces from the host (great for debugging minimal images that have no shell tools):

```
pid=$(docker inspect -f '{{.State.Pid}}' web)
sudo nsenter -t "$pid" -n ss -tlnp        # host's ss binary, container's network namespace
sudo nsenter -t "$pid" -n tcpdump -ni eth0
```

Network namespaces with a virtual Ethernet pair, the basis of Docker's bridge networking and Kubernetes CNIs:

```
sudo ip netns add demo
sudo ip link add veth-host type veth peer name veth-demo
sudo ip link set veth-demo netns demo
sudo ip addr add 10.200.0.1/24 dev veth-host && sudo ip link set veth-host up
sudo ip netns exec demo ip addr add 10.200.0.2/24 dev veth-demo
sudo ip netns exec demo ip link set veth-demo up
sudo ip netns exec demo ip link set lo up
ping -c 2 10.200.0.2
sudo ip netns del demo
```

9.2.2 cgroups v2

Control groups organise processes into a hierarchy and apply resource limits and accounting per group. All current distros use the **unified cgroup v2** hierarchy mounted at `/sys/fs/cgroup`. systemd manages it: every service gets its own cgroup under `system.slice`, user sessions under `user.slice`.

```
stat -fc %T /sys/fs/cgroup        # cgroup2fs = v2
systemd-cgls                      # tree of cgroups and processes
systemd-cgtop                     # live usage
cat /proc/self/cgroup             # 0::/user.slice/user-1000.slice/session-3.scope
ls /sys/fs/cgroup/system.slice/nginx.service/
cat /sys/fs/cgroup/system.slice/nginx.service/memory.current
```

Key controller files:

File	Meaning
`cpu.max`	`quota period`, e.g. `50000 100000` = 0.5 CPU
`cpu.weight`	Relative share under contention (default 100)
`memory.max`	Hard limit; exceeding it triggers cgroup OOM
`memory.high`	Soft limit; throttles and reclaims above it
`memory.current`, `memory.stat`	Usage and breakdown
`memory.events`	Counts of `high`, `max`, `oom`, `oom_kill` events
`io.max`	Per-device IOPS/bandwidth limits
`pids.max`	Max processes (fork bomb protection)
`cpu.stat`	Includes `nr_throttled`, `throttled_usec`

Set limits through systemd rather than writing files directly:

```
sudo systemctl set-property nginx.service MemoryMax=512M CPUQuota=150%
sudo systemd-run --scope -p MemoryMax=100M -p CPUQuota=20% ./batch.sh   # ad-hoc
```

How this maps to containers: `docker run --memory=512m --cpus=1.5` writes `memory.max` and `cpu.max` in the container's cgroup. Kubernetes `resources.limits.memory` becomes `memory.max`; `limits.cpu` becomes `cpu.max` (CFS quota); `requests.cpu` becomes `cpu.weight`.

> **Interview:** "Why is my container's latency spiky even though average CPU is low?" CPU limits are enforced per scheduling period (100 ms by default). A multi-threaded app can burn its whole quota in the first 20 ms and is then **throttled** for the remaining 80 ms. Check `nr_throttled` and `throttled_usec` in `cpu.stat`. Fixes: raise or remove the CPU limit, or reduce thread counts.

9.2.3 The rest of a container

- **Root filesystem**: image layers stacked with **overlayfs** (read-only lower layers + writable upper layer). `mount | grep overlay` on a Docker host shows them.
- **Capabilities**: runtimes drop most capabilities by default; `--privileged` gives them all back (avoid).
- **seccomp**: a filter blocking dangerous syscalls (default Docker profile blocks several dozen, such as `kexec_load`, `mount` without CAP_SYS_ADMIN).
- **LSM**: SELinux/AppArmor labels confine the container further.
- **Runtimes**: `runc` / `crun` implement all of this per the OCI spec; containerd or CRI-O manage images and lifecycle; Docker and Kubernetes sit on top.

Since all containers share the host kernel, a kernel vulnerability or a container running as root with excessive privileges can mean host compromise. That's why you run as non-root, drop capabilities, use read-only root filesystems, and keep nodes patched. For stronger isolation, gVisor or microVMs (Firecracker, Kata Containers) add a separate kernel boundary.

9.3 Linux in the Cloud

9.3.1 Cloud images and cloud-init

Cloud VMs boot from images (AMIs on AWS) that are generic. **cloud-init** runs on first boot to personalise them: it reads **metadata** (instance ID, hostname, network, SSH public key) and **user data** you supplied, then sets hostname, creates users, installs SSH keys, grows the root partition, writes files, installs packages and runs scripts.

```
Boot -> cloud-init stages:
  init-local  (find datasource, configure network)
  init        (disk setup, growpart, SSH host keys, users/keys)
  config      (packages, write_files, ntp ...)
  final       (runcmd, user scripts, "cloud-init finished")
```

User data can be a shell script (starts with `#!`) or a **cloud-config** YAML (starts with `#cloud-config`):

```yaml
#cloud-config
hostname: web-01
```

```yaml
timezone: UTC
package_update: true
package_upgrade: true
packages:
  - nginx
  - jq
  - chrony
users:
  - default
  - name: deploy
    groups: [sudo]
    shell: /bin/bash
    sudo: "ALL=(ALL) NOPASSWD:ALL"
    ssh_authorized_keys:
      - ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAA... deploy@ci
write_files:
  - path: /etc/sysctl.d/90-web.conf
    content: |
      net.core.somaxconn = 4096
  - path: /var/www/html/index.html
    content: "<h1>web-01</h1>"
runcmd:
  - sysctl --system
  - systemctl enable --now nginx
final_message: "cloud-init done after $UPTIME seconds"
```

Debugging cloud-init:

```
cloud-init status --long
cloud-init status --wait          # block until finished (useful in provisioning scripts)
sudo cat /var/log/cloud-init-output.log   # stdout/stderr of your scripts
sudo less /var/log/cloud-init.log         # detailed internal log
sudo cloud-init query userdata            # what user data the instance received
sudo cloud-init schema --system           # validate cloud-config
sudo cloud-init clean --logs && sudo reboot   # re-run as if first boot (testing images)
```

9.3.2 EC2 specifics

User data on EC2 runs once, as root, at first boot, via cloud-init. A typical shell-style user data:

```
#!/bin/bash
set -euxo pipefail
exec > >(tee /var/log/user-data.log) 2>&1     # capture everything

dnf -y install nginx amazon-cloudwatch-agent
TOKEN=$(curl -sX PUT "http://169.254.169.254/latest/api/token" \
        -H "X-aws-ec2-metadata-token-ttl-seconds: 300")
IID=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
        http://169.254.169.254/latest/meta-data/instance-id)
```

```
echo "<h1>$IID</h1>" > /usr/share/nginx/html/index.html
systemctl enable --now nginx
```

Key points:

- The **Instance Metadata Service (IMDS)** at `169.254.169.254` provides instance identity, network info and **temporary IAM role credentials**. Require **IMDSv2** (token-based, shown above): it blocks the classic SSRF attack where a vulnerable web app is tricked into fetching credentials from IMDSv1.
- Applications should get AWS credentials from the **instance profile** (IAM role), never from access keys in files.
- Default users vary by AMI: `ec2-user` (Amazon Linux, RHEL), `ubuntu` (Ubuntu), `rocky` (Rocky), `admin` (Debian).
- **SSM Agent** (preinstalled on Amazon Linux and Ubuntu AMIs) enables Session Manager shells, Run Command and Patch Manager without opening port 22.
- Instance store volumes are ephemeral (lost on stop); EBS persists. EBS volumes are NVMe devices on Nitro instances; use `/dev/disk/by-id` or UUIDs.
- The **EC2 serial console** and "detach root volume, attach to rescue instance" are your recovery paths for an unbootable instance.

9.3.3 Immutable infrastructure

Mature teams avoid configuring long-lived servers by hand. They bake **golden images** (Packer + Ansible), launch them via Auto Scaling groups or Kubernetes node groups, keep user data minimal (fetch config, join cluster), and *replace* instances rather than patching them. Everything in this course still applies: you use it to build the image, debug the instance when it misbehaves, and write the automation.

9.4 Automation Hand-off: Ansible

Once you know how to configure a server by hand, the next step is never doing it by hand again. **Ansible** is the most common Linux configuration management tool: agentless (it uses SSH and Python on the target), declarative YAML, and **idempotent** (running it twice changes nothing the second time).

9.4.1 Concepts

Term	Meaning
Inventory	List of hosts and groups (static INI/YAML or dynamic from AWS etc.)
Module	A unit of work (`apt` , `dnf` , `copy` , `template` , `systemd` , `user` , `lineinfile`)
Task	One module call with arguments
Play / Playbook	Tasks mapped to host groups / a YAML file of plays
Handler	Task run only when notified by a change (e.g. restart nginx after config change)
Role	Reusable bundle of tasks, templates, handlers, defaults

Term	Meaning
Facts	Variables gathered from hosts (`ansible_facts['os_family']`)

9.4.2 Inventory and ad-hoc commands

```
# inventory.ini
[web]
web1 ansible_host=10.0.1.11
web2 ansible_host=10.0.1.12

[db]
db1 ansible_host=10.0.3.21

[all:vars]
ansible_user=ubuntu
ansible_ssh_common_args='-o ProxyJump=bastion'
```

```
pipx install ansible-core     # or: python3 -m pip install --user ansible
ansible -i inventory.ini all -m ping
ansible -i inventory.ini web -a "uptime"
ansible -i inventory.ini web -b -m apt -a "name=nginx state=present update_cache=true"
ansible -i inventory.ini all -m setup -a "filter=ansible_distribution*"
```

9.4.3 A playbook that encodes this course

```
# site.yml
- name: Baseline and web servers
  hosts: web
  become: true
  vars:
    app_user: deploy
    sysctl_settings:
      net.core.somaxconn: 4096
      vm.swappiness: 10

  tasks:
    - name: Install packages
      ansible.builtin.package:
        name: [nginx, chrony, fail2ban, sysstat]
        state: present

    - name: Create deploy user
      ansible.builtin.user:
        name: "{{ app_user }}"
        groups: sudo
        append: true
        shell: /bin/bash
```

```
    - name: Authorize deploy key
      ansible.posix.authorized_key:
        user: "{{ app_user }}"
        key: "{{ lookup('file', 'files/deploy.pub') }}"

    - name: Harden sshd
      ansible.builtin.copy:
        dest: /etc/ssh/sshd_config.d/10-hardening.conf
        content: |
          PermitRootLogin no
          PasswordAuthentication no
        mode: "0644"
        validate: /usr/sbin/sshd -t -f %s
      notify: Reload ssh

    - name: Kernel parameters
      ansible.posix.sysctl:
        name: "{{ item.key }}"
        value: "{{ item.value }}"
        sysctl_file: /etc/sysctl.d/90-ansible.conf
        reload: true
      loop: "{{ sysctl_settings | dict2items }}"

    - name: Nginx site from template
      ansible.builtin.template:
        src: templates/site.conf.j2
        dest: /etc/nginx/conf.d/site.conf
        mode: "0644"
        validate: nginx -t -c /etc/nginx/nginx.conf
      notify: Reload nginx

    - name: Services enabled and running
      ansible.builtin.systemd:
        name: "{{ item }}"
        enabled: true
        state: started
      loop: [nginx, chrony, fail2ban]

  handlers:
    - name: Reload ssh
      ansible.builtin.systemd:
        name: ssh
        state: reloaded
    - name: Reload nginx
      ansible.builtin.systemd:
        name: nginx
        state: reloaded
```

```
ansible-playbook -i inventory.ini site.yml --check --diff   # dry run with diffs
ansible-playbook -i inventory.ini site.yml --limit web1
ansible-playbook -i inventory.ini site.yml
```

Notes:

- Prefer specific modules over `shell` / `command`; modules are idempotent and report "changed" accurately.
- `validate:` runs a check against the candidate file *before* replacing the real one: a bad sshd config never lands.
- Handlers run once at the end even if notified many times.
- For rolling changes, add `serial: 1` (or `"25%"`) to the play, with a health-check task.
- Secrets go in **Ansible Vault** or are fetched from a secrets manager, never plaintext in Git.
- `sshd -t -f %s` checks the drop-in fragment on its own, which catches syntax errors; after the reload, confirm effective settings with `sshd -T`. The service is named `sshd` on the RHEL family and `ssh` on Ubuntu.

Beyond Ansible: Terraform/OpenTofu provisions the infrastructure (VPCs, instances), Packer builds images, Ansible configures them, and Kubernetes or ECS runs the containers. Linux knowledge sits under all of them.

9.5 Lab: Harden, Containerize by Hand, and Automate

Goal: apply hardening controls, build namespace/cgroup isolation by hand, and capture the setup as an Ansible playbook.

1. **Baseline scan.** On a fresh Ubuntu or Rocky VM:

```
bash sudo apt install -y lynis || sudo dnf install -y lynis sudo lynis audit system --quick | tee ~/lynis-before.txt | tail -30
```

Note the hardening index.

2. **auditd.** Install (`auditd` on Ubuntu, `audit` on RHEL), add the identity and sudoers rules from 9.1.5, load them, then trigger and find an event:

```
bash sudo augenrules --load sudo useradd audittest sudo ausearch -k identity -i | tail -20 sudo userdel audittest
```

3. **fail2ban.** Install, create the jail from 9.1.4, then from another machine attempt 6 failed logins (`ssh nosuchuser@VM`) and confirm the ban with `sudo fail2ban-client status sshd`. Unban yourself.

4. **MAC.** On Rocky: install nginx, move a file into `/usr/share/nginx/html` with `mv` from your home directory, observe the 403, find the AVC with `ausearch -m AVC -ts recent`, and fix it with `restorecon`. On Ubuntu: run `sudo aa-status` and identify which processes are confined.

5. **Namespaces by hand.**

```
bash sudo unshare --pid --fork --mount-proc --uts bash -c 'hostname box; hostname; ps aux' lsns -t pid
```

6. **cgroups by hand.** Run a CPU hog limited to 20% of one CPU and watch throttling:

```bash sudo systemd-run --unit=hog -p CPUQuota=20% sh -c 'while :; do :; done' top -b -n 1 | head -15 # ~20% CPU cat /sys/fs/cgroup/system.slice/hog.service/cpu.max # 20000 100000 grep throttled /sys/fs/cgroup/system.slice/hog.service/cpu.stat sudo systemctl stop hog```

7. **Network namespace.** Run the veth example from 9.2.1 and ping between host and namespace. Then run `python3 -m http.server 8000` inside the namespace (`sudo ip netns exec demo python3 -m http.server 8000 &`) and curl it from the host at `10.200.0.2:8000`.

8. **cloud-init.** If you're on a cloud VM, run `cloud-init status --long`, read `/var/log/cloud-init-output.log`, and `sudo cloud-init query userdata`. Launch a second instance with the cloud-config from 9.3.1 and verify each item was applied.

9. **Ansible.** From your workstation, write `inventory.ini` with your VM and run:

```bash ansible -i inventory.ini all -m ping ansible-playbook -i inventory.ini site.yml --check --diff ansible-playbook -i inventory.ini site.yml ansible-playbook -i inventory.ini site.yml # second run: changed=0 (idempotent)```

(Create `files/deploy.pub` and a minimal `templates/site.conf.j2` first; adjust the `sudo` group and `ssh` service name for RHEL.)

10. Re-run Lynis and compare the hardening index with step 1.

Key takeaways

- Harden with layers: patching, minimal services, key-only SSH, firewall, MAC, auditing, and off-host logs. Use CIS/OpenSCAP/Lynis to measure rather than guess.
- SELinux problems are usually labels (`restorecon`, `semanage fcontext`), booleans (`setsebool -P`) or ports (`semanage port`). Don't disable it; fix the cause.
- auditd with `auid` tells you which human did what, even through sudo.
- Containers are processes with namespaces (visibility) plus cgroups (resource limits), an overlayfs root, dropped capabilities, and seccomp. `unshare`, `nsenter` and `/sys/fs/cgroup` let you see and debug this directly.
- CPU limits cause throttling within each 100 ms period; check `cpu.stat`.
- cloud-init personalises cloud images on first boot; its logs are `/var/log/cloud-init-output.log` and `/var/log/cloud-init.log`. Use IMDSv2 and instance roles on EC2.
- Ansible turns manual steps into idempotent, reviewable code; use modules, handlers, `validate:`, `--check --diff`, and Vault for secrets.

Part 10 — Real-World Troubleshooting, Interview Prep and Cheat Sheet

This final Part puts everything together. First, six incident scenarios with a diagnostic walkthrough for each: the order in which you'd look, what the output tells you, and how to fix and prevent the problem. Then a 45-question interview bank, and a dense command cheat sheet to keep at hand.

The general approach for every incident:

```
1. Scope      What exactly is broken? Since when? One host or many? What changed?
2. Observe    Gather evidence with read-only commands before changing anything.
3. Hypothesize  Form the simplest explanation consistent with ALL the evidence.
4. Test       Change one thing, verify the effect.
5. Fix + prevent  Restore service, then fix the root cause, add monitoring/automation.
6. Write it up  Timeline, cause, fix, follow-ups (blameless postmortem).
```

10.1 Scenario: Disk Full

Symptom: The application returns 500s; logs show `No space left on device`. Deploys fail.

Step 1: Which filesystem, and is it bytes or inodes?

```
df -hT
df -i
```

```
Filesystem     Type  Size  Used Avail Use% Mounted on
/dev/nvme0n1p1 ext4   30G   30G     0 100% /
/dev/nvme1n1   xfs   100G   41G   59G  41% /data
```

If `df -i` shows 100% `IUse%` with free bytes, it's inode exhaustion: count files per directory (`find / -xdev -type f | cut -d/ -f2-4 | sort | uniq -c | sort -rn | head`) and remove the swarm of small files (sessions, cache, mail queue).

Step 2: Where is the space?

```
sudo du -xh / --max-depth=1 2>/dev/null | sort -h | tail
sudo du -xh /var --max-depth=2 2>/dev/null | sort -h | tail
sudo find / -xdev -type f -size +500M -exec ls -lh {} + 2>/dev/null
```

Usual suspects: `/var/log` (an app logging in debug mode, or no logrotate), `/var/lib/docker` (images, stopped containers, container logs in `/var/lib/docker/containers/*/*-json.log`), the journal (`/var/log/journal`), core dumps (`/var/lib/systemd/coredump`), old kernels in `/boot`, package caches, forgotten backups or heap dumps in `/tmp` or home directories.

Step 3: If `du` totals far less than `df` reports, look for deleted-but-open files:

```
sudo lsof +L1 | sort -k7 -n | tail
```

Step 4: Free space safely.

```
sudo truncate -s 0 /var/log/app/debug.log      # not rm, if a process has it open
sudo journalctl --vacuum-size=500M
docker system df && docker system prune -af --filter "until=168h"   # check what it removes
first
sudo apt clean     # or: sudo dnf clean all
sudo apt autoremove --purge    # old kernels on Ubuntu
# For deleted-open files: restart the process, or truncate via /proc/<pid>/fd/<n>
```

Prevent: logrotate with `maxsize`, journald `SystemMaxUse=`, Docker log driver limits (`"log-opts": {"max-size": "50m", "max-file": "3"}` in `/etc/docker/daemon.json`), separate volumes for `/var/lib/docker` and data, and alerting at 80%/90% with time-to-full prediction.

> **Tip:** Root reserved blocks: ext4 reserves 5% for root by default, so root-owned daemons may keep working when users see 100%. On a big data volume you can reduce it with `tune2fs -m 1 /dev/...`.

10.2 Scenario: High CPU / High Load

Symptom: Latency alerts; load average 24 on an 8-vCPU host.

Step 1: Is it CPU, or something that looks like it?

```
uptime; nproc
vmstat 1 5
top -b -n 1 | head -20
```

Decide from `vmstat`:

- High `r`, low `id`, high `us`: genuine CPU saturation in user space (application work).
- High `sy`: kernel time (syscall storms, lock contention, lots of small I/O or network packets).
- High `b` and `wa`, CPU mostly idle: it's I/O, not CPU. Go to `iostat -xz 1`.
- High `st`: the hypervisor is taking CPU (burstable instance out of credits, noisy neighbour). Check CloudWatch `CPUCreditBalance`; move to a non-burstable type.

Step 2: Which process and which threads?

```
pidstat -u 1 5
top -H -p <PID>                # hottest threads
ps -eLo pid,tid,pcpu,comm --sort=-pcpu | head
```

Step 3: What is it doing?

```
sudo perf top -p <PID>          # hot functions
sudo strace -c -f -p <PID>      # syscall summary (briefly!)
# Java: jstack <PID> (convert the hot TID to hex and find it in the dump), or async-profiler
# Python: py-spy top --pid <PID>
```

Step 4: Check the context. Was there a deploy (`journalctl --since "2 hours ago" | grep -i -E 'started|deploy'`, CI history)? Traffic spike (access logs per minute)? A cron job (`systemctl list-timers`, `grep CRON /var/log/syslog`)? Runaway process left by a previous run? Crypto miner (unknown process name, high CPU, outbound connections to odd ports: `ss -tnp`)?

Fix: roll back the deploy, scale out, rate-limit, `renice` a batch job, or kill a runaway process (SIGTERM first). **Prevent:** CPU limits/cgroups for batch work, autoscaling on CPU and latency, profiling in staging, alerts on load per core.

10.3 Scenario: Service Won't Start

Symptom: After a config change and restart, `myapp` is down. `systemctl restart myapp` returns an error.

Step 1: Read what systemd and the service say.

```
systemctl status myapp -l --no-pager
journalctl -u myapp -b -n 50 --no-pager
```

```
x myapp.service - My API service
     Loaded: loaded (/etc/systemd/system/myapp.service; enabled; preset: enabled)
     Active: failed (Result: exit-code) since Mon 2026-10-12 11:02:41 UTC; 4s ago
    Process: 4410 ExecStart=/opt/myapp/bin/server --port 8080 (code=exited, status=1/FAILURE)
   Main PID: 4410 (code=exited, status=1/FAILURE)
Oct 12 11:02:41 web1 server[4410]: FATAL: listen tcp :8080: bind: address already in use
```

Step 2: Interpret the result code.

What you see	Meaning	Next check
`status=203/EXEC`	Binary not found or not executable	`ls -l` the ExecStart path; absolute path? `+x`? shebang valid?
`status=200/CHDIR`	`WorkingDirectory=` missing/inaccessible	Directory exists, user can enter it
`status=217/USER`	`User=` doesn't exist	`getent passwd myapp`
`status=226/NAMESPACE`	Sandboxing path problem	`ReadWritePaths=` points to non-existent dir

What you see	Meaning	Next check
`status=1/FAILURE`	The app itself exited with error	App's own log message (config parse error, bind failure)
`Result: timeout`	Didn't become ready in time	`Type=` mismatch (`forking` / `notify` vs app behaviour), slow dependency
`start-limit-hit`	Restarted too often	Fix cause, then `systemctl reset-failed myapp`
`Result: oom-kill`	Hit `MemoryMax=`	Raise limit or fix memory use
Active but immediately "inactive (dead)"	`Type=simple` with a daemon that forks and exits	Use foreground flag or `Type=forking`

Step 3: Reproduce by hand, as the service user.

```
sudo systemctl cat myapp                          # including drop-ins
sudo -u myapp env $(grep -v '^#' /etc/myapp/env | xargs) /opt/myapp/bin/server --port 8080
```

If it works by hand but not under systemd, the difference is environment (PATH, variables), sandboxing (`ProtectSystem` , `PrivateTmp`), limits, or SELinux.

Step 4: Check the common environmental causes.

```
sudo ss -tlnp | grep ':8080'                      # port taken? by whom?
sudo -u myapp test -r /etc/myapp/config.yaml && echo readable
namei -l /etc/myapp/config.yaml                   # traversal permissions
sudo ausearch -m AVC -ts recent                   # SELinux denials (RHEL family)
sudo journalctl -k | grep -i apparmor             # AppArmor (Ubuntu)
systemd-analyze verify /etc/systemd/system/myapp.service
df -h / /var; df -i                               # can't write PID/log/temp files?
```

In the example above, a manually started copy (from a debugging session last week, under `nohup`) is holding port 8080. `ss -tlnp` shows its PID; stop it and start the unit.

Prevent: validate configs before restart (`nginx -t` , `sshd -t` , app `--check-config` as `ExecStartPre=`), deploy via automation with health checks, never run ad-hoc copies of services outside systemd.

10.4 Scenario: Can't SSH to a Server

Symptom: `ssh web1` hangs or is rejected.

Step 1: Classify the failure from the client.

```
ssh -vvv web1
```

Client output	Layer	Likely cause
`Could not resolve hostname`	DNS	Wrong name, DNS down, `~/.ssh/config` HostName typo
Hangs at `Connecting to ... port 22`, then timeout	Network	Security group/NACL/firewall, wrong IP (instance restarted with new public IP), host down, route missing
`Connection refused`	Host	sshd not running or listening on a different port/address
`Connection closed by ... [preauth]`	sshd	`AllowUsers/AllowGroups`, `MaxStartups`, fail2ban ban, TCP wrappers
`Permission denied (publickey)`	Auth	Wrong user, wrong key, key not in `authorized_keys`, bad permissions, SELinux label
`REMOTE HOST IDENTIFICATION HAS CHANGED`	Host key	Host rebuilt (verify, then `ssh-keygen -R`) or MITM
Login succeeds then immediately closes	Shell/session	User shell is `nologin`, home missing, disk full, `ForceCommand`

Step 2: Network path.

```
dig +short web1.example.com
nc -zv -w 5 web1.example.com 22
# From a host in the same VPC:
nc -zv 10.0.1.11 22
```

In AWS also check: the instance's security group allows 22 from *your current IP* (home IPs change), NACLs allow inbound 22 and **outbound ephemeral ports** (1024–65535) for the return traffic, the subnet's route table, and whether you're using the public vs private IP correctly.

Step 3: If you can get any other access (SSM Session Manager, EC2 serial console, cloud console, another admin), inspect the server side:

```
sudo systemctl status ssh          # or sshd
sudo ss -tlnp | grep sshd
sudo sshd -t                       # config error after a recent edit?
sudo journalctl -u ssh -n 50       # or /var/log/auth.log, /var/log/secure
ls -ld ~user ~user/.ssh; ls -l ~user/.ssh/authorized_keys   # 755/700/600, owned by user
sudo fail2ban-client status sshd
df -h /                            # full disk can break logins
sudo restorecon -Rv /home/user/.ssh   # SELinux contexts
```

Server log examples and their meaning:

```
Authentication refused: bad ownership or modes for directory /home/deploy   -> fix
permissions
User deploy from 203.0.113.5 not allowed because not listed in AllowGroups  -> group
membership
```

```
Connection closed by authenticating user ubuntu 203.0.113.5 port 50122 [preauth]  -> key
rejected
```

Step 4: If there's no other access (unbootable or locked out): stop the instance, detach the root volume, attach it to a rescue instance, mount it, fix `sshd_config` / `authorized_keys` /fstab, reattach and boot. On AWS, the EC2 serial console or SSM can avoid that.

Prevent: SSM or a bastion as a second path, config changes via Ansible with `validate`, always keep a session open when editing sshd, monitor sshd availability.

10.5 Scenario: DNS Failing

Symptom: The app logs `could not resolve host: api.payments.internal` intermittently or constantly. `curl https://api.payments.internal` fails; IPs work.

Step 1: Reproduce with the system resolver and with DNS directly.

```
getent hosts api.payments.internal         # the way the app resolves
dig api.payments.internal                  # via configured nameserver
dig @10.0.0.2 api.payments.internal        # VPC resolver directly (AWS: VPC CIDR base + 2)
dig @1.1.1.1 example.com                   # can we resolve anything externally?
```

Interpret:

- `getent` fails, `dig` works: problem in `/etc/hosts`, `/etc/nsswitch.conf`, or search domains (the app uses a short name and the search list is wrong).
- `dig` returns `NXDOMAIN`: the record doesn't exist in the zone the resolver sees (private hosted zone not associated with this VPC, typo, wrong environment).
- `dig` returns `SERVFAIL`: the resolver couldn't get an answer (upstream down, DNSSEC failure, forwarding rule broken).
- `dig` times out (`connection timed out; no servers could be reached`): can't reach the resolver at all (firewall blocking UDP/TCP 53, resolver down, wrong IP in config).

Step 2: Inspect resolver configuration.

```
cat /etc/resolv.conf; ls -l /etc/resolv.conf
resolvectl status
grep hosts /etc/nsswitch.conf
grep -v '^#' /etc/hosts
```

Step 3: Check the systemd-resolved layer (Ubuntu): the stub at `127.0.0.53` must be running, and the per-link upstream servers must be correct.

```
systemctl status systemd-resolved
resolvectl query api.payments.internal
```

```
sudo resolvectl flush-caches
journalctl -u systemd-resolved --since "30 min ago"
```

Step 4: Intermittent failures point to: one of several nameservers being broken (resolvers are tried in order with timeouts), UDP packet loss, conntrack table full on busy nodes (`dmesg | grep conntrack`), AWS per-ENI DNS packet limits (1024 packets/s to the VPC resolver) on very chatty hosts, or Kubernetes `ndots:5` causing many extra lookups per name. Watch it on the wire:

```
sudo tcpdump -ni any port 53 -c 50
```

Fix: correct the netplan/nmcli DNS settings or private zone association, restart the resolver, and use FQDNs with a trailing dot in latency-sensitive configs. **Prevent:** local caching (systemd-resolved, NodeLocal DNSCache in Kubernetes), monitoring DNS resolution latency and errors as a first-class SLI.

> **Note:** "It's always DNS" is a joke because it's frequently true. Rule it out early with `getent hosts` and `dig`.

10.6 Scenario: Server Doesn't Boot After Changes (Bonus)

Symptom: After a reboot, the instance fails health checks; console shows `You are in emergency mode` or `Give root password for maintenance`.

Walkthrough: view the serial console / system log. `emergency mode` almost always means a failed mount: a typo or a missing device in `/etc/fstab` without `nofail`. From the emergency shell (or a rescue instance with the root volume attached):

```
journalctl -xb | grep -iE 'fail|mount|timed out'
systemctl --failed
mount -o remount,rw /
vi /etc/fstab                # fix UUID or add nofail
findmnt --verify
systemctl daemon-reload && mount -a
reboot
```

Other boot failures: kernel panic after an update (choose the previous kernel in GRUB, then fix the default with `grubby --set-default` or `grub-set-default`), missing initramfs drivers after moving to a new instance type (rebuild with `dracut -f` / `update-initramfs -u`), SELinux relabel needed (`touch /.autorelabel`), full `/boot` preventing kernel installation.

10.7 Lab: Break-Fix Game

Goal: practise the scenarios under realistic conditions. Work in pairs: one person breaks a VM (secretly), the other fixes it using only the methods above, narrating each step. Swap roles. Snapshot the VM first.

Suggested breakages (the breaker picks two at a time):

1. Fill the root filesystem with a hidden, open, deleted file: `sudo bash -c 'exec 3>/var/tmp/x; dd if=/dev/zero bs=1M count=<size> >&3; rm /var/tmp/x; sleep infinity' &`
2. Exhaust inodes in `/var/tmp` with a loop of `touch`.
3. Start a CPU hog with an innocent-looking name: `bash -c 'exec -a logsync bash -c "while :; do :; done"' &`.
4. Change `ExecStart` in a unit to a wrong path, or set `User=` to a non-existent user, and restart it.
5. Bind something to a service's port so it can't start: `python3 -m http.server 80`.
6. Add `DenyUsers <you>` in a drop-in in `/etc/ssh/sshd_config.d/` and reload (keep a console!).
7. `chmod 777 ~/.ssh` for the login user.
8. Point `/etc/resolv.conf` (or netplan nameservers) at an unreachable IP.
9. Add a bogus line to `/etc/fstab` *without* `nofail` and reboot (needs console access to recover).
10. On Rocky: move a web page into the docroot with `mv` so it has the wrong SELinux label.

For each fix, the fixer writes a three-line postmortem: symptom, root cause and evidence, fix and prevention.

10.8 Interview Questions and Answers

Beginner

1. What is the difference between Linux the kernel and a Linux distribution? The kernel manages hardware, processes, memory and system calls. A distribution bundles the kernel with userland tools, a C library, init system, package manager, default configuration and a support lifecycle (Ubuntu, RHEL, Alpine…).

2. What does `/etc` contain, and `/var`? `/etc` holds system-wide configuration files. `/var` holds variable data that changes at runtime: logs (`/var/log`), service state (`/var/lib`), caches and spools.

3. What's the difference between an absolute and a relative path? Absolute starts from `/` and is the same from anywhere; relative is resolved from the current working directory.

4. Explain `chmod 754 file`. Owner rwx (7), group r-x (5), others r-- (4).

5. What's the difference between a hard link and a symbolic link? A hard link is another directory entry for the same inode (same filesystem, can't link directories, survives deletion of the original name). A symlink is a separate file containing a path; it can cross filesystems and breaks if the target is removed.

6. How do you find which process is listening on port 8080? `sudo ss -tlnp | grep :8080` or `sudo lsof -i :8080`.

7. What do `>`, `>>`, `2>&1` and `|` do? Redirect stdout to a file (truncate), append, send stderr to wherever stdout currently goes, and pipe stdout into the next command's stdin.

8. How do you view the last 100 lines of a log and keep following it? `tail -n 100 -f file` (`-F` to survive rotation), or `journalctl -u svc -n 100 -f` for systemd services.

9. What's the difference between `apt update` and `apt upgrade`? `update` refreshes the package index from repositories; `upgrade` installs newer versions of installed packages.

10. How do you check disk space and directory sizes? `df -h` for filesystem usage, `df -i` for inodes, `du -sh dir` or `du -h --max-depth=1 | sort -h` for directories.

11. What is the root user, and why use sudo instead? UID 0, which bypasses permission checks. sudo grants scoped privileges per command, requires the user's own credentials, and logs who did what.

12. What does `kill -9` do and why avoid it? Sends SIGKILL, which can't be caught: the process dies immediately without cleanup (flushing data, removing locks, closing connections gracefully). Try SIGTERM first.

13. How do you make a script executable and run it? `chmod +x script.sh` then `./script.sh`; the shebang line picks the interpreter.

14. What is an environment variable, and how do you set one for child processes? A key-value string in a process's environment, inherited by children. `export NAME=value`.

15. How do you search for a string recursively in a directory? `grep -rn "string" dir/` (`-i` for case-insensitive, `-l` for filenames only).

Intermediate

16. Describe the Linux boot process. Firmware (UEFI/BIOS) → bootloader (GRUB) loads the kernel and initramfs and passes the command line → kernel initialises hardware and unpacks initramfs → initramfs loads drivers and mounts the real root filesystem → `switch_root` to systemd as PID 1 → systemd starts units to reach `default.target`.

17. What is the initramfs for? It's a temporary root filesystem in RAM that contains the drivers and tools (NVMe, LVM, RAID, LUKS) needed to find and mount the real root filesystem, which the kernel can't do alone if those drivers are modules.

18. Explain SUID, SGID and the sticky bit. SUID on an executable runs it with the owner's UID (e.g. `passwd`). SGID on an executable runs with the file's group; on a directory, new files inherit the directory's group. Sticky on a directory (e.g. `/tmp`) lets only a file's owner delete or rename it.

19. What is umask? A mask of permission bits removed from newly created files and directories. With `022`, files (requested 666) become 644 and directories (777) become 755.

20. `systemctl enable` vs `systemctl start`? `start` starts the unit now; `enable` creates symlinks so it starts at boot. `enable --now` does both.

21. How do you change a setting in a vendor-provided systemd unit? `systemctl edit unit` to create a drop-in in `/etc/systemd/system/unit.d/override.conf`; clear list settings like `ExecStart=` before redefining them. Never edit `/usr/lib/systemd/system`. systemctl edit reloads automatically; after manual edits run `systemctl daemon-reload`.

22. What is load average, and what does a load of 8 mean? Exponentially damped 1/5/15-minute averages of tasks runnable or in uninterruptible (D) sleep. On 8 cores, 8 means fully utilised; on 2 cores it means heavy saturation. With idle CPUs, high load indicates I/O-blocked tasks.

23. `free` shows almost no free memory. Is the server in trouble? Not necessarily. Linux uses spare RAM as page cache. Look at the `available` column, swap activity (`vmstat si/so`) and memory PSI.

24. What is a zombie process and how do you get rid of it? A process that has exited but whose parent hasn't called `wait()` to collect its status. It uses no resources besides a PID entry. You can't kill it; fix or restart the parent so it reaps (or so the zombie is re-parented to PID 1 and reaped).

25. `df` shows the disk 100% full but `du` shows much less. Why? Most often a deleted file still held open by a process (`lsof +L1`); the space is freed when the process closes it or restarts. Alternatively, files hidden under a mount point, or the reserved root blocks.

26. How do you extend an LVM logical volume and its filesystem? Add space to the VG if needed (`pvcreate`, `vgextend`), then `lvextend -r -L +10G /dev/vg/lv`, which also grows the filesystem (`resize2fs` for ext4, `xfs_growfs` for XFS).

27. Why use UUIDs and `nofail` in `/etc/fstab`? Device names like `/dev/sdb` aren't stable across boots. `nofail` prevents a missing secondary volume from dropping the system into emergency mode.

28. What's the difference between "connection refused" and a timeout? Refused: the host answered with a TCP RST because nothing listens on that address/port (or a REJECT rule). Timeout: packets are dropped somewhere (firewall, security group, routing, host down).

29. How does `dig` differ from `getent hosts`? `dig` sends DNS queries directly to a nameserver. `getent hosts` uses the system's NSS resolution (including `/etc/hosts`, search domains and nsswitch order), which is what applications use.

30. How do you securely reach a database in a private subnet from your laptop? SSH local port forwarding through a bastion (`ssh -N -L 15432:db.internal:5432 bastion`), or SSM Session Manager port forwarding; then connect to `localhost:15432`.

31. Why is `ProxyJump` preferred over agent forwarding? With agent forwarding, root on the intermediate host can use your agent socket to authenticate as you. ProxyJump tunnels the connection end to end; the bastion only relays encrypted TCP.

32. What does `set -euo pipefail` do? Exit on command failure (`-e`), treat unset variables as errors (`-u`), and make a pipeline fail if any command in it fails (`pipefail`), rather than only the last.

33. How do you prevent two instances of a cron job running at the same time? `flock -n /run/lock/job.lock command`, or use a systemd timer, which won't start the service again while it's still active.

34. A service hits "Too many open files" even after editing limits.conf. Why? systemd services don't go through PAM, so `limits.conf` doesn't apply. Set `LimitNOFILE=` in the unit (drop-in), restart, and check `/proc/<pid>/limits`.

35. cron vs systemd timers? Timers log to the journal, avoid overlapping runs, support `Persistent=` catch-up and randomized delays, can depend on other units and be sandboxed. cron is simpler and universal but has a minimal environment, special `%` handling and output that's easy to lose.

Advanced

36. What are namespaces and cgroups, and how do they make containers? Namespaces isolate what a process can see: PIDs, network stack, mounts, hostname, IPC, users, cgroups. cgroups (v2) limit and account resource use: CPU, memory, I/O, PIDs. A container is a process in its own set of namespaces, constrained by a cgroup, with an overlayfs root filesystem, reduced capabilities and a seccomp filter, all sharing the host kernel.

37. A container's latency spikes, but average CPU usage is well below its limit. What's going on? CFS quota throttling: the limit is enforced per 100 ms period, so bursty multi-threaded workloads exhaust the quota early and are paused for the rest of the period. Check `nr_throttled` / `throttled_usec` in the cgroup's `cpu.stat`; raise or remove the CPU limit or reduce parallelism.

38. How does the OOM killer choose a victim, and how do you influence it? When memory can't be reclaimed, the kernel scores processes mainly by memory footprint (`/proc/<pid>/oom_score`), adjusted by `oom_score_adj` (-1000 to 1000). Within a cgroup with `memory.max`, only that group's processes are candidates. Influence it with `OOMScoreAdjust=` in units, memory limits per service, and systemd-oomd.

39. Explain how you would troubleshoot high I/O wait. Confirm with `vmstat` (`b`, `wa`) and `iostat -xz` (`await`, `aqu-sz`, `%util`) which device is saturated. Identify the process with `iotop` / `pidstat -d`. Check for swapping, log storms, backups or compaction jobs, and cloud volume limits (IOPS/throughput caps, burst balance). Fix by moving workloads, tuning the app, provisioning more IOPS, or separating volumes.

40. What is SELinux, and how do you fix "permission denied" that only happens with SELinux enforcing? A mandatory access control system that labels processes and files and enforces a policy beyond Unix permissions. Find the AVC (`ausearch -m AVC -ts recent`), then fix the cause: relabel files (`semanage fcontext` + `restorecon`), enable a boolean (`setsebool -P`), or label a port (`semanage port -a`). Keep it enforcing.

41. What happens when you type `curl https://example.com` and press Enter? The shell parses and expands the line, finds `curl` in PATH, forks and execs it. curl resolves the name via NSS (`/etc/hosts`, then DNS through the configured resolver), opens a TCP socket (three-way handshake via the kernel's routing table and default gateway, through netfilter rules), performs a TLS handshake (SNI, certificate validation against the CA bundle), sends an HTTP request, receives and prints the response, and exits; the shell collects its exit status.

42. How would you harden a new Linux server? Patch and enable automatic security updates; create named users with sudo via groups; key-only SSH, no root login, restricted sources; default-deny firewall; remove unneeded packages and services; SELinux/AppArmor enforcing; auditd rules and persistent, shipped logs; time sync; file integrity monitoring; scan against a CIS benchmark; capture all of it in Ansible or a golden image.

43. How do you debug a process that is hanging and not logging anything? Check its state (`ps -o stat` , `/proc/<pid>/status`). If `D` , look at `/proc/<pid>/stack` and storage/NFS health. Otherwise `strace -p` to see the blocking syscall (e.g. `futex` for lock wait, `read` on a socket, `connect` timing out), `lsof -p` for which file or socket that FD is, and language-specific stack dumps (jstack, py-spy dump, gdb).

44. What is the difference between `TIME-WAIT` and `CLOSE-WAIT` , and which one indicates an application bug? TIME-WAIT is normal on the side that closed first, ensuring late packets don't corrupt new connections; many of them on a busy client is expected. CLOSE-WAIT means the peer closed but the local application never closed its socket; a growing number indicates a socket leak in the application.

45. How do you make a configuration change to 300 servers safely? Encode it in Ansible (or your config tool), validate files before replacing (`validate:`), test with `--check --diff` and in staging, roll out with `serial` batches and health checks so a failure stops the rollout, keep it idempotent and in version control with code review, and have a rollback path. In immutable setups, bake a new image and replace instances gradually.

46. Explain the difference between `Wants=` , `Requires=` and `After=` . `Wants=` and `Requires=` declare dependencies (soft and hard: with `Requires=` , if the dependency fails or stops, so does this unit). `After=` only controls ordering. They are independent; a typical pairing is `Wants=` plus `After=` .

47. How does `sudo` work given that it needs root privileges? The `sudo` binary is owned by root with the SUID bit, so it runs with effective UID 0, checks the sudoers policy and authenticates the user, then executes the requested command as the target user and logs the event.

10.9 Command Cheat Sheet

System information

Task	Command
OS and version	`cat /etc/os-release` , `hostnamectl`
Kernel	`uname -r` , `cat /proc/cmdline`
CPU / memory	`nproc` , `lscpu` , `free -h` , `cat /proc/meminfo`
Uptime and load	`uptime` , `cat /proc/loadavg`
Boot timing	`systemd-analyze blame` , `systemd-analyze critical-chain`
Hardware	`lsblk` , `lspci` , `lsusb` , `dmidecode -t system`
Kernel messages	`dmesg -T` , `journalctl -k`

Files and text

Task	Command
Find big files	`find / -xdev -type f -size +500M -exec ls -lh {} +`

Task	Command
Find recent changes	`find /etc -mmin -60 -type f`
Biggest directories	`du -xh --max-depth=1 / | sort -h`
Search content	`grep -rn --include='*.conf' pattern /etc`
Count by field	`awk '{print $1}' f | sort | uniq -c | sort -rn | head`
Replace in place	`sed -i 's/old/new/g' file`
JSON field	`jq -r '.items[].metadata.name'`
Compare files	`diff -u a b`, `diff <(cmd1) <(cmd2)`
Archive	`tar -czf out.tgz dir`, `tar -xzf out.tgz -C dest`
Sync	`rsync -avz --delete src/ host:/dest/`

Users and permissions

Task	Command
Who am I / groups	`id`, `whoami`, `groups`
Add user to group	`usermod -aG group user`
Service account	`useradd -r -s /usr/sbin/nologin svc`
Edit sudoers	`visudo -f /etc/sudoers.d/name`
Permissions along path	`namei -l /path/to/file`
ACL	`setfacl -m u:bob:rw f`, `getfacl f`
SUID audit	`find / -xdev -perm -4000 -type f`
Capabilities	`getcap file`, `setcap cap_net_bind_service=+ep file`

Processes

Task	Command
Top consumers	`ps -eo pid,user,%cpu,%mem,cmd --sort=-%cpu | head`
Process tree	`pstree -p`, `ps -ef --forest`
Find by name	`pgrep -a name`
Signals	`kill -TERM pid`, `kill -HUP pid`, `pkill -f pattern`
Open files / ports	`lsof -p pid`, `lsof -i :443`, `lsof +L1`
Syscalls	`strace -f -p pid -e trace=network`

Task	Command
Limits	`cat /proc/pid/limits`, `ulimit -n`
Priority	`nice -n 10 cmd`, `renice 5 -p pid`, `ionice -c3 -p pid`
Survive logout	`tmux new -s name`, `nohup cmd &`

systemd and logs

Task	Command
Status	`systemctl status svc`
Start at boot and now	`systemctl enable --now svc`
Failed units	`systemctl --failed`
Show unit with drop-ins	`systemctl cat svc`
Override	`systemctl edit svc`
Reload unit files	`systemctl daemon-reload`
Clear failure state	`systemctl reset-failed svc`
Timers	`systemctl list-timers`
Service logs	`journalctl -u svc -b -n 100 -f`
Errors this boot	`journalctl -p err -b`
Previous boot	`journalctl -b -1`
Time window	`journalctl --since "1 hour ago"`
Journal size	`journalctl --disk-usage`, `journalctl --vacuum-size=500M`
Test logrotate	`logrotate -d /etc/logrotate.d/app`

Packages

Task	Debian/Ubuntu	RHEL/Rocky/AL2023	Alpine
Refresh	`apt update`	`dnf makecache`	`apk update`
Install	`apt install pkg`	`dnf install pkg`	`apk add pkg`
Upgrade all	`apt upgrade`	`dnf upgrade`	`apk upgrade`
Remove	`apt remove pkg`	`dnf remove pkg`	`apk del pkg`
Search	`apt search x`	`dnf search x`	`apk search x`
Files in package	`dpkg -L pkg`	`rpm -ql pkg`	`apk info -L pkg`

Task	Debian/Ubuntu	RHEL/Rocky/AL2023	Alpine
Owner of file	`dpkg -S /path`	`rpm -qf /path`	`apk info --who-owns /path`
Pin version	`apt-mark hold pkg`	`dnf versionlock add pkg`	`apk add pkg=1.2.3-r0`
Security updates	`unattended-upgrades`	`dnf upgrade --security`	`apk upgrade`

Storage

Task	Command
Devices and fs	`lsblk -f`, `blkid`
Usage / inodes	`df -hT`, `df -i`
Partition	`parted -s /dev/sdb mklabel gpt mkpart primary 1MiB 100%`
Format	`mkfs.ext4 -L data /dev/sdb1`, `mkfs.xfs /dev/sdb1`
Mount / check fstab	`mount /dev/sdb1 /data`, `findmnt --verify`, `mount -a`
Who uses mount	`lsof +f -- /data`, `fuser -vm /data`
LVM overview	`pvs`, `vgs`, `lvs`
Grow LV + fs	`lvextend -r -l +100%FREE /dev/vg/lv`
Grow cloud disk	`growpart /dev/nvme0n1 1 && resize2fs /dev/nvme0n1p1` (or `xfs_growfs /`)
RAID status	`cat /proc/mdstat`, `mdadm --detail /dev/md0`
Disk I/O	`iostat -xz 1`, `iotop -oPa`

Networking

Task	Command
Addresses / links	`ip -br addr`, `ip -br link`
Routes	`ip route`, `ip route get 8.8.8.8`
Neighbours	`ip neigh`
Listening sockets	`ss -tulpn`
Connections by state	`ss -tan state established`
DNS	`dig +short name`, `dig @server name`, `getent hosts name`, `resolvectl status`

Task	Command
Port test	`nc -zv host 443`, `timeout 3 bash -c ‘`